Metroidvania Developer System: the objects guide

Every MDS object you use to make a game, from the ones every game needs to the ones few games touch. For each one: what it is, how to add it in the Godot editor, how it's used in a game, and every property the **Inspector** shows (with its default), grouped as the Inspector groups them.

> Written from the plugin's source on `main` (commit `1a03c71`, MDS 3.4.0). Property descriptions come from the code's own doc comments, which Godot also shows in **Help > Search Help (F1)** and as Inspector tooltips. Where the code has none, the guide adds one for the important objects, and marks the rest *(not documented)*.

Contents

Working with MDS in the Godot editor
Importance ranking
1. Foundations: every MDS game has these
2. Progression: abilities, items and the things that gate the map
3. Combat
4. Enemies and bosses
5. Building rooms: terrain and scenery
6. NPCs, dialogue, shops and equipment
7. Game UI (the Gameplay Kit's screens)
8. Presentation: camera moments, music and level views
9. Environment effects
10. The player's insides: state machine, input buffer and states
Appendix A: editor-only objects (not for your game)
Appendix B: code helpers (no Inspector, called from scripts)

Working with MDS in the Godot editor

1. **Enable the plugin:** Project > Project Settings > Plugins > *Metroidvania Developer System* > Enabled. It adds the **MetDevSys** dock, the input actions (`move_*`, `jump`, `attack`, `dash`, `special`, `interact`, `map`, `pause`...) and the collision layer names (1 world, 3 player, 4 enemy, 5–8 hitboxes and hurtboxes, 9 interact).
2. **Draw the world:** in the MetDevSys dock (non-linear mode), draw rooms and areas on the world map, add gates, connect them, and give gates `requires` (`dash`, `item:red_key`, `flag:x`, `boss:Name`). This is the `*.mdsworld.json` file (`MDSWorld`).
3. **Make each room's scene:** right-click a room > create its scene; its **Inspect** tab > **Write to scene** adds the `MDSGate` nodes. Paint it in the **Room view** (tiles, or freeform terrain and stamps from an asset pack; **Convert to freeform** and **Decorate freeform** do most of the work), and place objects from the **Effects** tool's *Gameplay Objects* menu, or with the Scene dock as below.
4. **Make the game scene:** Scenes > **Create game scene...** with *Include gameplay kit* builds `MDSWorldGame` with the player, the kit's hub, camera, music, minimap and every UI screen. Open it and press **F6**.
5. **Tune in the Inspector:** select a node and edit its properties. Objects that hold data (`MDSMovementConfig`, `MDSAbilityData`, `MDSItemData`, `MDSEnemyData`...) are **resources**: create them as `.tres` files and drag them onto the property that takes them, so many nodes share one.
6. **Check progression:** the MetDevSys dock's **Progress** tab shows which rooms each ability, item or flag opens, and **Issues** reports problems (an ability pickup without data, an item nothing grants, an arena without a boss).
7. **Look at the example:** `examples/gameplay_demo/` is a 12-room game using most objects below; `build_gameplay_demo.gd` builds every room in code.

Reading the tables: *Type* is the property's type (empty when Godot infers it from the default); a bold row is an Inspector group (a foldable section). Enum defaults read `Enum.VALUE`; the Inspector shows them as a drop-down.

Importance ranking

Rank	Object	Group	In one line
1	`MDSWorldGame`	Foundations	Game runtime for non-linear worlds: the counterpart of MetSys' MetSysGame, with no MetSys dependency
2	`MDSGate`	Foundations	A room transition, like Hollow Knight's TransitionPoint
3	`MDSPlayer`	Foundations	The Gameplay Kit's player: a CharacterBody2D with components (composition), in the `player` group so `MDSGate`, `MDSBench`, `MDSWorldMapView`,...
4	`MDSGameplay`	Foundations	The Gameplay Kit's hub: a child of the `MDSWorldGame` root that owns everything the kit adds to a game's progress (items, currency, equipment, story...
5	`MDSRoomCamera`	Foundations	Camera for `MDSWorldGame` rooms of any shape, driving a `Camera2D` or, when the Phantom Camera addon is installed, `PhantomCamera2D` nodes
6	`MDSMovementConfig`	Foundations	How the player moves (`MDSPlayer`): one feel profile, in px and seconds
7	`MDSCombatStats`	Foundations	Health, mana and toughness of the player, an enemy or a boss (`MDSHealth`).
8	`MDSBench`	Foundations	A bench to rest on: the save point of a room, after Hollow Knight: Silksong's benches
9	`MDSGateBarrier`	Foundations	A physical gate that stays shut until a condition is met, then opens visibly and stays open (also after saving and loading)
10	`MDSWorldMapView`	Foundations	In-game map for non-linear worlds, drawn from the same .mdsworld.json the editor uses
11	`MDSWorld`	Foundations	Non-linear world map: rooms (scenes) placed freely in world pixels and connected by Hollow Knight-style named gates
12	`MDSAbilityPickup`	Progression	An ability to pick up (the dash, the double jump...): touching it (or pressing interact at it, without `auto_collect`) grants `ability`...
13	`MDSAbilityData`	Progression	One unlockable ability of the Gameplay Kit: `id` is what `MDSWorldGame.grant_ability` stores and what gates, barriers and objectives require (`dash`...
14	`MDSPickup`	Progression	An item to pick up: a key, a consumable, a charm, a collectible (`item`, `amount`)
15	`MDSItemData`	Progression	Any pickup that is not an ability: keys, consumables, equipment (`MDSEquipmentData`), upgrades and collectibles
16	`MDSBreakable`	Progression	A wall, floor or block that breaks: solid terrain (layer 1) with an `MDSHurtbox`
17	`MDSSwitch`	Progression	A lever, a pressure plate or a crystal to strike: it sets the story flag `flag` (`MDSGameplay.set_flag`), so barriers and gates that require...
18	`MDSHealthUpgrade`	Progression	A health fragment: every `fragments_per_upgrade` of them raise maximum health by one (`MDSGameplay.raise_max_health`)
19	`MDSManaUpgrade`	Progression	A mana fragment: every `fragments_per_upgrade` of them raise maximum mana by `mana_per_upgrade`
20	`MDSSlotUpgrade`	Progression	A notch for equipment: one more slot (`MDSGameplay.equip_slots`) for charms
21	`MDSCurrencyPickup`	Progression	Currency (Hollow Knight's geo): `amount` of it, pulled to the player who comes near, never persistent
22	`MDSCheckpoint`	Progression	A checkpoint in a room of hazards: touched, it is where spikes and pits (`MDSHazard` RESPAWN, lava) send the player back to, in this room, instead of...
23	`MDSHazard`	Progression	Spikes, thorns or a pit: an area of `size` (centred on the node) with an `MDSHitbox` of the HAZARD team
24	`MDSClimbable`	Progression	A ladder or vines: with the `climb` ability, up or down while in it climbs
25	`MDSSwimZone`	Progression	Water the player is in: with the `swim` ability it swims freely, without it it sinks slowly and moves slower

Rank	Object	Group	In one line
26	`MDSGrapplePoint`	Progression	A ring the player's grapple hooks (`grapple` ability): the nearest one in reach and in sight when grapple is pressed pulls the player to it
27	`MDSLoreTablet`	Progression	An ancient stone tablet that can be inspected by the player
28	`MDSCorpse`	Progression	Where the player last died, holding the currency it carried (death penalty CORPSE, like Hollow Knight's shade)
29	`MDSAttackController`	Combat	The player's attacks, a child of `MDSPlayer`: they run alongside its movement (it can attack while running and jumping), each by its `MDSAttackData`...
30	`MDSAttackData`	Combat	One attack: its damage, its timing in frames at 60 fps (startup, then the hitbox is active, then recovery) and what it can follow into...
31	`MDSHealth`	Combat	Health of the player, an enemy, a boss or a breakable: a child of it
32	`MDSHitbox`	Combat	What hits: an Area2D with a collision shape that, while `active`, hits the `MDSHurtbox`es of the other team it overlaps with its `attack`
33	`MDSHurtbox`	Combat	Where something can be hit: an Area2D with a collision shape, on the `player_hurtbox` layer (team PLAYER) or the `enemy_hurtbox` layer (ENEMY,...
34	`MDSKnockback`	Combat	Pushes its `body` back when it is hit, and stuns it for `hitstun_time`: the player's Hurt state and enemies wait for `is_stunned` to end
35	`MDSProjectile`	Combat	A shot: spells and sub-weapons of the player (fired on `special`, costing mana), a turret's bolt
36	`MDSDamageNumber`	Combat	A hit's damage, floating up from the body it hurt and fading
37	`MDSEnemy`	Enemies and bosses	An enemy: a CharacterBody2D in the `enemy` group (the scanner counts it, dark rooms light it, the camera director watches it) on the `enemy` layer,...
38	`MDSEnemyData`	Enemies and bosses	An enemy's definition (`MDSEnemy`): its stats, what it drops, its currency and XP, and its bestiary entry (the pause menu's Bestiary lists the...
39	`MDSDropTable`	Enemies and bosses	Loot: each entry {item: MDSItemData (or an item id), chance: 0 to 1, min, max} is rolled on its own when an enemy dies (`roll`).
40	`MDSWalker`	Enemies and bosses	An enemy that walks: Patrol (to and fro, turning at walls and ledges), Turn, Chase (toward the player it sees, stopping at ledges), Attack (a lunge...
41	`MDSFlyer`	Enemies and bosses	An enemy that flies: Hover (bobbing about where it started), Swoop (a dive at where the player was), Return (back home), Hurt, Dead
42	`MDSCharger`	Enemies and bosses	An enemy that charges: Idle (until it sees the player), Windup (it shakes, telegraphing, for `windup_time`), Charge (straight ahead at...
43	`MDSTurret`	Enemies and bosses	An enemy fixed in place that shoots: Idle (until it sees the player), Aim (turning toward it for `aim_time`), Fire (an `MDSProjectile` at it, then...
44	`MDSAmbusher`	Enemies and bosses	An enemy that lies in wait: Hidden (in the ground, harmless and unhittable), Reveal (it bursts out when the player comes within `reveal_range`),...
45	`MDSPlayerDetector`	Enemies and bosses	An enemy's eyes: a circle of `sight_range` around it, and a line of sight through terrain (layer 1)
46	`MDSSpawner`	Enemies and bosses	Spawns waves of enemies (an arena's adds, a gauntlet): wave i brings `waves enemies, taken in turn from` enemies `(scenes of` MDSEnemy`; empty:`...
47	`MDSBoss`	Enemies and bosses	A boss: an `MDSEnemy` in the `boss` group, recorded by `boss_name` when defeated (`respawn` is always NEVER, and `mds_boss_name` metadata names it,...
48	`MDSBossPhase`	Enemies and bosses	A phase of a boss (`MDSBoss`): it starts when the boss's health falls to `health_threshold` of its maximum (1: from the start), plays...
49	`MDSBossArena`	Enemies and bosses	Where a boss fight happens: when the player walks into it (`size`, centred on the node), its `doors` close, the boss music starts...
50	`MDSArenaDoor`	Enemies and bosses	A door of a boss arena (`MDSBossArena`): open (no collision, its bars up) until the fight starts, shut during it, open again after
51	`MDSEnemyState`	Enemies and bosses	A state of an `MDSEnemy`'s state machine (patrol, chase, swoop, fire...)

Rank	Object	Group	In one line
52	`MDSFreeform`	Building rooms	Freeform terrain: a closed outline drawn with curves instead of tiles, for organic caves (rounded ledges, bulging moss walls, arches)
53	`MDSFreeformStyle`	Building rooms	Look of an `MDSFreeform` shape: a repeating fill texture, textured strips along its top edges (moss, grass...) and undersides (drips, roots), an...
54	`MDSStampSet`	Building rooms	Large sprites placed freely (not on the tile grid): moss clumps, leaf clusters, hanging moss, background bubbles, foreground silhouettes..
55	`MDSDrawing`	Building rooms	A picture placed in a room as it is drawn: what the Room view's Trace drawing makes with "The drawing itself", or any image
56	`MDSBackdropView`	Building rooms	Draws an `MDSBackdrop` behind a room: the sky over the whole view and the planes behind the room are drawn off screen in a `SubViewport` at...
57	`MDSBackdrop`	Building rooms	The distance behind the rooms: a sky gradient and planes of art at increasing depths (`MDSBackdropLayer`), drawn by an `MDSBackdropView`
58	`MDSBackdropLayer`	Building rooms	One plane of an `MDSBackdrop`: art at a depth, scrolling at a fraction of the camera's speed, drawn smaller, hazier and greyer the farther back it is.
59	`MDSDepth25D`	Building rooms	2.5D for flat rooms, made only of their own 2D art
60	`MDSDepthStyle`	Building rooms	How `MDSDepth25D` fakes depth: how far walls reach back, how faces are lit, how dark the view gets away from the player, shadows and floor paving
61	`MDSNPC`	NPCs, dialogue, shops and equipment	A character to talk to
62	`MDSDialogueTrigger`	NPCs, dialogue, shops and equipment	A conversation with anything: put it on (or beside) a door, a chest, a statue, a sign, a lever, a sleeping boss, and the player talks to that
63	`MDSDialogueData`	NPCs, dialogue, shops and equipment	A conversation: its lines in order (`MDSDialogueLine`), shown by `MDSDialogueBox`.
64	`MDSDialogueLine`	NPCs, dialogue, shops and equipment	One line of a conversation (`MDSDialogueData`)
65	`MDSShop`	NPCs, dialogue, shops and equipment	A shopkeeper: talking to it says its `dialogue` (a greeting, if any), then opens the game's shop screen (the node in group `mds_shop_screen`, an...
66	`MDSShopInventory`	NPCs, dialogue, shops and equipment	What a shop (`MDSShop`) sells: each entry {item: MDSItemData, price (0: the item's value), stock (-1: endless), condition (an MDS condition string;...
67	`MDSEquipmentData`	NPCs, dialogue, shops and equipment	A charm: equipped while resting on a bench (`MDSGameplay.equip`), it costs `slot_cost` of the player's `MDSGameplay.equip_slots`, changes stats while...
68	`MDSItemEffect`	NPCs, dialogue, shops and equipment	What using an item does (`MDSGameplay.use_item`): override `apply`
69	`MDSHealEffect`	NPCs, dialogue, shops and equipment	Restores `amount` health (`MDSHealth.heal`).
70	`MDSMaxHealthEffect`	NPCs, dialogue, shops and equipment	Raises maximum health by `amount` (kept in the save, see `MDSGameplay.max_health`) and fills it.
71	`MDSManaEffect`	NPCs, dialogue, shops and equipment	Restores `amount` mana, and raises maximum mana by `max_up`.
72	`MDSBuffEffect`	NPCs, dialogue, shops and equipment	For `duration` seconds, adds `amount` to a stat of the player: `damage`, `defense`, or a field of its `MDSMovementConfig` (`max_run_speed`...)

Rank	Object	Group	In one line
73	`MDSLevelCurve`	NPCs, dialogue, shops and equipment	Experience levels, with the LEVELS progression setting: reaching level n takes `base_xp * growth^(n - 2)` more XP than level n - 1, and each level...
74	`MDSTextReveal`	NPCs, dialogue, shops and equipment	How a line of dialogue comes onto the screen in an `MDSDialogueBox`
75	`MDSTextRevealLetters`	NPCs, dialogue, shops and equipment	Types the line out letter by letter (a typewriter), with a beat after punctuation.
76	`MDSTextRevealWords`	NPCs, dialogue, shops and equipment	Shows the line a word at a time, with a beat after the words that end in punctuation
77	`MDSHud`	Game UI (the Gameplay Kit's screens)	The heads-up display, in the game's UI layer: the player's health as masks, its mana as a bar, its currency (and level, with LEVELS progression) top...
78	`MDSToast`	Game UI (the Gameplay Kit's screens)	Short notices at the top of the screen, one after another: what the kit announces (`MDSGameplay.notice`: an item picked up, "Requires Dash" when a...
79	`MDSAbilityPopup`	Game UI (the Gameplay Kit's screens)	Shown when the player gains an ability (`MDSGameplay` calls `show_ability`): its icon, its name, what it does and the key or button of its input...
80	`MDSPauseMenu`	Game UI (the Gameplay Kit's screens)	The pause menu: `pause` (Esc, Start) opens it and closes it, `map` (Tab, Back) opens it on the map
81	`MDSMapPinEditor`	Game UI (the Gameplay Kit's screens)	A box to write a pin of the player's own on the map: its title, a short description and its kind (`kinds`: a flag, a note, a danger...)
82	`MDSDialogueBox`	Game UI (the Gameplay Kit's screens)	Shows a conversation (`MDSDialogueData`) at the bottom of the screen: the speaker's name and portrait, the text coming in, and the line's choices as...
83	`MDSShopScreen`	Game UI (the Gameplay Kit's screens)	The screen an `MDSShop` opens (it finds it in group `mds_shop_screen`): what is on sale with prices and stock, the item's description, the currency...
84	`MDSInventoryScreen`	Game UI (the Gameplay Kit's screens)	The pause menu's Inventory tab: the items carried by category (keys, consumables, upgrades, collectibles; equipment has its own tab), with the...
85	`MDSEquipmentScreen`	Game UI (the Gameplay Kit's screens)	The pause menu's Equipment tab: the charms owned with their slot costs, the ones worn marked, the slots used, and the focused charm's effects
86	`MDSOptionsMenu`	Game UI (the Gameplay Kit's screens)	The options (the pause menu's Options tab, and the title screen's): a volume per audio bus, the window, accessibility (game speed 50 to 100%, assist...
87	`MDSTitleScreen`	Game UI (the Gameplay Kit's screens)	The title and save slot selection screen: Continue or New game for 3 slots, via `MDSWorldGame.has_save_game`, `MDSWorldGame.read_save_game`,...
88	`MDSGameplayScreen`	Game UI (the Gameplay Kit's screens)	The base of the kit's screens that take over the game (pause menu, dialogue, shop, title): it covers the screen, pauses the game while open...
89	`MDSAreaTitle`	Game UI (the Gameplay Kit's screens)	An area's name, faded in near the top of the screen when the player enters the area (the way Hollow Knight and Silksong title their areas), held, and...
90	`MDSObjectiveBanner`	Game UI (the Gameplay Kit's screens)	A banner with the area's objective (`areas[name].objective` in the world file), shown when the player arrives in an area whose objective isn't done,...

Rank	Object	Group	In one line
91	`MDSCameraDirector`	Presentation	Borrows an `MDSRoomCamera`'s camera for a moment when a fight needs framing, then gives it back exactly where the room's own camera would be
92	`MDSMusic`	Presentation	Exploration music per area for an `MDSWorldGame`: each area's `music` (a stream path in the world file) loops at its `music_volume_db`, and changing...
93	`MDSLevelOverview`	Presentation	A pull-back over the level: pictures of the rooms around the live one (`MDSRoomPictures`), each in its real place on the world map, while the camera...
94	`MDSRoomPictures`	Presentation	Pictures of rooms, made before they are needed: an `MDSLevelOverview` (a death screen that pulls back over the level, a map preview) shows pictures...
95	`MDSRoomDresser`	Presentation	Fits props to the floor and declutters rooms as they load (`MDSRoomDressing`), for rooms that are generated or dressed at runtime
96	`MDSEnvironmentEffect`	Environment effects	Base of the environment effects: weather (`MDSDustStorm`, `MDSRain`, `MDSSnowfall`, `MDSLightning`, `MDSFog`), hazards (`MDSSteamVent`, `MDSLava`)...
97	`MDSLava`	Environment effects	A pool of lava (or acid, or cursed ooze): a heaving surface with a bright skin, molten flow with crust plates drifting on it, bubbles bursting, a...
98	`MDSLavaFall`	Environment effects	Lava falls: molten rock pouring straight down over a ledge, slow and thick, with straight sides (the flow never curves)
99	`MDSSteamVent`	Environment effects	A vent forcing out hot steam under pressure: thin wisps curl out of it, its mouth glows as a warning, then a jet blasts out (white-hot at the vent,...
100	`MDSRockfall`	Environment effects	Falling rocks: stones break off at the top of the rectangle (put it under a ceiling) and drop, tumbling, to crash on the ground in a puff of dust and...
101	`MDSWater`	Environment effects	A pool or a flooded passage: a rolling surface with a bright line along it, what is behind showing through, bent and tinted deeper toward the bottom,...
102	`MDSWaterfall`	Environment effects	A waterfall: a sheet of water pouring straight down the rectangle, with straight sides and straight streaks racing along it (the flow never curves),...
103	`MDSHotWaterfall`	Environment effects	Hot water falls: scalding mineral water pouring straight down the rectangle, milky and pale where it runs thick, with a warm glow deep in it,...
104	`MDSRain`	Environment effects	Rain: streaks at three depths falling at an angle, splashes along the bottom of the rectangle (put it on the ground) and a low mist
105	`MDSSnowfall`	Environment effects	Snowfall: soft flakes at four depths drifting down, swaying, carried by the wind
106	`MDSFog`	Environment effects	Drifting fog or mist: warped banks rolling slowly sideways, over the whole rectangle or hugging its bottom (`ground`: marsh mist, cave floor haze)
107	`MDSDustStorm`	Environment effects	A dust storm blowing from one side: banks of dust billowing past in gusts, wisps racing ahead of them, dark grit and pale motes driven along (the...
108	`MDSLightning`	Environment effects	Lightning: every few seconds a forked bolt cracks down through the sky (behind the terrain), the whole rectangle flashes and flickers, and the...
109	`MDSLightShafts`	Environment effects	Shafts of light slanting down from openings above (god rays), shimmering slowly, with dust motes drifting in them
110	`MDSFallingLeaves`	Environment effects	Leaves or petals drifting down: each tumbles as it falls, showing its paler back, swaying on the breeze
111	`MDSFireflies`	Environment effects	Fireflies, glowing spores or glitter: motes wandering on slow loops, pulsing on and off
112	`MDSWisps`	Environment effects	Will-o'-wisps, like the Wisp Thicket's in Silksong: ghostly flames drifting on slow, looping paths, a flickering tongue of fire trailing behind each...
113	`MDSEmbers`	Environment effects	Embers rising from fires below, wobbling and flickering out as they cool (a volcano, a burning village, a forge), or ash and soot drifting down
114	`MDSGnatSwarm`	Environment effects	A gnat swarm, like Bilewater's in Silksong: clouds of tiny gnats buzzing around swarm centres that drift about, wings catching the light
115	`MDSHeatHaze`	Environment effects	Heat haze: everything drawn behind the rectangle shimmers, rippling upward, strongest low down

Rank	Object	Group	In one line
116	`MDSIncenseSmoke`	Environment effects	Incense smoke, like the Citadel's choral chambers in Silksong: ribbons of smoke curling up from censers along the rectangle's bottom edge (put it on...
117	`MDSForgeSparks`	Environment effects	Forge sparks, like the Deep Docks' in Silksong: sparks spraying from a `source` (a forge, a grinding wheel, a cut chain; the middle of the top edge...
118	`MDSDrips`	Environment effects	Dripping water, like the Wormways' and the Deep Docks' in Silksong: drops gathering under the rectangle's top edge (a ceiling), swelling, falling and...
119	`MDSHangingMoss`	Environment effects	Hanging moss and vines, like Greymoor's and the Moss Grotto's in Silksong: leafy strands hanging from the rectangle's top edge (put it under a...
120	`MDSSwayingGrass`	Environment effects	Swaying grass, like the meadows of Silksong's Far Fields and Hollow Knight's Greenpath: blades along the rectangle's bottom edge (put it on the...
121	`MDSCobwebs`	Environment effects	Cobwebs, like the Weavenest's and Shellwood's in Silksong: spider webs strung across the rectangle's corners (and a whole one in its middle if you...
122	`MDSSilkThreads`	Environment effects	Silk threads, like the Weavenest's in Silksong: strands of silk drifting and turning slowly in the air, light running along them, and threads hanging...
123	`MDSVoidTendrils`	Environment effects	Void tendrils, like the Abyss's in Silksong and Hollow Knight: tendrils of black void rising from the rectangle's bottom edge (put it on the floor)...
124	`MDSParallaxBackground`	Environment effects	A parallax background: layers behind the room, each moving at its own share of the camera's speed (`MDSParallaxLayer.scroll_scale`), the far ones...
125	`MDSParallaxLayer`	Environment effects	One layer of an `MDSParallaxBackground`: a sky, a silhouette drawn by a shader (mountains, hills, a forest, a city, ruins, cave rock, dunes), clouds,...
126	`MDSAnimatedBackground`	Environment effects	An animated background drawn by a shader: soft, out of focus, as if far behind the room and seen through a lens
127	`MDSMachineryBackground`	Environment effects	A background of giant machinery, soft and out of focus as if far behind the room: one kind of machine at a time (`machine`), several of them at...
128	`MDSCreatures`	Environment effects	A background of living things, made for boss rooms: one kind of creature at a time, many of them, each glowing and carrying its own light, which...
129	`MDSStateMachine`	The player's insides	A finite state machine shared by the player, enemies and bosses: its `MDSState` children are the states, by name
130	`MDSState`	The player's insides	A state of an `MDSStateMachine`: a child of it, named after the state ("Idle", "Dash")
131	`MDSPlayerState`	The player's insides	A state of `MDSPlayer`'s state machine
132	`MDSInputBuffer`	The player's insides	Remembers presses for a moment, so a jump pressed just before landing still jumps, and a dash or an attack pressed during another move still happens...
133	`MDSPlayerVisual`	The player's insides	The player's placeholder look: a small cloaked figure drawn in code (no art needed), facing the way it moves, squashed when crouching or sliding,...
134	`MDSGameplayDebug`	The player's insides	Runtime debug hotkeys (debug builds only, when project setting debug_keys is true): F1: overlay (state names, hitboxes, velocity) F2: grant all...

1. Foundations: every MDS game has these

The game root, the doors between rooms, the player, the Gameplay Kit's hub, the camera and how the player moves. Get these right first.

1. `MDSWorldGame`

Is a: `MDSWorldGame` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/mds_world_game.gd`

Game runtime for non-linear worlds: the counterpart of MetSys' MetSysGame, with no MetSys dependency. Reads the same .mdsworld.json the MetDevSys panel edits.

Use it as the root of your game scene (or extend it): What it does:

- Loads one room scene at a time, placed at its world position (so world coordinates are the same in every room and on the map), like Hollow Knight.
- Follows `MDSGate` transitions: entering a gate (or pressing Interact at a door) loads the target room and puts the player at the entry gate. Gate and link requirements from the map can be enforced; map links can be fast-travel pairs; a gate can play a cinematic between rooms.
- Optionally switches rooms without gates when the player walks into a touching room.
- Dresses each room as it loads: 2.5D (`MDSDepth25D`), the area's backdrop (`MDSBackdropView`), the area's weather (rain, dust storms, snow... see `MDSEnvironment`), darkness with lights around the player and enemies.
- Tracks visited rooms, the current area, abilities, collected objects, defeated enemies and bosses, and area objectives, with `get_save_data` / `set_save_data`. An exploration file keeps the map explored (and the save points found) even when the game isn't saved, so a restarted game shows the map as far as the player got.
- Markers on the map (`add_map_marker`): a boss room, a locked door, a key, a quest's target, the player's own pins (placed on the pause menu's map), each with a title and a description; kept like the explored map and shown on every map of the game.
- Saves the game at save points, like Hollow Knight's benches: `save_game` writes `save_file` (an `MDSBench` calls it when the player rests on it) and records the bench as the place to come back to (`return_to_save_point`, after dying). A game started again continues from that save (`load_on_start`); `new_game` starts over.
- Carries followers (the `carry_over_group`) through gates with the player.
- Works with the panel's "Play from here" (`mds_play_from`).

Add it in Godot: The **root of your game scene**: Scenes > **Create game scene...** (MetDevSys dock) builds it with everything under it; by hand, Scene > New Scene > **Other Node** > search `MDSWorldGame`, then add the player, camera and UI.

Use it in your game: Make it the **root of your game scene** (Scenes > Create game scene builds the whole tree for you). Set `world_file` to your `.mdsworld.json`, assign `player`, `camera`, `map_view` and `room_camera`, and press F6. It loads the start room (or continues a save), moves the player through gates, saves at benches and keeps the explored map. Your own code talks to it: `grant_ability()`, `has_ability()`, `load_room()`, `save_game()`, `add_map_marker()`, and its signals (`room_changed`, `area_changed`, `ability_gained`, `transition_blocked`...). Turn `enforce_requirements` on to refuse gates whose map `requires` aren't met.

Inspector properties:

Property	Type	Default	What it does
`world_file`	String (file)	`""`	The world file (`*.mdsworld.json`) to play. Empty: the project setting `metroidvania_developer_system/world_file`.
`starting_room`	String	`""`	Room id (as on the map) loaded when the game starts without save data.
`starting_gate`	String	`""`	Gate the player starts at in the starting room (empty: the room's first save point or its middle).

Property	Type	Default	What it does
`player`	Node2D		The player node (a CharacterBody2D in the `player` group). Empty: the first node of the game in group `player`.
`camera`	Camera2D		The Camera2D (under the player). Empty: a Camera2D found under the player.
`map_view`	MDSWorldMapView		The minimap (`MDSWorldMapView`) the game keeps up to date.
`room_camera`	MDSRoomCamera		Camera controller for irregular rooms and room transitions. Empty: an MDSRoomCamera child if there is one, else the Camera2D is limited to the room's bounding box.
`enforce_requirements`	bool	`false`	Refuse gate transitions whose map requirements the player lacks.
`seamless_rooms`	bool	`false`	Load the neighboring room when the player walks into it, without a gate.
`fade_time`	float	`0.2`	Fade to black between rooms (seconds, 0 = instant). With a room camera, its transition settings are used instead.
`gate_cooldown`	float	`0.35`	Ignores gates briefly after arriving, so the player can't bounce straight back.
`keep_momentum`	bool	`true`	Keep the player's velocity through gates, like Hollow Knight: a run or a jump carries into the next room. Off: the player arrives standing still.
`up_exit_speed`	float	`650.0`	Coming up through a gate in the floor of the next room ("bot" gates), the player gets at least this upward speed (px/s), so it clears the hole and can land beside it instead of falling straight back. 0 = off.
`land_beside_holes`	bool	`true`	Coming up through a gate in the floor of the next room, the player is set down at the rim of the hole, on the nearest floor beside it (within reach of `up_exit_speed`'s hop), as Hollow Knight does: else, with nothing pushing it sideways, it would fall straight back through the hole. Rooms with no floor beside the hole keep the player over it.
`remove_room_players`	bool	`true`	Room scenes may contain a player of their own, to test them on their own (F6). It is removed when the room is loaded into the game, which has the real player.
Presentation			
`depth_25d`	int (enum)	`0`	2.5D in every room (`MDSDepth25D`): as the world settings say (`depth_25d`), always, or never. A room that has its own MDSDepth25D keeps it.
`depth_style`	MDSDepthStyle		Style of the 2.5D. Empty: the world setting `depth_style` (a .tres), else the defaults.
`backdrop_view`	MDSBackdropView		Draws the distance behind the rooms: the `MDSBackdrop` of the room (`rooms[id].backdrop` in the world file), else of its area (`areas[name].backdrop`), else the world's (`settings.backdrop`). Empty: made when a backdrop is needed.
`room_darkness`	bool	`true`	Dim dark rooms (`darkness` of the room or its area, see `get_darkness`).

Property	Type	Default	What it does
`room_weather`	bool	`true`	Add each room's weather as it loads: the room's (`rooms[id].weather`), else its area's (`areas[name].weather`), else the world's (`settings.weather`), sized to the room (see `MDSEnvironment`).
`room_parallax`	bool	`true`	Add each room's parallax background as it loads: the room's (`rooms[id].parallax`), else its area's, else the world's (`settings.parallax`), see `MDSParallaxBackground`.
`room_background`	bool	`true`	Add each room's animated background as it loads: the room's (`rooms[id].background`); in a boss room its area's or the world's `boss_background`; else its area's, else the world's (`settings.background`), see `MDSAnimatedBackground`.
`light_groups`	PackedStringArray	`["player", "enemy", "lantern"]`	Nodes in these groups carry a soft light in dark rooms (the player, enemies, lanterns). The world setting `light_groups` replaces the list.
Persistence			
`save_file`	String	`"user://mds_save.json"`	Where `save_game` writes the game, as JSON (resting on a bench saves there). Empty: no save file (`get_save_data` still gives the data, to save it your own way).
`load_on_start`	bool	`true`	When the game starts, continue from `save_file` if there is one (at the bench it was saved at), else start a new game in `starting_room`. "Play from here" and a save your own code loads first win.
`exploration_file`	String	`"user://mds_exploration.json"`	A file of its own that keeps the visited rooms, the map's reveal, the save points found and the room the player is in, written as each room is entered: dying or quitting without saving never forgets the map, and a restarted game shows it as far as the player got. The editor's World map shows it too (Show on map > Explored in the last playtest, see `read_exploration`). Empty: off.
`carry_over_group`	StringName	`&"carry_over"`	Nodes in this group that are near the gate the player leaves through (a guard giving chase) come along into the next room, and stay where they end up.
`carry_over_distance`	float	`400.0`	How close (px) to the exit gate a node of the carry-over group must be to come along.

Signals (Node dock > Signals, to connect): `room_loaded(room_id: String)`, `room_changed(from_room: String, to_room: String)`, `area_changed(from_area: String, to_area: String)`, `transition_blocked(transition: Dictionary, missing: PackedStringArray)`, `ability_gained(ability: String)`, `objective_completed(area: String)`, `game_saved(save_point: Node)`, `game_loaded`, `defeated(id: String, boss: String)`, `map_pin_added(id: String)`, `map_pin_removed(id: String)`

2. `MDSGate`

Is a: `MDSGate` → `Area2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/mds_gate.gd`

A room transition, like Hollow Knight's TransitionPoint.

Put one per exit in a room scene and name it like the gate on the world map (`left1`, `right1`, `top1`, `bot1`, `door1`...). When the player enters it, `player_entered` reports where to go, read from the `.mdsworld.json` file, so connections are edited on the map instead of in every scene:

- `mode` INTERACT makes a door: it waits for the Interact action and shows a prompt with where it leads.
- `link` makes it one end of a map link (an elevator, a stag station, a teleport): it leads to the other end of the link instead of a gate connection.
- `transition_scene` plays a cinematic between the rooms (`MDSWorldGame`).
- An `MDSGateBarrier` can keep it shut until its requirements are met.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSGate`.

Use it in your game: One per exit in each **room scene**, named exactly like the gate on the world map (`left1`, `right1`, `top1`, `bot1`, `door1`). The room's **Inspect tab > Write to scene** creates them for you. Its collision shape is the doorway: walking into it (TOUCH) or pressing Up in it (INTERACT, for doors) loads the connected room. Connections live on the map, not in the scene.

Inspector properties:

Property	Type	Default	What it does
`world_file`	String (file)	`""`	World file to read connections from. Empty: the project setting `metroidvania_developer_system/world_file`.
`gate_name`	String	`""`	Gate name on the world map. Empty: this node's name.
`room_id`	String	`""`	Room id on the world map. Empty: looked up from the owning scene's path.
`player_group`	StringName	`&"player"`	Bodies in this group trigger the transition.
`spawn_offset`	float	`96.0`	Distance from the gate, towards the room's inside, where players entering through it are placed.
`mode`	Mode	`Mode.TOUCH`	TOUCH: the player goes through on contact. INTERACT: a door, entered with `interact_action` while the player stands in it.
`interact_action`	StringName	`&"ui_up"`	The action that opens a door (INTERACT mode).
`prompt`	String	`"Up: {to}"`	Shown over the gate while the player can use it (INTERACT). `{to}` is replaced with where it leads.
`link`	bool	`false`	Leads to the other end of a map link (World map > Links) from this room, instead of the gate connection. The link can name the gate at each end; else the other room's first link gate (or its middle) is the way in.
`transition_scene`	PackedScene		A scene played over everything between the rooms (a travel cinematic). Its root may have a `play(transition)` method (awaited) or a `finished` signal.

Signals (Node dock > Signals, to connect): `player_entered(transition: Dictionary)`

3. `MDSPlayer`

Is a: `MDSPlayer` → `CharacterBody2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/player/mds_player.gd`

The Gameplay Kit's player: a CharacterBody2D with components (composition), in the `player` group so `MDSGate`, `MDSBench`, `MDSWorldMapView`, `MDSRoomCamera` and dark rooms' lights find it. `gameplay/player/mds_player.tscn` is the ready-made one: Missing parts are made when it starts, so a bare MDSPlayer works too.

How it moves is `movement` (`MDSMovementConfig`): run and air control, a jump that is cut short when released, coyote time, a jump buffer, lighter gravity at the apex, corner correction, slopes, moving platforms, dropping through one-way platforms (down + jump). Each movement ability is a state that can only be entered while the player has it (`has_ability`).

With `MDSWorldGame` it keeps its velocity through gates (the game restores it on arrival), and while the game changes rooms it is Locked (no input, no gravity, velocity untouched). On a bench it sits (`mds_bench_rest`) and is healed (`heal_full`). Hits from MDS's environment effects reach its `MDSHealth` (`mds_environment_hurt`, `take_damage`).

Add it in Godot: Scene dock > **Instantiate Child Scene** (Ctrl+Shift+A) > `addons/MetroidvaniaDeveloperSystem/gameplay/player/mds_player.tscn`, under the `MDSWorldGame`.

Use it in your game: Instance `res://addons/MetroidvaniaDeveloperSystem/gameplay/player/mds_player.tscn` as a direct child of the game. Give it an `MDSMovementConfig` to tune the feel (the Inspector previews the jump arc) and `MDSCombatStats` for

health. Replace `Visual` with an `AnimatedSprite2D` for real art: states play animations named after them (`idle`, `run`, `jump`, `dash`...). It needs an `MDSGameplay` beside it in the game, or gates won't see it (it's on the `player` collision layer, 3).

Inspector properties:

Property	Type	Default	What it does
`movement`	MDSMovementConfig		How the player moves. Empty: the defaults.
`stats`	MDSCombatStats		Health, mana and defense (its `MDSHealth` uses them). Empty: the game's (`MDSGameplay`), else 5 health.
`body_size`		`Vector2(28, 60)`	The size of the body standing (its collision rectangle; the origin is its middle).
`input_enabled`	bool	`true`	Off: the player ignores its controls (cutscenes).
`abilities_without_game`	PackedStringArray	`[]`	Abilities the player has when there is no `MDSWorldGame` (a room scene played on its own).
`safe_ground_time`	float	`0.2`	Seconds standing on terrain before the spot counts as safe ground (where hazards send it back).

Signals (Node dock > Signals, to connect): `landed`, `jumped`, `died`, `respawned`, `state_changed(from: StringName, to: StringName)`

4. `MDSGameplay`

Is a: `MDSGameplay` → `Node` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/mds_gameplay.gd`

The Gameplay Kit's hub: a child of the `MDSWorldGame` root that owns everything the kit adds to a game's progress (items, currency, equipment, story flags, health and mana, enemies that come back after a rest, shops' stock, the corpse, play time and deaths). `MDSWorldGame` stays the one source of rooms, abilities, stored objects and defeats; this node is saved with it (in `ext.gameplay` of the save, see `mds_get_save_data`) and answers its `item:` and `flag:` conditions (`is_condition_met`): Those work wherever MDS takes a condition: gate `requires`, `MDSGateBarrier`, area objectives, dialogue lines and shop entries. Every change emits its signal and lets the game check its objectives.

Kit nodes find it with `find` (or `find_from`); it finds the game as its parent, else `MDSWorldGame.instance`. There are no autoloads.

Add it in Godot: A direct child of the `MDSWorldGame`: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSGameplay`. Create game scene adds it.

Use it in your game: A **direct child of `MDSWorldGame`** (Create game scene adds it). It holds items, currency, flags, health and mana, equipment, the bestiary, and saves them with the game. It also gives gates the player's collision layer, shows ability popups and runs the death flow. Call it from code: `MDSGameplay.find(game).add_item("red_key")`, `set_flag()`, `add_currency()`, `equip()`. Fill `items_database` / `abilities_database` with your `.tres` files so ids resolve to names and icons.

Inspector properties:

Property	Type	Default	What it does
`game`	MDSWorldGame		The game. Empty: the parent, else `MDSWorldGame.instance`.
`stats`	MDSCombatStats		The player's base health and mana (empty: 5 health, 33 mana).
`starting_equip_slots`	int	`3`	Equipment slots a new game starts with (`MDSEquipmentData`, `MDSSlotUpgrade` adds more).
`fragments_per_upgrade`	int	`4`	Health fragments (`MDSHealthUpgrade`) that make one more health.
`mana_fragments_per_upgrade`	int	`3`	Mana fragments (`MDSManaUpgrade`) that make more mana, and how much.
`mana_per_upgrade`	int	`33`	Maximum mana gained per completed mana upgrade.

Property	Type	Default	What it does
`items_database`	Array[MDSItemData]	`[]`	Items and abilities known by id (their names, icons and effects for the HUD, the inventory, popups and "Requires ..." notices). Items picked up are added, and the built-in abilities (`DATA_DIR`) are always known.
`abilities_database`	Array[MDSAbilityData]	`[]`	Your `MDSAbilityData` files, so ability ids show their names and icons.
`level_curve`	MDSLevelCurve		Experience levels, with the LEVELS progression.
`show_ability_popups`	bool	`true`	Show `MDSAbilityPopup` when an ability is gained.
`death_delay`	float	`1.2`	Seconds between the player's death and the trip back to its bench.
`death_overview_time`	float	`0.0`	Seconds an `MDSLevelOverview` child of the game pulls back over the level after a death (skippable; 0: none).

Signals (Node dock > Signals, to connect): `item_changed(id: String, count: int)`, `currency_changed(total: int)`, `flag_changed(flag: String, value: Variant)`, `equipment_changed`, `player_health_changed(current: int, max: int)`, `player_mana_changed(current: int, max: int)`, `player_died`, `boss_started(boss: String)`, `boss_ended(boss: String, won: bool)`, `notice(text: String)`, `level_up(level: int)`

5. `MDSRoomCamera`

Is a: `MDSRoomCamera` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/mds_room_camera.gd`

Camera for `MDSWorldGame` rooms of any shape, driving a `Camera2D` or, when the Phantom Camera addon is installed, `PhantomCamera2D` nodes.

`Irregular rooms.` An L-, T- or U-shaped room is split into camera zones: the largest rectangles that fit in its shape on the map. The camera is limited to the zone the player is in, so it never shows the solid rock in the room's notches, and it glides to the next zone when the player moves into it (like Hollow Knight's camera locks). Zones smaller than the screen grow to the screen size, staying inside the room.

`Room transitions.` Cut, fade to black, slide (the view pans from the old room to the new one while the player waits, like classic Metroid) or blend (the limits glide over and the player keeps moving).

`Backends.` Camera2D: MDS sets the camera's limits (animated) itself. Phantom Camera: one PhantomCamera2D per zone, the active zone gets the highest priority and the PhantomCameraHost tweens between them. The Camera2D needs a PhantomCameraHost child, which is added when missing.

Add it as a child of the MDSWorldGame (Create game scene does). Every setting can also come from the world file (World settings > Camera in the panel) when `use_world_settings` is on.

`Fights.` An `MDSCameraDirector` child borrows the camera for a moment: framing a threat, revealing an arena, punching in, focusing on a node (`get_director`).

`Zoom between rooms.` Like a PhantomCamera2D per room, each room (and each camera zone of an irregular room) can have a zoom of its own (`auto_zoom`): set per room (Inspect tab > Camera zoom), or fitted to the zone so big rooms are seen whole and small ones closer (`zoom_factor`, `min_zoom`, `max_zoom`). The zoom glides there with the room transition and zone changes (`zoom_tween`, `zoom_time`), or snaps.

Add it in Godot: A direct child of the `MDSWorldGame`: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSRoomCamera`. Create game scene adds it.

Use it in your game: A child of the game (Create game scene adds it, with a `Camera2D` under the player). It keeps the view inside the room, whatever its shape, and plays the transition between rooms (`room_transition`: cut, fade, slide, blend). `zoom` is the normal zoom; camera zones painted on the map can change it per part of a room. `get_director()` gives its `MDSCameraDirector`.

Inspector properties:

Property	Type	Default	What it does
`backend`	Backend	`Backend.AUTO`	AUTO: Phantom Camera when it's installed, else Camera2D.
`camera`	Camera2D		The Camera2D that renders the game. Empty: the MDSWorldGame's camera, else a new one.
`target`	Node2D		Node the camera follows. Empty: the MDSWorldGame's player.
`use_world_settings`	bool	`true`	Values in the world file (settings.camera) override the ones set here.
`zoom`	enum	`Vector2.ONE`	The camera's normal zoom.
`transition_style`	Motion	`Motion.GLIDE`	GLIDE: zones, slides, blends and the director's moves glide. CUT: the camera never glides: zone changes and room transitions cut, follow smoothing is off and the director cuts to its view and back (a camera that eases late makes the parallax and the backdrop late too).
Zoom between rooms			
`auto_zoom`	AutoZoom	`AutoZoom.OFF`	OFF: always `zoom`. ROOM: a room's own zoom (its `ROOM_ZOOM_KEY` room value, Inspect tab > Camera zoom), else `zoom`. FIT: the zoom that fills the screen with the camera zone the player is in (out in big rooms and zones, in in small ones), times `zoom_factor`, between `min_zoom` and `max_zoom`; a room's own zoom still wins.
`zoom_factor`	float (range)	`1.0`	FIT: scales the zoom that fills the zone (above 1 closer, below 1 farther out).
`min_zoom`	float (range)	`0.5`	FIT: the farthest out it zooms.
`max_zoom`	float (range)	`2.0`	FIT: the closest in it zooms.
`zoom_tween`	bool	`true`	Zoom changes glide (on) or snap (off). They glide with the room transition (slide, blend) and zone changes, and over `zoom_time` after a cut or a fade.
`zoom_time`	float	`0.6`	Seconds the zoom glides after a cut or a fade into a room.
`zoom_trans`	Tween.TransitionType	`Tween.TRANS_SINE`	*(not documented)*
`zoom_ease`	Tween.EaseType	`Tween.EASE_IN_OUT`	*(not documented)*
`zoom_tween_threshold`	float (range)	`0.03`	Zoom changes smaller than this share (0.03: 3%) snap instead of gliding.
Irregular rooms			
`confine`	Confine	`Confine.ROOM_SHAPE`	How the camera is kept inside a room: by its shape (zones), by its bounding box, or not.
`zone_blend_time`	float	`0.5`	Seconds the camera takes to glide between zones of a room.
`zone_trans`	Tween.TransitionType	`Tween.TRANS_SINE`	*(not documented)*
`zone_ease`	Tween.EaseType	`Tween.EASE_IN_OUT`	*(not documented)*
`zone_hysteresis`	float	`32.0`	How far (px) the player must leave a zone before the camera switches zones. Stops flicker at zone edges.
Room transitions			
`room_transition`	Transition	`Transition.FADE`	How the view changes rooms: CUT, FADE, SLIDE or BLEND.
`transition_time`	float	`0.35`	Seconds for fade, slide and blend.
`transition_trans`	Tween.TransitionType	`Tween.TRANS_SINE`	*(not documented)*
`transition_ease`	Tween.EaseType	`Tween.EASE_IN_OUT`	*(not documented)*
`freeze_player_on_slide`	bool	`true`	Freeze the player while a slide pans the view.
Follow			

Property	Type	Default	What it does
`follow_smoothing`	float	`0.0`	Camera2D position smoothing speed (0 = off). With Phantom Camera: follow damping.
`follow_offset`	enum	`Vector2.ZERO`	Offset of the view from the player.
`phantom_follow_mode`	int	`2`	PhantomCamera2D follow mode (0 none, 1 glued, 2 simple, 5 framed).
`phantom_priority`	int	`20`	Priority given to the active zone's PhantomCamera2D (others get 0).

Signals (Node dock > Signals, to connect): `zone_changed(zone: Rect2)`

6. `MDSMovementConfig`

Is a: `MDSMovementConfig` → `Resource` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/resources/mds_movement_config.gd`

How the player moves (`MDSPlayer`): one feel profile, in px and seconds. MDS rooms are 1152 x 648 px by default. The Inspector previews the jump arc these values give (height and distance), next to that room size.

Add it in Godot: FileSystem dock > right-click a folder > **Create New > Resource...** > search `MDSMovementConfig` > Save as `.tres`. Then drag the file onto the property that takes it (or click that property > **New MDSMovementConfig**).

Use it in your game: A resource: right-click in the FileSystem dock > Create New > Resource > MDSMovementConfig, save it, and drag it onto the player's `movement`. Select it to see the jump-arc preview in the Inspector (height and distance against a 1152 x 648 room). Defaults: a jump of about 90 px, a double jump of about 180, a dash of 96 px. Design your gaps and ledges around these numbers.

Inspector properties:

Property	Type	Default	What it does
Run			
`max_run_speed`	float	`260.0`	Top running speed (px/s).
`ground_accel`	float	`2000.0`	px/s² while running, and while stopping.
`ground_decel`	float	`2400.0`	Slowing down on the ground (px/s²).
`air_accel`	float	`1400.0`	Steering in the air (px/s²).
`air_decel`	float	`1000.0`	Slowing down in the air (px/s²).
Gravity			
`gravity`	float	`1500.0`	Gravity while rising (px/s²).
`fall_gravity_multiplier`	float	`1.6`	Falling (going down) is this much heavier, for a snappy arc.
`max_fall_speed`	float	`700.0`	Fastest fall (px/s).
`apex_threshold`	float	`40.0`	Near the top of a jump (slower than this, px/s, jump held) gravity is lighter.
`apex_gravity_multiplier`	float	`0.5`	Gravity near the top of a jump while jump is held.
Jump			
`jump_velocity`	float	`520.0`	Upward speed of a jump: sets the jump's height (about 90 px by default).
`jump_cut_multiplier`	float	`0.45`	Velocity kept when jump is released early (a short hop).
`coyote_time`	float	`0.10`	A jump still works this long after running off a ledge.
`jump_buffer_time`	float	`0.12`	A jump pressed this long before landing still happens on landing.
`max_air_jumps`	int	`1`	Extra jumps in the air, with the double_jump ability.

Property	Type	Default	What it does
`corner_correction_px`	float	`6.0`	Nudges the player past a ceiling corner it would bump its head on, up to this many px.
Walls			
`wall_slide_speed`	float	`100.0`	Slide speed down a wall (wall_jump).
`wall_jump_velocity`		`Vector2(300, -500)`	Away from the wall (x) and up (y).
`wall_jump_lock`	float	`0.15`	Seconds after a wall jump before steering works again.
Dash and slide			
`dash_speed`	float	`600.0`	Dash speed (dash, air_dash).
`dash_time`	float	`0.16`	How long a dash lasts (s): dash_speed x dash_time is its distance.
`dash_cooldown`	float	`0.35`	Seconds before the next dash.
`air_dashes`	int	`1`	Dashes in the air before landing again, with air_dash.
`slide_speed`	float	`400.0`	Slide speed (slide).
`slide_time`	float	`0.35`	How long a slide lasts (s).
Climb, swim, glide			
`climb_speed`	float	`140.0`	Climbing speed on ladders and vines.
`swim_speed`	float	`180.0`	Swimming speed.
`swim_gravity`	float	`300.0`	Gravity in water: sinking without the swim ability, drifting with it.
`glide_fall_speed`	float	`80.0`	Falling speed while gliding.
Grapple and pound			
`grapple_range`	float	`260.0`	How far the grapple reaches a grapple point.
`grapple_pull_speed`	float	`700.0`	How fast the grapple pulls.
`ground_pound_speed`	float	`900.0`	Speed of the ground pound's slam.
Floors			
`max_floor_angle`	float (range)	`45.0`	Steepest floor walked on (degrees).
`floor_snap_length`	float	`8.0`	How far the body sticks to floors going down slopes.

7. `MDSCombatStats`

Is a: `MDSCombatStats` → `Resource` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/resources/mds_combat_stats.gd`

Health, mana and toughness of the player, an enemy or a boss (`MDSHealth`).

Add it in Godot: FileSystem dock > right-click a folder > **Create New > Resource...** > search `MDSCombatStats` > Save as `.tres`. Then drag the file onto the property that takes it (or click that property > **New MDSCombatStats**).

Use it in your game: A resource for health, mana, defense, contact damage and resistances. On the player (or on `MDSGameplay.stats` for the whole game), and inside every `MDSEnemyData`.

Inspector properties:

Property	Type	Default	What it does
`max_health`	int	`5`	Maximum health.
`max_mana`	int	`0`	Maximum mana.
`defense`	int	`0`	Taken off every hit (a hit always does at least 1).
`contact_damage`	int	`1`	Damage done by touching it (enemies).

Property	Type	Default	What it does
`resistances`	Dictionary	`{}`	Damage type -> multiplier ({"fire": 0.5, "ice": 2.0}); a type not listed takes 1.
`knockback_resistance`	float (range)	`0.0`	0 to 1: how much less it is pushed back by hits.

8. `MDSBench`

Is a: `MDSBench` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_bench.gd`

A bench to rest on: the save point of a room, after Hollow Knight: Silksong's benches. A wrought-iron bench with a cushioned seat and a lantern on a hook, drawn and animated by a shader (shaders/environment/bench.gdshader). The lantern flickers and lights the bench, and motes drift up in its light.

When the player (a node of `MDSEnvironmentEffect.affect_groups`) stands at it, a warm rim shows and `prompt_text` floats above it. Pressing one of `rest_actions` (Up) sits the player down:

- the player is held still on the bench (or its `mds_bench_rest(bench)` runs, for a sitting animation of its own) until one of `leave_actions` is pressed;
- its health is restored (`heal_full()`, `restore_health()`, or `health = max_health`), with `heal_on_rest`;
- the running `MDSWorldGame` saves the game here (`MDSWorldGame.save_game`), with `save_on_rest`: a game started again continues on this bench, dying brings the player back to it (`MDSWorldGame.return_to_save_point`), and the map shows it;
- the bench glows, threads of light rise from it and a ring of light spreads out.

It is in the `bench` and `save_point` groups, so the map, the analysis and "Play from here" know it as a save point, and in `lantern`, so it carries a light in dark rooms. Place it from the Room view's Effects tool (it stands on the floor where you click), or add it like any node.

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSBench` under the room's `Effects` node). Its rectangle is set by `size`.

Use it in your game: The save point. Place it on a floor in a room; the player rests with Up (interact): full health, mana refilled, the game saved, and this becomes where the player returns after dying. Enemies that respawn on rest come back. It's also an environment effect, so the Room view's Effects tool can place it.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`frame_color`		`Color(0.17, 0.15, 0.19)`	*(not documented)*
`trim_color`		`Color(0.80, 0.63, 0.36)`	*(not documented)*
`seat_color`		`Color(0.42, 0.27, 0.18)`	*(not documented)*
`cushion_color`		`Color(0.52, 0.16, 0.22)`	*(not documented)*
`glow_color`		`Color(1.0, 0.78, 0.46)`	The lantern's light, the glow while resting and the ring of light when saving.
`lantern`	bool	`true`	A lantern hanging from a hook beside the back.
`motes`	float (range)	`0.6`	*(not documented)*
Rest			
`rest_actions`	PackedStringArray	`["interact", "ui_up", "move_up"]`	Input actions that sit the player down when it is at the bench (the first ones that exist).
`leave_actions`	PackedStringArray	`["ui_left", "ui_right", "ui_down", "move_left", "move_right", "move_down", "jump", "ui_accept", "ui_cancel"]`	Input actions that get the player up again.

Property	Type	Default	What it does
`reach`	float (range)	`36.0`	How far (px) to each side of the bench the player can be and still sit down.
`save_on_rest`	bool	`true`	Resting saves the game (the running `MDSWorldGame`'s `MDSWorldGame.save_game`).
`heal_on_rest`	bool	`true`	Resting restores the player's health.
`prompt_text`	String	`"Rest"`	Shown above the bench while the player is at it ("" for none).
`saved_text`	String	`"Game saved"`	Shown for a moment after the game is saved here ("" for none).
`spawn_offset`		`Vector2(0, -32)`	Where the player is put when a game loads on this bench, from the bench's feet: up by about half the player's height for a player whose origin is its middle (0 for one whose origin is its feet).

Signals (Node dock > Signals, to connect): `rested(body: Node2D)`, `left(body: Node2D)`

9. `MDSGateBarrier`

Is a: `MDSGateBarrier` → `StaticBody2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/mds_gate_barrier.gd`

A physical gate that stays shut until a condition is met, then opens visibly and stays open (also after saving and loading). Put it in a doorway, next to (or under) the `MDSGate` it guards.

The condition is the gate's requirements on the map (`gate`'s `requires`) or `requires`: abilities, keys (`MDSWorldGame.grant_ability`) and stored objects (`object:<id>`) are one mechanism, so a key is just an ability or a stored object.

It needs a collision shape child. Its look is its other children (sprites...), which slide up and fade as it opens, or an AnimationPlayer child with an "open" animation; with neither it draws a plain bar of its collision's size.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSGateBarrier`.

Use it in your game: A door that stays shut until `requires` is met (`double_jump`, `item:red_key`, `flag:bridge_lowered`, `boss:Warden`), then opens for good (remembered in the save). Put it in front of the gate it guards and give it a CollisionShape2D child. `consume_items` uses the key up. Leave `requires` empty to use the gate's map requirements (set `gate`).

Inspector properties:

Property	Type	Default	What it does
`gate`	MDSGate		The gate it guards: its map requirements are the condition. Empty: a parent MDSGate.
`requires`	PackedStringArray	`[]`	Requirements instead of the gate's: ability names, "object:<id>" or "boss:<name>".
`consume_items`	bool	`false`	Opening takes the items it requires (`item:red_key` uses the key up; see `MDSGameplay`).
`open_time`	float	`0.6`	Seconds the opening takes.
`lift`	float	`0.0`	How far (px) the look slides up as it opens (0: its collision's height).
`bar_color`		`Color(0.35, 0.3, 0.26)`	Colour of the bar drawn when it has no look of its own.

Signals (Node dock > Signals, to connect): `opened`

10. `MDSWorldMapView`

Is a: `MDSWorldMapView` → `Control` · **File:** `addons/MetroidvaniaDeveloperSystem/ui/world_map_view.gd`

In-game map for non-linear worlds, drawn from the same .mdsworld.json the editor uses.

Reveal rules follow Hollow Knight: a room shows once it is visited, or dimmed when the player owns the map of its area (Cornifer/Shakra); everything else stays hidden. Each room explored (visited) has a check mark (`show_explored_marks`); rooms only on a bought map have none. The save points (benches) found show as bench marks, the one last rested on brighter, and a pin marks where the player is. `MDSWorldGame` does all of this for the map view it is given, and keeps the map explored across restarts (its exploration file).

With `follow_player` it is a minimap: a corner of the screen showing the rooms around the player (`view_span` px of the world across), instead of everything revealed.

`Markers` (pins) mark places on the map: a boss room, a locked door, a key, a quest's target, the player's own marks. `set_marker` puts one ({position: world px, layer, kind, label, description, color, icon}): a badge with its kind's icon and color (`marker_kinds`: pin, boss, locked_door, key, quest, danger, treasure, note, or your own with `define_marker_kind`), its label (a title) beside it; pointed at (the mouse over it, or the middle of the map on a gamepad), its description shows in a box (`show_marker_info`, `marker_focused`). They are saved with the map. In a game, use `MDSWorldGame.add_map_marker`: the game keeps them and shows them on its maps. With `pins_editable` (the pause menu's map), a click on the map asks for a pin there (`pin_requested`) and a click on a marker for it to go (`pin_clicked`).

`Zoom.` With `zoomable` (the pause menu's map) the full map zooms and moves: the mouse wheel and `map_zoom_in` / `map_zoom_out` (RT / LT, = / -) zoom, from `min_zoom` (the whole map small in the middle) to `max_zoom`, a drag and `map_pan_*` (the right stick, I J K L) move it. Marks and markers keep their size on the screen, and clicks put pins where they point at any zoom.

Add it in Godot: Under a `CanvasLayer` of the game (Create game scene puts the UI in `UI`): Scene dock > **Add Child Node** (Ctrl+A) > search `MDSWorldMapView`.

Use it in your game: The in-game map. With `follow_player` on it's the minimap (Create game scene puts one at `UI/Map`); the pause menu has a full-screen one. Assign it to `MDSWorldGame.map_view` and the game keeps it up to date: rooms appear as they're visited, benches, the player, markers and pins. `map_area("Area")` reveals an area (a bought map); `reveal_all` shows everything.

Inspector properties:

Property	Type	Default	What it does
`world_file`	String (file)	`""`	The world file to draw (the game sets it).
`layer`	int	`0`	The map layer shown (follows the player's room).
`reveal_all`	bool	`false`	Show every room (for debugging or a "full map" item).
`show_area_names`	bool	`true`	Write area names on the map.
`show_room_names`	bool	`false`	Write room names on the map.
`show_objective`	bool	`true`	Draw the current area's objective under the map (`objective`, set by MDSWorldGame).
`background`		`Color(0.08, 0.07, 0.07, 0.92)`	*(not documented)*
`padding`	float	`24.0`	Space around the map (px).
`show_save_points`	bool	`true`	Draw the save points found (benches), the one last rested on brighter.
`save_point_color`		`Color(1.0, 0.86, 0.55)`	*(not documented)*
`show_explored_marks`	bool	`true`	A check mark on each room explored (visited), when the room is big enough on the map to hold it.
`explored_mark_color`		`Color(0.62, 0.95, 0.55)`	*(not documented)*
`player_marker_color`		`Color(1.0, 0.42, 0.32)`	The pin marking where the player is.

Property	Type	Default	What it does
Zoom			
`zoomable`	bool	`false`	The full map zooms and moves (see the class description, `zoom`).
`max_zoom`	float (range)	`8.0`	How far it zooms in: this many times the whole map's fit.
`min_zoom`	float (range)	`0.25`	How far it zooms out: this share of the whole map's fit (1: no further than every room).
`pan_speed`	float	`700.0`	How fast map_pan_* move it, held all the way (screen px a second).
Pins			
`show_pins`	bool	`true`	Draw the markers (`pins`).
`pins_editable`	bool	`false`	Clicks place pins and take them away (see `pin_requested`, `pin_clicked`).
`show_marker_info`	bool	`true`	The description of the marker pointed at shows in a box beside it.
`pin_colors`	Dictionary	`{}`	This map's own colors for kinds of markers (kind -> Color), over `marker_kinds`'.
Minimap			
`follow_player`	bool	`false`	A minimap: centred on the player, showing `view_span` px of the world across its shorter side, instead of fitting every revealed room.
`view_span`	float (range)	`3000.0`	How much of the world a minimap shows across its shorter side (px).
`frame_color`		`Color(0.85, 0.78, 0.62, 0.8)`	A frame around the minimap.

Signals (Node dock > Signals, to connect): `pin_requested(world_pos: Vector2, map_layer: int)`, `pin_clicked(id: String)`, `marker_focused(id: String)`

11. `MDSWorld`

Is a: `MDSWorld` → `MDSAnnotations` → `RefCounted` · **File:** `addons/MetroidvaniaDeveloperSystem/core/world.gd`

Non-linear world map: rooms (scenes) placed freely in world pixels and connected by Hollow Knight-style named gates. Saved as `*.mdsworld.json`.

The format combines three proven ideas:

- Tiled/LDtk "free" world layouts: every room is a scene at an x/y position in world pixels, of any size, with no grid constraint.
- Hollow Knight transitions: each room has named gates (left1, right1, top1, bot1, door1...) and each gate stores its target room and the entry gate there. A gate whose target doesn't lead back is a one-way transition (drops, collapsing floors).
- Hollow Knight map zones: rooms belong to named, colored areas.

Rooms, gates and areas are also plain annotations, so `MDSAnalysis` and `MDSValidator` work on worlds exactly like on MetSys maps. At runtime use `get_transition` (see `MDSGate`).

Add it in Godot: Edited with the **MetDevSys** dock (world map); saved as `*.mdsworld.json`. In code: `MDSWorld.load_world(path)`, or `game.world`.

Use it in your game: Not a node: the world file (`*.mdsworld.json`) as an object. The MetDevSys panel edits it; your code reads it through `game.world` (rooms, areas, gates, requirements, settings).

Inspector properties:

No Inspector properties of its own.

2. Progression: abilities, items and the things that gate the map

What the player finds and what blocks the way until they have it. These drive the Progress tab's spheres.

12. `MDSAbilityPickup`

Is a: `MDSAbilityPickup` → `Area2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/world/mds_ability_pickup.gd`

An ability to pick up (the dash, the double jump...): touching it (or pressing interact at it, without `auto_collect`) grants `ability` (`MDSWorldGame.grant_ability`) and it is gone for good (`MDSWorldGame.store_object`: not there when its room loads again, nor after a save and load). `MDSGameplay` then shows the `MDSAbilityPopup`, if the game has one.

It writes `mds_grants = [ability id]` metadata (in the editor too), so the Progress and Issues tabs know where the ability is found, and it is in the `pickup` group.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSAbilityPickup`. Also on the Room view's **Effects > Gameplay Objects** menu.

Use it in your game: Put one where the player earns an ability and set `ability` to its `.tres` (the kit ships `gameplay/data/*.tres`: double_jump, dash, wall_jump, glide...). Touching it grants the ability for good and shows the popup. It also tells the Progress tab where the ability is found.

Inspector properties:

Property	Type	Default	What it does
`ability`	MDSAbilityData		The ability granted (`MDSAbilityData`).
`auto_collect`	bool	`true`	Touching it collects it; off: interact at it.
`radius`	float	`22.0`	Its touch radius (px).
`glow_color`		`Color(1.0, 0.85, 0.45)`	*(not documented)*

Signals (Node dock > Signals, to connect): `collected(ability: String)`

13. `MDSAbilityData`

Is a: `MDSAbilityData` → `Resource` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/resources/mds_ability_data.gd`

One unlockable ability of the Gameplay Kit: `id` is what `MDSWorldGame.grant_ability` stores and what gates, barriers and objectives require (`dash` or `ability:dash`). The player's ability states check it with `MDSPlayer.has_ability`; `MDSAbilityPickup` grants it, and `MDSAbilityPopup` shows `display_name`, `description`, `icon` and the glyph of `input_action` when it is gained.

Add it in Godot: FileSystem dock > right-click a folder > **Create New > Resource...** > search `MDSAbilityData` > Save as `.tres`. Then drag the file onto the property that takes it (or click that property > **New MDSAbilityData**).

Use it in your game: One per ability: `id` (what gates and states check), name, description, icon and the input action shown in the popup. The kit's are in `gameplay/data/`; add your own next to a new `MDSPlayerState`.

Inspector properties:

Property	Type	Default	What it does
`id`	String	`""`	Lowercase id, as the world file and the save store it ("double_jump").
`display_name`	String	`""`	Name shown in the popup and menus.
`description`	String (multiline)	`""`	What it does, shown in the popup.
`icon`	Texture2D		*(not documented)*

Property	Type	Default	What it does
`category`	Category	`Category.MOVEMENT`	*(not documented)*
`requires`	PackedStringArray	`[]`	Other abilities it needs to work (informational: shown by the popup and the analysis).
`input_action`	StringName	`&""`	The input action that uses it (its key or button is shown by the popup), or empty.

14. `MDSPickup`

Is a: `MDSPickup` → `Area2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/world/mds_pickup.gd`

An item to pick up: a key, a consumable, a charm, a collectible (`item`, `amount`). Touching it (or interact at it, without `auto_collect`) adds it (`MDSGameplay.add_item`). A `persistent` one is gone for good once taken (`MDSWorldGame.store_object`); drops from enemies are not persistent. With `magnet_radius` it flies to a player that comes near.

It writes `mds_grants = ["item:<id>"]` metadata, so the Progress tab knows where each key is found, and it is in the `pickup` group. Subtypes: `MDSHealthUpgrade`, `MDSManaUpgrade`, `MDSSlotUpgrade`, `MDSCurrencyPickup`.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSPickup`. Also on the Room view's **Effects > Gameplay Objects** menu.

Use it in your game: Keys, consumables, charms, collectibles. Set `item` to an `MDSItemData` (or make one inline) and `amount`. Persistent pickups are gone once taken; enemy drops aren't. Its grant (`item:<id>`) counts for the Progress tab, so `item:` gates and barriers know where the key is.

Inspector properties:

Property	Type	Default	What it does
`item`	MDSItemData		The item given (`MDSItemData`).
`amount`	int	`1`	How many.
`persistent`	bool	`true`	Gone for good once taken (saved). Off for drops.
`auto_collect`	bool	`true`	Touching collects it; off: press interact at it.
`magnet_radius`	float	`0.0`	A player this close (px) pulls it in (0: never).
`magnet_speed`	float	`520.0`	*(not documented)*
`radius`	float	`16.0`	Its touch radius (px).
`color`		`Color(0.95, 0.8, 0.4)`	*(not documented)*

Signals (Node dock > Signals, to connect): `collected(id: String, amount: int)`

15. `MDSItemData`

Is a: `MDSItemData` → `Resource` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/resources/mds_item_data.gd`

Any pickup that is not an ability: keys, consumables, equipment (`MDSEquipmentData`), upgrades and collectibles. `id` is what `MDSGameplay.add_item` counts and what `item:<id>` (or `item:<id>>=<n>`) conditions test.

Add it in Godot: FileSystem dock > right-click a folder > **Create New > Resource...** > search `MDSItemData` > Save as `.tres`. Then drag the file onto the property that takes it (or click that property > **New MDSItemData**).

Use it in your game: An item: `id`, name, description, icon, category (key, consumable, upgrade, equipment, collectible), stack size, price, and an `effect` when used. Reference it from pickups, shops and drop tables.

Inspector properties:

Property	Type	Default	What it does
`id`	String	`""`	The id requirements and code use (`item:<id>`).
`display_name`	String	`""`	Name shown in menus and notices.
`description`	String (multiline)	`""`	*(not documented)*
`icon`	Texture2D		*(not documented)*
`category`	Category	`Category.KEY`	KEY, CONSUMABLE, UPGRADE, EQUIPMENT or COLLECTIBLE: its inventory tab and behaviour.
`max_stack`	int	`99`	How many can be carried (more are not picked up).
`value`	int	`0`	What it sells for, or costs in a shop without a price of its own.
`effect`	MDSItemEffect		What `MDSGameplay.use_item` does with it (consumables), or null.

16. `MDSBreakable`

Is a: `MDSBreakable` → `StaticBody2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/world/mds_breakable.gd`

A wall, floor or block that breaks: solid terrain (layer 1) with an `MDSHurtbox`. Only attacks whose `MDSAttackData.breaks` list its `kind` damage it (`wall`: any attack; `cracked_floor`: a ground pound; `reinforced`: the super attack; any name of your own), and only once `requires` is met. After `hits` such hits it breaks and stays broken (`MDSWorldGame.store_object`: gone when its room loads again, and after a save and load). A secret wall is a `wall` with no requirement.

It writes `requires` as `mds_requires` metadata, so the scanner ties it to the nearest exit and the Progress tab knows what opens it. Its look is a block of `size` drawn with cracks (its other children, when it has any, are its look instead).

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSBreakable`. Also on the Room view's **Effects > Gameplay Objects** menu.

Use it in your game: A wall or floor that breaks under attacks whose `breaks` list its `kind` (`wall`: any attack; `cracked_floor`: a ground pound; `reinforced`: the super attack). Set `hits`, an optional `requires`, and its `size`. Broken walls stay broken. Great for hiding upgrades.

Inspector properties:

Property	Type	Default	What it does
`kind`	String	`"wall"`	Which attacks break it: `wall` (any), `cracked_floor` (ground pound), `reinforced` (super attack), or your own.
`hits`	int	`1`	Hits needed to break it.
`requires`	PackedStringArray	`[]`	Conditions to meet before hits count.
`break_effect`	PackedScene		A scene spawned where it breaks (debris).
`size`		`Vector2(64, 128)`	The block's size (px), centred on the node.
`color`		`Color(0.42, 0.37, 0.32)`	*(not documented)*

Signals (Node dock > Signals, to connect): `broken`, `damaged(left: int)`, `deflected(info: MDSDamageInfo)`

17. `MDSSwitch`

Is a: `MDSSwitch` → `Area2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/world/mds_switch.gd`

A lever, a pressure plate or a crystal to strike: it sets the story flag `flag` (`MDSGameplay.set_flag`), so barriers and gates that require `flag:<name>` open, objectives complete and dialogue changes. `trigger`: hit it with an attack (HIT), interact at it (INTERACT) or step on it (STEP). `mode`: ONE_SHOT stays on (kept by the flag, also after a save and load), TOGGLE flips it on and off, TIMED turns it off again after `time` seconds.

It writes `mds_grants = ["flag:<name>"]` metadata, so the analysis treats it as granting the flag.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSSwitch`. Also on the Room view's **Effects > Gameplay Objects** menu.

Use it in your game: A lever, plate or crystal that sets story flag `flag`. `trigger`: HIT (strike it), INTERACT (press Up) or STEP; `mode`: ONE_SHOT, TOGGLE or TIMED. Barriers and gates with `flag:<name>` open when it flips.

Inspector properties:

Property	Type	Default	What it does
`flag`	String	`""`	The story flag it sets (`flag:<name>` opens barriers).
`mode`	Mode	`Mode.ONE_SHOT`	ONE_SHOT (stays on), TOGGLE or TIMED.
`time`	float	`5.0`	Seconds a TIMED switch stays on.
`trigger`	Trigger	`Trigger.HIT`	HIT (strike it), INTERACT (press Up) or STEP (stand on it).
`prompt`	String	`"Pull"`	Shown over it while the player can interact (INTERACT).
`size`		`Vector2(40, 48)`	*(not documented)*
`on_color`		`Color(0.5, 1.0, 0.6)`	*(not documented)*
`off_color`		`Color(0.85, 0.5, 0.35)`	*(not documented)*

Signals (Node dock > Signals, to connect): `switched(on: bool)`

18. `MDSHealthUpgrade`

Is a: `MDSHealthUpgrade` → `MDSPickup` → `Area2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/world/mds_health_upgrade.gd`

A health fragment: every `fragments_per_upgrade` of them raise maximum health by one (`MDSGameplay.raise_max_health`). They are counted as the item `health_fragment` (so `item:health_fragment>=4` works) and are persistent.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSHealthUpgrade`. Also on the Room view's **Effects > Gameplay Objects** menu.

Use it in your game: A health fragment: every `fragments_per_upgrade` (4) raise maximum health by one. Counted as item `health_fragment`, so `item:health_fragment>=4` works in requirements.

Inspector properties (plus those of `MDSPickup`):

Property	Type	Default	What it does
`fragments_per_upgrade`	int	`4`	*(not documented)*

19. `MDSManaUpgrade`

Is a: `MDSManaUpgrade` → `MDSPickup` → `Area2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/world/mds_mana_upgrade.gd`

A mana fragment: every `fragments_per_upgrade` of them raise maximum mana by `mana_per_upgrade`. Counted as the item `mana_fragment`; persistent.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSManaUpgrade`. Also on the Room view's **Effects > Gameplay Objects** menu.

Inspector properties (plus those of `MDSPickup`):

Property	Type	Default	What it does
`fragments_per_upgrade`	int	`3`	*(not documented)*
`mana_per_upgrade`	int	`33`	*(not documented)*

20. `MDSSlotUpgrade`

Is a: `MDSSlotUpgrade` → `MDSPickup` → `Area2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/world/mds_slot_upgrade.gd`

A notch for equipment: one more slot (`MDSGameplay.equip_slots`) for charms. Counted as the item `equip_slot`; persistent.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSSlotUpgrade`. Also on the Room view's **Effects > Gameplay Objects** menu.

Inspector properties (plus those of `MDSPickup`):

No Inspector properties of its own.

21. `MDSCurrencyPickup`

Is a: `MDSCurrencyPickup` → `MDSPickup` → `Area2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/world/mds_currency_pickup.gd`

Currency (Hollow Knight's geo): `amount` of it, pulled to the player who comes near, never persistent. Enemies drop it.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSCurrencyPickup`. Also on the Room view's **Effects > Gameplay Objects** menu.

Inspector properties (plus those of `MDSPickup`):

No Inspector properties of its own.

22. `MDSCheckpoint`

Is a: `MDSCheckpoint` → `Area2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/world/mds_checkpoint.gd`

A checkpoint in a room of hazards: touched, it is where spikes and pits (`MDSHazard` RESPAWN, lava) send the player back to, in this room, instead of the last safe ground. It is not a save point (benches are) and nothing is saved.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSCheckpoint`. Also on the Room view's **Effects > Gameplay Objects** menu.

Use it in your game: In rooms full of spikes or pits: touched, it's where hazards send the player back to (instead of the last safe ground). Not a save point.

Inspector properties:

Property	Type	Default	What it does
`size`		`Vector2(48, 96)`	*(not documented)*
`respawn_offset`		`Vector2(0, -32)`	Where the player is put back, from the node (its feet): up by about half the player's height.
`color`		`Color(0.55, 0.85, 1.0)`	*(not documented)*

Signals (Node dock > Signals, to connect): `reached`

23. `MDSHazard`

Is a: `MDSHazard` → `Area2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/combat/mds_hazard.gd`

Spikes, thorns or a pit: an area of `size` (centred on the node) with an `MDSHitbox` of the HAZARD team. DAMAGE hurts the player and knocks it back; RESPAWN hurts it, then puts it back on the last safe ground (or the `MDSCheckpoint` touched in the room), as Hollow Knight's spikes do. While the player touches it no ground counts as safe. The player's down attack in the air bounces off it (pogo). Nothing is saved: benches stay the only save points.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSHazard`. Also on the Room view's **Effects > Gameplay Objects** menu.

Use it in your game: Spikes, thorns or a pit: an area of `size`. RESPAWN mode hurts and sends the player back to safe ground (Hollow Knight's spikes); DAMAGE only hurts. A down attack bounces off it (pogo).

Inspector properties:

Property	Type	Default	What it does
`mode`	Mode	`Mode.RESPAWN`	RESPAWN (hurt and back to safe ground) or DAMAGE (hurt and knocked back).
`damage`	int	`1`	Health lost per touch.
`knockback`		`Vector2(240, -340)`	*(not documented)*
`size`		`Vector2(128, 32)`	The hazard's area (px, centred).
`look`	Look	`Look.SPIKES`	SPIKES, THORNS, PIT or NONE (your own art).
`color`		`Color(0.78, 0.78, 0.82)`	*(not documented)*

24. `MDSClimbable`

Is a: `MDSClimbable` → `Area2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/world/mds_climbable.gd`

A ladder or vines: with the `climb` ability, up or down while in it climbs. A marker of `size` (centred on the node); nothing is saved.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSClimbable`. Also on the Room view's **Effects > Gameplay Objects** menu.

Inspector properties:

Property	Type	Default	What it does
`size`		`Vector2(40, 240)`	*(not documented)*
`look`	Look	`Look.VINES`	*(not documented)*
`color`		`Color(0.35, 0.6, 0.3)`	*(not documented)*

25. `MDSSwimZone`

Is a: `MDSSwimZone` → `Area2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/world/mds_swim_zone.gd`

Water the player is in: with the `swim` ability it swims freely, without it it sinks slowly and moves slower. `MDSWater` makes one over its rectangle when its `MDSWater.swimmable` is on; add one yourself for water of your own. `size` is centred on the node.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSSwimZone`. Also on the Room view's **Effects > Gameplay Objects** menu.

Inspector properties:

Property	Type	Default	What it does
`size`		`Vector2(480, 200)`	*(not documented)*

26. `MDSGrapplePoint`

Is a: `MDSGrapplePoint` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/world/mds_grapple_point.gd`

A ring the player's grapple hooks (`grapple` ability): the nearest one in reach and in sight when grapple is pressed pulls the player to it. A marker: nothing is saved.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSGrapplePoint`. Also on the Room view's **Effects > Gameplay Objects** menu.

Inspector properties:

Property	Type	Default	What it does
`enabled`	bool	`true`	*(not documented)*
`radius`	float	`14.0`	*(not documented)*
`color`		`Color(0.75, 0.9, 1.0)`	*(not documented)*

27. `MDSLoreTablet`

Is a: `MDSLoreTablet` → `Area2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/world/mds_lore_tablet.gd`

An ancient stone tablet that can be inspected by the player. Reading it adds a Journal entry (flag `journal_<tablet_id>`) containing its title and text, and presents the lore through the game's `MDSDialogueBox` (or a toast).

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSLoreTablet`. Also on the Room view's **Effects > Gameplay Objects** menu.

Inspector properties:

Property	Type	Default	What it does
`tablet_id`	String	`""`	*(not documented)*
`title`	String	`""`	*(not documented)*
`text`	String (multiline)	`""`	*(not documented)*
`prompt`	String	`"Inspect"`	*(not documented)*
`size`		`Vector2(32, 48)`	*(not documented)*
`tablet_color`		`Color(0.6, 0.62, 0.65)`	*(not documented)*

Signals (Node dock > Signals, to connect): `read_tablet`

28. `MDSCorpse`

Is a: `MDSCorpse` → `Area2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/world/mds_corpse.gd`

Where the player last died, holding the currency it carried (death penalty CORPSE, like Hollow Knight's shade). `MDSGameplay` places it when the room of `MDSGameplay.corpse` loads; touching it gives the currency back (`MDSGameplay.recover_corpse`). Dying again before that loses it. Not saved by itself: the kit's save keeps `ext.gameplay.corpse`.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSCorpse`. Also on the Room view's **Effects > Gameplay Objects** menu.

Inspector properties:

Property	Type	Default	What it does
`currency`	int	`0`	The currency it holds (shown over it).
`radius`	float	`22.0`	*(not documented)*
`color`		`Color(0.62, 0.66, 0.95)`	*(not documented)*
`bob_time`	float	`2.4`	Seconds a full bob up and down takes.

Signals (Node dock > Signals, to connect): `recovered(amount: int)`

3. Combat

How hits are made, taken and felt.

29. `MDSAttackController`

Is a: `MDSAttackController` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/combat/mds_attack_controller.gd`

The player's attacks, a child of `MDSPlayer`: they run alongside its movement (it can attack while running and jumping), each by its `MDSAttackData` frame data at 60 fps (startup, active, recovery) with one `MDSHitbox` placed in front, above or below.

- `attack`: the ground combo (`ground_combo`, 3 hits; the next one follows when pressed during the recovery and listed in `MDSAttackData.cancel_into`), the air combo (2 hits), up with `move_up` held, down in the air with `move_down` held. A down hit on an enemy, a hazard or a breakable bounces the player up (pogo).
- Hold `attack` for `charge_time` with the `charge_attack` ability: the charged attack on release.
- `special`: the `spell` (an `MDSProjectile`), costing its mana.
- With the `super_attack` ability every attack also breaks `reinforced` breakables; with `parry`, an attack pressed within `parry_window` of being hit blocks the hit and stuns the attacker. Hits give mana (`mana_per_hit`), freeze the game a moment (hit-stop) and shake the camera a little.

Add it in Godot: Part of the player or an enemy: they make it when it's missing. Add your own with **Add Child Node** > search `MDSAttackController` to set its properties.

Use it in your game: The player's attacks (a child of `MDSPlayer`): a 3-hit ground combo, air combo, up and down attacks (down = pogo), charge attack, spell (`special`) and parry. Replace the default combos by filling `ground_combo`, `air_combo`... with `MDSAttackData` resources.

Inspector properties:

Property	Type	Default	What it does
`ground_combo`	Array[MDSAttackData]	`[]`	*(not documented)*
`air_combo`	Array[MDSAttackData]	`[]`	*(not documented)*
`up_attack`	MDSAttackData		*(not documented)*
`down_attack`	MDSAttackData		*(not documented)*
`charge_attack`	MDSAttackData		*(not documented)*
`spell`	MDSAttackData		*(not documented)*
`pogo_velocity`	float	`520.0`	Upward speed of a pogo bounce.
`mana_per_hit`	int	`3`	*(not documented)*
`charge_time`	float	`0.6`	*(not documented)*
`parry_window`	float	`0.12`	*(not documented)*
`blocked_states`	PackedStringArray	`["Locked", "Rest", "Dead", "Hurt", "LedgeGrab", "LedgeClimb", "Climb", "Grapple", "Slide", "Dash", "AirDash", "GroundPound", "Swim"]`	States the player can't attack in.

Signals (Node dock > Signals, to connect): `attack_started(attack: MDSAttackData)`, `attack_ended(attack: MDSAttackData)`, `hit(target: Node, info: MDSDamageInfo)`, `parried(info: MDSDamageInfo)`

30. `MDSAttackData`

Is a: `MDSAttackData` → `Resource` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/resources/mds_attack_data.gd`

One attack: its damage, its timing in frames at 60 fps (startup, then the hitbox is active, then recovery) and what it can follow into (`cancel_into`: attack ids, for combos). `breaks` lists the `MDSBreakable` kinds it breaks (`wall`, `cracked_floor`, `reinforced`...).

Add it in Godot: FileSystem dock > right-click a folder > **Create New > Resource...** > search `MDSAttackData` > Save as `.tres`. Then drag the file onto the property that takes it (or click that property > **New MDSAttackData**).

Use it in your game: One attack's damage, knockback, hit-stop and frame data (startup, active, recovery frames at 60 fps), its hitbox size and offset, what breakables it `breaks`, and what it can `cancel_into` (combos).

Inspector properties:

Property	Type	Default	What it does
`id`	String	`"slash"`	*(not documented)*
`damage`	int	`1`	Damage per hit.
`damage_type`	String	`"physical"`	physical, fire, ice, shock, void, environment (or any name resistances use).
`knockback`		`Vector2(220, -140)`	Push given to what it hits: x away from the attacker, y up (negative).
`hitstop_ms`	int	`60`	How long the game freezes when it lands (ms; see the hit-stop feel setting).
`startup_frames`	int	`3`	Frames before the hitbox is active (60 fps).
`active_frames`	int	`5`	Frames the hitbox hits.
`recovery_frames`	int	`10`	Frames after, before the next attack (combos cancel here).
`cancel_into`	PackedStringArray	`[]`	Attacks it can be cancelled into during its recovery, for combos.
`breaks`	PackedStringArray	`["wall"]`	Breakable kinds it breaks.
`direction`	Direction	`Direction.FORWARD`	FORWARD, UP or DOWN (pogo).
`hitbox_size`		`Vector2(56, 40)`	The hitbox, from the attacker's middle, facing right (mirrored when facing left).
`hitbox_offset`		`Vector2(40, 0)`	*(not documented)*
`mana_cost`	int	`0`	Mana it costs (spells).
`projectile`	PackedScene		Fired as well as the hitbox (spells, sub-weapons): an `MDSProjectile` scene.
`one_hit_per_target`	bool	`true`	Hits each target once per swing.

31. `MDSHealth`

Is a: `MDSHealth` → `Node` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/combat/mds_health.gd`

Health of the player, an enemy, a boss or a breakable: a child of it. Hits come through its `MDSHurtbox` (`apply_damage`), from MDS's environment effects (lava, steam vents, rockfall call `mds_environment_hurt` or `take_damage` on the body, which forward here) and from scripts. A hit does `max(1, round(amount * resistance(type) * multiplier) - defense)`, then makes the owner invulnerable for `iframe_time` seconds and flashes its look white.

Add it in Godot: Part of the player or an enemy: they make it when it's missing. Add your own with **Add Child Node** > search `MDSHealth` to set its properties.

Use it in your game: Health of anything that can be hurt: player, enemy, boss. Reads `stats`; `iframe_time` (0.8 s on the player) makes it invulnerable after a hit. Signals `damaged`, `healed`, `died` and `health_changed` are what you connect to for effects and UI.

Inspector properties:

Property	Type	Default	What it does
`stats`	MDSCombatStats		Its maximum health, defense and resistances. Empty: 5 health, no defense.
`iframe_time`	float	`0.8`	Seconds of invulnerability after a hit (0: none, enemies usually).
`flash_on_hit`	bool	`true`	Flash the look white on a hit (`visual`, with the hit-flash shader).
`visual`	CanvasItem		What flashes. Empty: the owner's "Visual" child, else the owner.
`invulnerable`	bool	`false`	Takes no damage at all (debug, cutscenes).
`damage_taken_multiplier`	float	`1.0`	Damage taken is multiplied by this (the options' assist mode sets it on the player).

Signals (Node dock > Signals, to connect): `damaged(info: MDSDamageInfo)`, `healed(amount: int)`, `died`, `health_changed(current: int, max: int)`

32. `MDSHitbox`

Is a: `MDSHitbox` → `Area2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/combat/mds_hitbox.gd`

What hits: an Area2D with a collision shape that, while `active`, hits the `MDSHurtbox`es of the other team it overlaps with its `attack`. A PLAYER hitbox (an attack) is on the `player_hitbox` layer and finds `enemy_hurtbox`es; an ENEMY or HAZARD one (a claw, an enemy's touch, spikes) is on `enemy_hitbox` and finds the `player_hurtbox`. A team never hurts itself. With `one_hit_per_target` each target is hit once until `reset_hits` (a new swing); without it (an enemy's touch), a target is hit again as soon as its invulnerability ends.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSHitbox`.

Inspector properties:

Property	Type	Default	What it does
`attack`	MDSAttackData		*(not documented)*
`team`	Team	`Team.ENEMY`	*(not documented)*
`one_hit_per_target`	bool	`true`	*(not documented)*
`active`	bool	`true`	*(not documented)*
`source`	Node		Who attacks (the knockback pushes away from it). Empty: the parent.

Signals (Node dock > Signals, to connect): `hit_landed(target: Node, info: MDSDamageInfo)`

33. `MDSHurtbox`

Is a: `MDSHurtbox` → `Area2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/combat/mds_hurtbox.gd`

Where something can be hit: an Area2D with a collision shape, on the `player_hurtbox` layer (team PLAYER) or the `enemy_hurtbox` layer (ENEMY, NEUTRAL: enemies, breakables, switches). `MDSHitbox`es of the other teams find it and give it their hit (`receive`): its `health` takes it, times `damage_multiplier` (a weak spot, an armoured back). A body with `mds_take_hit(info) -> int` (a breakable, a switch) takes the hit itself instead.

Add it in Godot: Part of the player or an enemy: they make it when it's missing. Add your own with **Add Child Node** > search `MDSHurtbox` to set its properties.

Inspector properties:

Property	Type	Default	What it does
`health`	MDSHealth		The health hits go to. Empty: the body's `MDSHealth`.
`team`	Team	`Team.ENEMY`	*(not documented)*
`damage_multiplier`	float	`1.0`	*(not documented)*

Property	Type	Default	What it does
`enabled`	bool	`true`	*(not documented)*

Signals (Node dock > Signals, to connect): `hurt(info: MDSDamageInfo)`

34. `MDSKnockback`

Is a: `MDSKnockback` → `Node` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/combat/mds_knockback.gd`

Pushes its `body` back when it is hit, and stuns it for `hitstun_time`: the player's Hurt state and enemies wait for `is_stunned` to end. `resistance` (and the body's knockback resistance in its stats) takes some of the push away.

Add it in Godot: Part of the player or an enemy: they make it when it's missing. Add your own with **Add Child Node** > search `MDSKnockback` to set its properties.

Inspector properties:

Property	Type	Default	What it does
`body`	CharacterBody2D		The body pushed. Empty: the parent.
`resistance`	float (range)	`0.0`	0 to 1: how much of the push is taken away.
`hitstun_time`	float	`0.2`	*(not documented)*

Signals (Node dock > Signals, to connect): `knocked(vector: Vector2)`

35. `MDSProjectile`

Is a: `MDSProjectile` → `MDSHitbox` → `Area2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/combat/mds_projectile.gd`

A shot: spells and sub-weapons of the player (fired on `special`, costing mana), a turret's bolt. It flies at `velocity` (falling with `drop`), hits like an `MDSHitbox` of its team, and is gone after `lifetime` seconds, after `pierce` + 1 hits, or on touching terrain (unless `passes_walls`). Without a shape of its own it is a `radius` circle, drawn as a glowing ball.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSProjectile`.

Inspector properties (plus those of `MDSHitbox`):

Property	Type	Default	What it does
`velocity`		`Vector2(600, 0)`	*(not documented)*
`drop`	float	`0.0`	Pull down on it (px/s²; an arrow, a lobbed bomb).
`lifetime`	float	`2.0`	*(not documented)*
`pierce`	int	`0`	Targets it goes through before it is gone (0: the first one stops it).
`passes_walls`	bool	`false`	*(not documented)*
`radius`	float	`10.0`	*(not documented)*
`color`		`Color(0.65, 0.85, 1.0)`	*(not documented)*

36. `MDSDamageNumber`

Is a: `MDSDamageNumber` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/combat/mds_damage_number.gd`

A hit's damage, floating up from the body it hurt and fading. `MDSHealth` shows one for each hit while the kit's `feel/damage_numbers` setting is on (off by default). It is put in the body's parent, so it stays where the hit was as the body moves on.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSDamageNumber`.

Inspector properties:

Property	Type	Default	What it does
`rise`	float	`36.0`	How far (px) it rises, and in how many seconds it is gone.
`lifetime`	float	`0.7`	*(not documented)*
`offset`		`Vector2(0, -24)`	Where it starts, from the body's origin.
`jitter`	float	`6.0`	How far (px) to each side it may start, so numbers of quick hits don't overlap.
`color`		`Color(1.0, 0.92, 0.6)`	*(not documented)*
`player_color`		`Color(1.0, 0.45, 0.4)`	The color of the player's own damage.
`font_size`	int	`18`	Before the options' text size.

4. Enemies and bosses

Ready-made enemies, their data, and boss fights.

37. `MDSEnemy`

Is a: `MDSEnemy` → `CharacterBody2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/enemies/mds_enemy.gd`

An enemy: a CharacterBody2D in the `enemy` group (the scanner counts it, dark rooms light it, the camera director watches it) on the `enemy` layer, running on the same state machine as the player. Its parts are made when it starts if missing: an `MDSHealth` (from `data`'s stats), an `MDSHurtbox`, a contact `MDSHitbox` (touching it hurts), an `MDSKnockback`, an `MDSPlayerDetector` (sight with a line of sight through terrain) and an `MDSStateMachine` with the states of its kind (`_make_states`).

When it dies it drops its loot (currency and `MDSEnemyData.drops`, never persistent), counts for the bestiary and XP, and by `respawn`: NEVER is recorded (`MDSWorldGame.mark_defeated`: gone for good, like a boss), ON_REST comes back after the player rests on a bench, ON_ROOM_LOAD comes back whenever its room loads.

The templates are `MDSWalker`, `MDSFlyer`, `MDSTurret`, `MDSCharger` and `MDSAmbusher`; bosses are `MDSBoss`.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSEnemy`. Also on the Room view's **Effects > Gameplay Objects** menu.

Use it in your game: Base of every enemy. Usually you use a template (`gameplay/enemies/templates/*.tscn`) instead and set `data` (an `MDSEnemyData` for health, damage and drops) and `respawn` (NEVER, ON_REST, ON_ROOM_LOAD). For your own enemy, extend it and return your states from `_make_states()`.

Inspector properties:

Property	Type	Default	What it does
`data`	MDSEnemyData		Its `MDSEnemyData` (stats, drops, bestiary).
`respawn`	Respawn	`Respawn.ON_REST`	ON_ROOM_LOAD, ON_REST (after the player rests on a bench) or NEVER.
`facing`	int	`-1`	1 facing right, -1 left.
`body_size`		`Vector2(40, 36)`	Its collision body (px).
`gravity`	float	`1400.0`	*(not documented)*
`uses_gravity`	bool	`true`	*(not documented)*
`max_fall_speed`	float	`900.0`	*(not documented)*
`contact_damage`	int	`-1`	Damage of touching it (-1: its stats' contact_damage).
`dead_time`	float	`0.6`	Seconds it lingers after dying.
`look`	Look	`Look.CRAWLER`	Placeholder look drawn in code when it has no art.
`body_color`		`Color(0.72, 0.36, 0.3)`	*(not documented)*
`resume_state`	StringName	`&""`	The state it goes back to after being hurt (empty: the first state).

Signals (Node dock > Signals, to connect): `died`, `hurt(info: MDSDamageInfo)`

38. `MDSEnemyData`

Is a: `MDSEnemyData` → `Resource` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/resources/mds_enemy_data.gd`

An enemy's definition (`MDSEnemy`): its stats, what it drops, its currency and XP, and its bestiary entry (the pause menu's Bestiary lists the enemies defeated).

Add it in Godot: FileSystem dock > right-click a folder > **Create New > Resource...** > search `MDSEnemyData` > Save as `.tres`. Then drag the file onto the property that takes it (or click that property > **New MDSEnemyData**).

Use it in your game: An enemy kind: id, name, `stats` (health, contact damage), currency it drops, a `drops` table, XP and bestiary text. Share one `.tres` between all enemies of that kind.

Inspector properties:

Property	Type	Default	What it does
`id`	String	`""`	*(not documented)*
`display_name`	String	`""`	*(not documented)*
`stats`	MDSCombatStats		Health, contact damage, defense (`MDSCombatStats`).
`drops`	MDSDropTable		Its `MDSDropTable`.
`currency_drop`		`Vector2i(2, 5)`	Currency dropped: between x and y.
`xp`	int	`0`	Experience given, with the LEVELS progression.
`bestiary_text`	String (multiline)	`""`	*(not documented)*

39. `MDSDropTable`

Is a: `MDSDropTable` → `Resource` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/resources/mds_drop_table.gd`

Loot: each entry {item: MDSItemData (or an item id), chance: 0 to 1, min, max} is rolled on its own when an enemy dies (`roll`).

Add it in Godot: FileSystem dock > right-click a folder > **Create New > Resource...** > search `MDSDropTable` > Save as `.tres`. Then drag the file onto the property that takes it (or click that property > **New MDSDropTable**).

Inspector properties:

Property	Type	Default	What it does
`entries`	Array[Dictionary]	`[]`	*(not documented)*

40. `MDSWalker`

Is a: `MDSWalker` → `MDSEnemy` → `CharacterBody2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/enemies/templates/mds_walker.gd`

An enemy that walks: Patrol (to and fro, turning at walls and ledges), Turn, Chase (toward the player it sees, stopping at ledges), Attack (a lunge when close), Hurt, Dead.

Add it in Godot: Drag `addons/MetroidvaniaDeveloperSystem/gameplay/enemies/templates/mds_walker.tscn` from the FileSystem dock into a room scene (or Instantiate Child Scene).

Use it in your game: Patrols, chases when it sees the player, lunges. Drag `templates/mds_walker.tscn` onto a floor in a room.

Inspector properties (plus those of `MDSEnemy`):

Property	Type	Default	What it does
`patrol_speed`	float	`70.0`	*(not documented)*
`chase_speed`	float	`150.0`	*(not documented)*
`attack_range`	float	`70.0`	*(not documented)*
`lunge_speed`	float	`300.0`	*(not documented)*
`windup_time`	float	`0.3`	*(not documented)*

41. `MDSFlyer`

Is a: `MDSFlyer` → `MDSEnemy` → `CharacterBody2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/enemies/templates/mds_flyer.gd`

An enemy that flies: Hover (bobbing about where it started), Swoop (a dive at where the player was), Return (back home), Hurt, Dead. No gravity.

Add it in Godot: Drag `addons/MetroidvaniaDeveloperSystem/gameplay/enemies/templates/mds_flyer.tscn` from the FileSystem dock into a room scene (or Instantiate Child Scene).

Use it in your game: Hovers where it starts and swoops at the player. Place it in the air (`templates/mds_flyer.tscn`).

Inspector properties (plus those of `MDSEnemy`):

Property	Type	Default	What it does
`hover_radius`	float	`40.0`	*(not documented)*
`hover_speed`	float	`60.0`	*(not documented)*
`swoop_speed`	float	`320.0`	*(not documented)*
`swoop_time`	float	`1.1`	*(not documented)*
`swoop_cooldown`	float	`1.4`	*(not documented)*

42. `MDSCharger`

Is a: `MDSCharger` → `MDSEnemy` → `CharacterBody2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/enemies/templates/mds_charger.gd`

An enemy that charges: Idle (until it sees the player), Windup (it shakes, telegraphing, for `windup_time`), Charge (straight ahead at `charge_speed`), Stunned (it hit a wall: dazed and open to hits for `stun_time`), Hurt, Dead.

Add it in Godot: Drag `addons/MetroidvaniaDeveloperSystem/gameplay/enemies/templates/mds_charger.tscn` from the FileSystem dock into a room scene (or Instantiate Child Scene).

Use it in your game: Waits, telegraphs, then charges; stunned when it hits a wall (a chance to strike back).

Inspector properties (plus those of `MDSEnemy`):

Property	Type	Default	What it does
`windup_time`	float	`0.6`	*(not documented)*
`charge_speed`	float	`440.0`	*(not documented)*
`max_charge_time`	float	`1.8`	*(not documented)*
`stun_time`	float	`1.4`	*(not documented)*

43. `MDSTurret`

Is a: `MDSTurret` → `MDSEnemy` → `CharacterBody2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/enemies/templates/mds_turret.gd`

An enemy fixed in place that shoots: Idle (until it sees the player), Aim (turning toward it for `aim_time`), Fire (an `MDSProjectile` at it, then `fire_interval` before the next), Hurt, Dead. No gravity, no knockback.

Add it in Godot: Drag `addons/MetroidvaniaDeveloperSystem/gameplay/enemies/templates/mds_turret.tscn` from the FileSystem dock into a room scene (or Instantiate Child Scene).

Use it in your game: Fixed in place, aims and fires projectiles at the player in sight. Mount it on a wall or ledge with a clear line of sight.

Inspector properties (plus those of `MDSEnemy`):

Property	Type	Default	What it does
`projectile`	PackedScene		The shot (`MDSProjectile` scene). Empty: a plain bolt.
`projectile_speed`	float	`380.0`	*(not documented)*
`projectile_damage`	int	`1`	*(not documented)*
`aim_time`	float	`0.45`	*(not documented)*
`fire_interval`	float	`1.6`	*(not documented)*

44. `MDSAmbusher`

Is a: `MDSAmbusher` → `MDSEnemy` → `CharacterBody2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/enemies/templates/mds_ambusher.gd`

An enemy that lies in wait: Hidden (in the ground, harmless and unhittable), Reveal (it bursts out when the player comes within `reveal_range`), Attack (a lunge at the player), Retreat (back into the ground when the player is far), Dead.

Add it in Godot: Drag `addons/MetroidvaniaDeveloperSystem/gameplay/enemies/templates/mds_ambusher.tscn` from the FileSystem dock into a room scene (or Instantiate Child Scene).

Use it in your game: Hidden until the player comes close, then bursts out.

Inspector properties (plus those of `MDSEnemy`):

Property	Type	Default	What it does
`reveal_range`	float	`150.0`	*(not documented)*
`lunge_speed`	float	`320.0`	*(not documented)*
`hide_after`	float	`2.5`	*(not documented)*

45. `MDSPlayerDetector`

Is a: `MDSPlayerDetector` → `Area2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/enemies/mds_player_detector.gd`

An enemy's eyes: a circle of `sight_range` around it, and a line of sight through terrain (layer 1). `sees_player` is true while the player is in it and nothing solid is between them (with `needs_line_of_sight`).

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSPlayerDetector`. Also on the Room view's **Effects > Gameplay Objects** menu.

Inspector properties:

Property	Type	Default	What it does
`sight_range`	float	`360.0`	*(not documented)*
`needs_line_of_sight`	bool	`true`	*(not documented)*
`view_half_angle`	float (range)	`180.0`	Only in front of the enemy (its facing), with this half angle in degrees (180: all round).

Signals (Node dock > Signals, to connect): `player_seen(player: Node2D)`, `player_lost`

46. `MDSSpawner`

Is a: `MDSSpawner` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/enemies/mds_spawner.gd`

Spawns waves of enemies (an arena's adds, a gauntlet): wave i brings `waves enemies, taken in turn from` enemies `(scenes of` MDSEnemy`; empty: walkers), no more than` max_alive `at once, within` spread `px of it. The next wave starts`

when one is cleared. Spawned enemies come back whenever the room loads (ON_ROOM_LOAD): nothing is saved. It starts on its own (autostart), when arena's fight starts, or by start`.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSSpawner`. Also on the Room view's **Effects > Gameplay Objects** menu.

Use it in your game: Waves of enemies (an arena's adds, a gauntlet): set the scenes and how many per wave.

Inspector properties:

Property	Type	Default	What it does
`enemies`	Array[PackedScene]	`[]`	*(not documented)*
`waves`	PackedInt32Array	`[3]`	*(not documented)*
`max_alive`	int	`3`	*(not documented)*
`spread`	float	`200.0`	*(not documented)*
`interval`	float	`0.6`	Seconds between two spawns.
`autostart`	bool	`false`	*(not documented)*
`arena`	MDSBossArena		Starts with this arena's fight.

Signals (Node dock > Signals, to connect): `wave_started(index: int)`, `wave_cleared(index: int)`, `finished`

47. `MDSBoss`

Is a: `MDSBoss` → `MDSEnemy` → `CharacterBody2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/enemies/mds_boss.gd`

A boss: an `MDSEnemy` in the `boss` group, recorded by `boss_name` when defeated (`respawn` is always NEVER, and `mds_boss_name` metadata names it, so the scanner, `MDSWorldGame.is_boss_defeated` and `boss:<name>` objectives know it). Defeated, it is gone when its room loads again.

It waits (Dormant) until its `MDSBossArena` starts the fight (`start_fight`), then rests a moment between attacks, each picked from its current phase's `MDSBossPhase.attack_states`: Swipe (walks up and strikes), Charge (rushes across), Slam (leaps at the player and crashes down with a shockwave), Volley (a fan of shots). At each phase's `MDSBossPhase.health_threshold` it staggers (Stagger, its enter animation, a camera punch-in) and fights on with that phase's attacks at its speed. Without phases it has two: Swipe and Charge, then from half its health Slam, Volley and Charge, faster.

With an `MDSDialogueBox` in the game it speaks: `intro_dialogue` as the fight starts and `defeat_dialogue` as it falls, named `boss_name` (the game waits meanwhile).

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSBoss`. Also on the Room view's **Effects > Gameplay Objects** menu.

Use it in your game: A boss enemy: set `boss_name` (it must match the room's `boss` on the map so `boss:<name>` requirements and objectives work), its `data` (health), and optionally `phases` (`MDSBossPhase`: at what health, which attacks, how fast). Defeated, it's gone for good.

Inspector properties (plus those of `MDSEnemy`):

Property	Type	Default	What it does
`boss_name`	String	`""`	Its name; must match the room's `boss` on the map (`boss:<name>`).
`phases`	Array[MDSBossPhase]	`[]`	Its `MDSBossPhase`s. Empty: two phases (Swipe and Charge; from half health Slam, Volley and Charge, faster).
`arena`	Node		Its arena (which starts the fight). Empty: the fight starts when the player comes near.
`intro_dialogue`	MDSDialogueData		Said when the fight starts (with an `MDSDialogueBox` in the game).
`defeat_dialogue`	MDSDialogueData		Said when it falls, before it fades away.

Property	Type	Default	What it does
`music`	AudioStream		Its fight's music. Empty: the area's boss_music.
`walk_speed`	float	`110.0`	*(not documented)*
`charge_speed`	float	`520.0`	*(not documented)*
`leap_speed`	float	`640.0`	*(not documented)*
`volley_count`	int	`5`	*(not documented)*
`shot_speed`	float	`360.0`	*(not documented)*
`attack_damage`	int	`2`	Damage of its strikes.
`rest_time`	float	`0.9`	Seconds of rest between attacks (shorter in faster phases).
`wake_range`	float	`0.0`	Starts the fight on its own when the player comes this close, with no arena (0: never).

Signals (Node dock > Signals, to connect): `fight_started`, `phase_changed(index: int)`

48. `MDSBossPhase`

Is a: `MDSBossPhase` → `Resource` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/resources/mds_boss_phase.gd`

A phase of a boss (`MDSBoss`): it starts when the boss's health falls to `health_threshold` of its maximum (1: from the start), plays `enter_animation`, and attacks with `attack_states` (state names of the boss's state machine) at `speed_multiplier`.

Add it in Godot: FileSystem dock > right-click a folder > **Create New > Resource...** > search `MDSBossPhase` > Save as `.tres`. Then drag the file onto the property that takes it (or click that property > **New MDSBossPhase**).

Inspector properties:

Property	Type	Default	What it does
`health_threshold`	float (range)	`1.0`	Share of health at which the phase starts (1.0: from the start).
`attack_states`	PackedStringArray	`[]`	Attacks picked from in this phase (Swipe, Charge, Slam, Volley).
`speed_multiplier`	float	`1.0`	How much faster it is in this phase.
`enter_animation`	StringName	`&""`	*(not documented)*

49. `MDSBossArena`

Is a: `MDSBossArena` → `Area2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/enemies/mds_boss_arena.gd`

Where a boss fight happens: when the player walks into it (`size`, centred on the node), its `doors` close, the boss music starts (`MDSMusic.play_boss`: `music`, else the boss's, else the area's `boss_music`), the camera director frames the boss (`MDSCameraDirector.frame_threat`, or shows `frame_area`), the HUD shows the boss's health (`MDSGameplay.boss_started`) and the boss starts (`MDSBoss.start_fight`).

When the boss falls it is recorded by name (`MDSWorldGame.mark_defeated`), the music goes back to the area's (`MDSMusic.end_boss`), the director lets go, the doors open, `grants` (an ability) is granted, and `MDSGameplay.boss_ended` reports the win. If the player dies the arena resets when the room loads again, the boss at its first phase. A boss already defeated leaves the arena idle and its doors open.

Doors that should stay shut until the boss is beaten are `MDSGateBarrier`s with `requires = ["boss:<name>"]`.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSBossArena`. Also on the Room view's **Effects > Gameplay Objects** menu.

Use it in your game: An Area2D covering the fight: when the player walks in, `doors` close, boss music plays, the camera frames the boss, the HUD shows its health bar, and the boss starts. Assign `boss` and `doors`. Put an `MDSGateBarrier` with `requires = ["boss:<name>"]` on the way out.

Inspector properties:

Property	Type	Default	What it does
`boss`	MDSBoss		The boss this arena starts.
`doors`	Array[MDSArenaDoor]	`[]`	`MDSArenaDoor`s that close during the fight.
`frame_area`		`Rect2()`	What the camera shows when the fight starts (world px). Empty: the boss (and the player).
`size`		`Vector2(900, 480)`	The area that starts the fight when the player enters (px, centred).
`grants`	String	`""`	An ability granted for winning (empty: none).
`music`	AudioStream		Music for this fight (else the boss's, else the area's boss music).

Signals (Node dock > Signals, to connect): `fight_started(boss: MDSBoss)`, `fight_won(boss: MDSBoss)`

50. `MDSArenaDoor`

Is a: `MDSArenaDoor` → `StaticBody2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/enemies/mds_arena_door.gd`

A door of a boss arena (`MDSBossArena`): open (no collision, its bars up) until the fight starts, shut during it, open again after. `size` is centred on the node.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSArenaDoor`. Also on the Room view's **Effects > Gameplay Objects** menu.

Use it in your game: A door that shuts while a boss fight runs (assign it to the arena's `doors`). Its `size` should close the doorway.

Inspector properties:

Property	Type	Default	What it does
`size`		`Vector2(40, 180)`	*(not documented)*
`color`		`Color(0.35, 0.3, 0.32)`	*(not documented)*
`close_time`	float	`0.35`	*(not documented)*

Signals (Node dock > Signals, to connect): `shut`, `opened`

51. `MDSEnemyState`

Is a: `MDSEnemyState` → `MDSState` → `Node` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/enemies/mds_enemy_state.gd`

A state of an `MDSEnemy`'s state machine (patrol, chase, swoop, fire...). `enemy` is the enemy; its helpers (`MDSEnemy.walk`, `MDSEnemy.fall`, `MDSEnemy.sees_player`) are the shared moves. The templates (`MDSWalker`, `MDSFlyer`, `MDSTurret`, `MDSCharger`, `MDSAmbusher`) and `MDSBoss` define theirs; a game adds its own by extending this.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSEnemyState`. Also on the Room view's **Effects > Gameplay Objects** menu.

Inspector properties:

No Inspector properties of its own.

5. Building rooms: terrain and scenery

What rooms are made of. Mostly painted with the Room view, but every piece is a node or resource you can edit in the Inspector.

52. `MDSFreeform`

Is a: `MDSFreeform` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/mds_freeform.gd`

Freeform terrain: a closed outline drawn with curves instead of tiles, for organic caves (rounded ledges, bulging moss walls, arches). The scene stores only the control points and an `MDSFreeformStyle`; the visuals and collision are built when the node enters the tree, in the editor and in the game:

- a textured fill (repeating), an outline,
- strips along edges facing up (moss, grass) and down (drips, roots),
- clumps from the style's stamp set scattered along those edges,
- with `solid`, a StaticBody2D with the exact curved collision, on the style's collision layers, one-way for ledges (see `MDSFreeformStyle.role`). Made by the Room view's Freeform tool; can also be edited in the inspector.

A shape that continues into another one along a vertical line (Trace drawing cuts rock in two around a cave, since a shape has no holes) lists those lines' x (its own coordinates) in the metadata `mds_seams`; no outline is drawn along them.

An outline that crosses itself can't be filled or collided with: in the editor it is drawn red and reported (see `is_outline_simple`); the Room view's Repair fixes it.

Add it in Godot: Drawn with the **Room view** (Freeform tool, Trace drawing), or Scene dock > **Add Child Node** > search `MDSFreeform` under the room's `Freeform` node.

Use it in your game: Curved terrain with exact collision. Draw it with the Room view's Freeform tool, or let **Convert to freeform** turn a blockout into it. In the Inspector: `points` (the outline), `style` (an `MDSFreeformStyle`: the look and whether it's solid, a one-way ledge or decoration), `smooth`, and a collision override for a single shape.

Inspector properties:

Property	Type	Default	What it does
`points`	PackedVector2Array	`[]`	The outline's control points.
`style`	MDSFreeformStyle		Its look and collision role (`MDSFreeformStyle`).
`solid`	bool	`true`	Collides (terrain). Off for backgrounds and foregrounds.
`smooth`	bool	`true`	Round the outline through the points (Catmull-Rom). Off: straight edges.
`edge_clumps`	bool	`true`	Scatter the style's clumps along the edges.
`seed_value`	int	`0`	Seed for its random details (edge clumps).
Collision override			
`override_collision`	bool	`false`	Use the settings below instead of the style's role, layers and one-way.
`role`	MDSFreeformStyle.Role	`MDSFreeformStyle.Role.TERRAIN`	*(not documented)*
`collision_layer`	int (flags_2d_physics)	`1`	*(not documented)*
`collision_mask`	int (flags_2d_physics)	`1`	*(not documented)*
`one_way`	bool	`false`	One-way collision (with the override): a ledge jumped up through.
`one_way_margin`	float	`16.0`	*(not documented)*

53. `MDSFreeformStyle`

Is a: `MDSFreeformStyle` → `Resource` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/mds_freeform_style.gd`

Look of an `MDSFreeform` shape: a repeating fill texture, textured strips along its top edges (moss, grass...) and undersides (drips, roots), an outline, and clumps scattered along the edges from an `MDSStampSet`. Without textures the colors are used, so a style works with no art at all.

Save styles as `*.freeform.tres`: the Room view lists every such file in the project.

Add it in Godot: FileSystem dock > right-click a folder > **Create New > Resource...** > search `MDSFreeformStyle` > Save as `.tres`. Then drag the file onto the property that takes it (or click that property > **New MDSFreeformStyle**).

Use it in your game: The look of freeform terrain: fill texture or colour, outline, grass on top, roots underneath, edge clumps from a stamp set, and its role (terrain, one-way platform, decoration) and collision layers. The asset packs ship ready styles (`asset_packs/*/freeform/styles/*.tres`).

Inspector properties:

Property	Type	Default	What it does
`display_name`	String	`""`	*(not documented)*
Fill			
`fill_texture`	Texture2D		*(not documented)*
`fill_color`		`Color("#2a3442")`	*(not documented)*
`fill_scale`	float	`1.0`	World pixels per fill texture pixel (2 = texture drawn twice as big).
Outline			
`outline_color`		`Color("#080b10")`	*(not documented)*
`outline_width`	float	`3.0`	*(not documented)*
Top edges			
`top_texture`	Texture2D		Strip along edges that face up (within top_angle of straight up). The texture's top row is the outside.
`top_color`		`Color("#58a83e")`	*(not documented)*
`top_width`	float	`0.0`	*(not documented)*
`top_inset`	float	`6.0`	How far the strip's center sits inside the shape (px).
`top_angle`	float (range)	`55.0`	*(not documented)*
Bottom edges			
`bottom_texture`	Texture2D		Strip along edges that face down. The texture's top row is the inside.
`bottom_color`		`Color("#0b0f15")`	*(not documented)*
`bottom_width`	float	`0.0`	*(not documented)*
`bottom_inset`	float	`4.0`	*(not documented)*
Clumps			
`stamp_set`	MDSStampSet		*(not documented)*
`top_clumps`	String	`""`	Stamp category scattered along top edges ("" = none).
`bottom_clumps`	String	`""`	Stamp category hung under bottom edges ("" = none).
`clump_spacing`	float	`56.0`	Average distance between clumps (px).
`clump_scale`		`Vector2(0.7, 1.1)`	*(not documented)*
Material			

Property	Type	Default	What it does
`fill_material`	Material		A material for the fill (a shader skin: lit stone, ice, lava...). The fill texture and color still reach it as TEXTURE and COLOR.
`edge_material`	Material		A material for a band along the edges: a mesh `edge_inside` px into the shape and `edge_outside` px out of it, with UV.x along the edge (px), UV.y across it (0 at the outline, 1 at the inner edge, negative outside) and COLOR.r/g telling which way it faces (see `MDSEdgeBand`). The addon's shaders/terrain_skin.gdshader is an example.
`edge_inside`	float	`34.0`	*(not documented)*
`edge_outside`	float	`30.0`	*(not documented)*
`skip_edges_on_grid`	enum	`Vector2.ZERO`	Edges lying on lines of this grid (x = k *grid.x, y = k* grid.y, in the room's coordinates) get no band: where rock meets the room's outer boundary and runs on into the next room. Zero: every edge has one.
2.5D			
`extrude`	bool	`true`	Extruded by MDSDepth25D (solid terrain and platforms only).
`depth_side_color`		`Color(0, 0, 0, 0)`	Color of the extruded sides (alpha 0: the fill color, darkened).
`depth_top_color`		`Color(0, 0, 0, 0)`	Color of the receding top surfaces (alpha 0: the fill blended with the top color).
Collision			
`role`	Role	`Role.TERRAIN`	Terrain, platform or decoration (see `Role`).
`collision_layer`	int (flags_2d_physics)	`1`	Physics layers the shape's body is on.
`collision_mask`	int (flags_2d_physics)	`1`	Physics layers the shape's body detects.
`one_way`	bool	`false`	Collide only from above: bodies jump up through the shape and land on its top, like a ledge. Usually set with the Platform role.
`one_way_margin`	float	`16.0`	How deep (px) a body may sink into a one-way shape and still be pushed up onto it.
Defaults			
`solid`	bool	`true`	Collide by default (terrain) or not (backgrounds, foregrounds).
`default_layer`	String	`"terrain"`	Layer new shapes of this style go to: "terrain", "back" or "front".

54. `MDSStampSet`

Is a: `MDSStampSet` → `Resource` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/mds_stamp_set.gd`

Large sprites placed freely (not on the tile grid): moss clumps, leaf clusters, hanging moss, background bubbles, foreground silhouettes... One texture with regions, each with a category and an anchor (the point that sits on the spot you click, e.g. the bottom of a clump that grows up from an edge).

Save sets as `*.stamps.tres`: the Room view lists every such file.

Add it in Godot: FileSystem dock > right-click a folder > **Create New > Resource...** > search `MDSStampSet` > Save as `.tres`. Then drag the file onto the property that takes it (or click that property > **New MDSStampSet**).

Inspector properties:

Property	Type	Default	What it does
`display_name`	String	`""`	*(not documented)*
`texture`	Texture2D		*(not documented)*
`regions`	Array[Rect2]	`[]`	*(not documented)*

Property	Type	Default	What it does
`categories`	PackedStringArray	`[]`	*(not documented)*
`anchors`	PackedVector2Array	`[]`	Per region, 0..1 inside the region (0.5, 1 = bottom center).

55. `MDSDrawing`

Is a: `MDSDrawing` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/mds_drawing.gd`

A picture placed in a room as it is drawn: what the Room view's Trace drawing makes with "The drawing itself", or any image. The `region` of `texture` is drawn from this node's origin, scaled and turned by its transform.

Colors can be cut out: each pixel goes with the nearest of `keep_colors` and `cut_colors`, and is transparent when that is a cut one. Trace drawing cuts out the paper around a drawing, and splits it into pictures at three depths (behind the terrain, at the terrain's depth and in front of everything), each keeping the colors of its layer. The cut is a shader (shaders/drawing_cutout.gdshader) on an internal sprite, so the scene stores only these values.

A drawing has no collision: Trace drawing gives the terrain in it invisible freeform shapes (see `MDSFreeform`).

Add it in Godot: Drawn with the **Room view** (Freeform tool, Trace drawing), or Scene dock > **Add Child Node** > search `MDSDrawing` under the room's `Freeform` node.

Inspector properties:

Property	Type	Default	What it does
`texture`	Texture2D		*(not documented)*
`region`		`Rect2()`	The part of `texture` shown (px). Empty: all of it.
`keep_colors`		`PackedColorArray()`	Colors that stay.
`cut_colors`		`PackedColorArray()`	Colors made transparent where they are nearer than every kept color.
`softness`	float (range)	`0.35`	0: a hard cut halfway between a kept and a cut color; 1: soft across the blend between them.

56. `MDSBackdropView`

Is a: `MDSBackdropView` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/mds_backdrop_view.gd`

Draws an `MDSBackdrop` behind a room: the sky over the whole view and the planes behind the room are drawn off screen in a `SubViewport` at `MDSBackdrop.resolution` of the screen, looking through the same camera, and shown scaled up on a `CanvasLayer` far below the room's. Being a world of its own, the distance is never drawn into the room's world, and a dark room's darkness (lights, CanvasModulate on the room's canvas) doesn't reach it. Foreground planes are drawn in the room's world, in front of it.

Place it at the room's top-left corner with `room_size` its size (`MDSWorldGame` does this, with the backdrop of the room, of its area, or of the world). A plane of depth d moves at d of the camera's speed, around the room's middle.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSBackdropView`.

Inspector properties:

Property	Type	Default	What it does
`backdrop`	MDSBackdrop		The backdrop drawn (rebuilt when set).
`room_size`		`Vector2(1152, 648)`	Size of the room the planes are laid out around (its middle is where they rest).
`room`	Node		For `MDSBackdrop.sample_palette`: the room whose terrain colors tint the haze.

57. `MDSBackdrop`

Is a: `MDSBackdrop` → `Resource` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/mds_backdrop.gd`

The distance behind the rooms: a sky gradient and planes of art at increasing depths (`MDSBackdropLayer`), drawn by an `MDSBackdropView`. Made only of resources, so asset packs and the world file (per area or per room) can supply them. Save as `*.backdrop.tres`.

Add it in Godot: FileSystem dock > right-click a folder > **Create New > Resource...** > search `MDSBackdrop` > Save as `.tres`. Then drag the file onto the property that takes it (or click that property > **New MDSBackdrop**).

Inspector properties:

Property	Type	Default	What it does
`display_name`	String	`""`	*(not documented)*
`sky`	Gradient		The sky over the whole view, top (offset 0) to bottom (offset 1).
`layers`	Array[MDSBackdropLayer]	`[]`	Planes, farthest first.
`resolution`	float (range)	`0.5`	The distance is drawn off screen at this share of the screen's resolution and scaled up: it is soft anyway, and many translucent layers cost much less at half size.
`sample_palette`	bool	`false`	Mix the room's terrain colors (its freeform styles) into the haze, so the distance takes the room's palette.
`palette_mix`	float (range)	`0.3`	*(not documented)*

58. `MDSBackdropLayer`

Is a: `MDSBackdropLayer` → `Resource` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/mds_backdrop_layer.gd`

One plane of an `MDSBackdrop`: art at a depth, scrolling at a fraction of the camera's speed, drawn smaller, hazier and greyer the farther back it is.

Add it in Godot: FileSystem dock > right-click a folder > **Create New > Resource...** > search `MDSBackdropLayer` > Save as `.tres`. Then drag the file onto the property that takes it (or click that property > **New MDSBackdropLayer**).

Inspector properties:

Property	Type	Default	What it does
`source`	Source	`Source.TEXTURE`	*(not documented)*
`texture`	Texture2D		Source TEXTURE: an image repeated along the plane (every `spacing` px).
`stamp_set`	MDSStampSet		Source STAMPS: stamps of a category scattered along the plane.
`stamp_category`	String	`""`	*(not documented)*
`stamp_density`	float	`4.0`	Stamps per 1000 px of plane.
`stamp_scale`		`Vector2(0.8, 1.2)`	Random size of each stamp (min, max).
`scene`	PackedScene		Source SCENE: a scene placed on the plane (repeated with `repeat`).
Depth			
`depth`	float (range)	`0.3`	How the plane moves: 0 stays on the screen (infinitely far, like the sky), 1 moves with the room, above 1 runs faster than the camera (in front of the room).
`scale`	float	`1.0`	Size of the art (farther planes are usually drawn smaller).
`offset`	enum	`Vector2.ZERO`	Where the art sits on the plane, from the room's middle at rest (plane px). Positive y: lower.
`repeat`	bool	`true`	Repeat the art sideways so the view never runs off its edge.
`spacing`	float	`0.0`	Distance between repeats (0: the texture's width).
`jitter`	float	`0.0`	Random vertical shift of each stamp (px).

Property	Type	Default	What it does
`foreground`	bool	`false`	Drawn in front of the room (at full resolution, in its world) instead of behind it.
`seed_value`	int	`0`	*(not documented)*
Look			
`haze_color`		`Color(0.55, 0.6, 0.55)`	Color the plane is mixed toward (the air between it and the viewer).
`haze`	float (range)	`0.3`	*(not documented)*
`tint`	enum	`Color.WHITE`	Multiplies the art's colors after `desaturate`.
`desaturate`	float (range)	`0.0`	Greys the art out (0 = its own colors), so borrowed art takes the scene's colors.
`alpha`	float (range)	`1.0`	*(not documented)*
`sway`	float (range)	`0.0`	Sway in a slow draught (hanging things sway more the farther they hang).

59. `MDSDepth25D`

Is a: `MDSDepth25D` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/mds_depth_25d.gd`

2.5D for flat rooms, made only of their own 2D art. Add one to a room (or turn on `depth_25d` in the world settings and `MDSWorldGame` adds one to every room). Every frame it reads where the camera looks and fakes depth around that point:

- `Extruded terrain`: every wall, floor and platform gets the side faces of a solid block, running back toward the vanishing point (the middle of the view), so they swing as the camera moves. Solid freeform shapes (Terrain and Platform roles, styles with `MDSFreeformStyle.extrude`), the collision of tile layers, and static bodies. Decorations and background/foreground shapes are never extruded.
- `Tilted floors`: the tops of floors and ledges become planes receding into the screen, paved in perspective.
- `Light`: faces are lit from the key light and fall off with distance from the player; the view darkens away from the player and toward its edges.
- `Shadows`: bodies in `shadow_groups` cast soft shadows on the ground under them that spread and fade as they rise. Nothing here collides: it's all drawing. Colors come from the freeform styles (their fill and top textures or colors, or `MDSFreeformStyle.depth_side_color`), and from the tileset's art for tiles. No dependency on MetSys or game classes.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSDepth25D`.

Inspector properties:

Property	Type	Default	What it does
`style`	MDSDepthStyle		How the depth looks. Empty: the defaults of `MDSDepthStyle`.
`sources`	Array[Node]	`[]`	Nodes whose terrain is extruded. Empty: the parent (the room).
`player`	Node2D		The body the light follows. Empty: the first node in `player_group`.
`player_group`	StringName	`&"player"`	*(not documented)*
`shadow_groups`	Array[StringName]	`[&"player", &"enemy"]`	Bodies (CharacterBody2D, RigidBody2D) in these groups cast shadows.
`extrude_tiles`	bool	`true`	*(not documented)*
`extrude_bodies`	bool	`true`	*(not documented)*
`light_falloff`	bool	`true`	*(not documented)*
`preview_in_editor`	bool	`false`	Run in the editor (the Room view's 2.5D preview). Off: the node only works in the game.
`room_camera`	MDSRoomCamera		Where the camera looks, from an `MDSRoomCamera` (empty: the viewport's camera).

60. `MDSDepthStyle`

Is a: `MDSDepthStyle` → `Resource` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/mds_depth_style.gd`

How `MDSDepth25D` fakes depth: how far walls reach back, how faces are lit, how dark the view gets away from the player, shadows and floor paving. Save as a .tres to share it, or leave the node's style empty for these defaults.

Add it in Godot: FileSystem dock > right-click a folder > **Create New > Resource...** > search `MDSDepthStyle` > Save as `.tres`. Then drag the file onto the property that takes it (or click that property > **New MDSDepthStyle**).

Inspector properties:

Property	Type	Default	What it does
Extrusion			
`depth_x`	float (range)	`0.10`	How far back walls reach, as a fraction of the way to the vanishing point (the middle of the view), sideways...
`depth_y`	float (range)	`0.20`	...and up and down: deeper, so floors read as planes without side walls turning into tunnels.
`face_z`	int	`-7`	Draw order of the faces and shadows: behind the terrain and the stamps on it (plants, ivy: the Room view's StampsBack group is at z -6), in front of background shapes (FreeformBack, z -8), which stand farther back than the floor planes.
Lighting			
`light_dir`		`Vector2(0, -1)`	Where the key light comes from (faces turned toward it are lit, the others shaded).
`light_reach`	float	`900.0`	Faces this far (px) from the player are at their darkest.
`light_inner`	float	`260.0`	The view darkens from this distance (px) around the player...
`light_dark`		`Color(0.66, 0.66, 0.74)`	...to `light_dark` at `light_reach`.
`vignette`	float (range)	`0.3`	How much the view darkens toward its edges (0 = no vignette).
`light_z`	int	`25`	Draw order of the light falloff and vignette (multiplied over everything below it).
Shadows			
`shadow_strength`	float (range)	`0.7`	Opacity of the soft shadow under a body standing on the ground (they spread and fade as it rises).
`shadow_mask`	int (flags_2d_physics)	`1`	Physics layers the shadows fall on.
Floors			
`paving`	float	`44.0`	Distance (px) between the paving joints of floor tops.
`floor_courses`	int (range)	`3`	Courses of paving across a floor's depth.
`joints`	float (range)	`0.55`	Strength of the joint lines (0 = plain faces).
Colors			
`plain_side`		`Color(0.32, 0.31, 0.3)`	Sides and tops of terrain that has no freeform style (tile layers, static bodies).
`plain_top`		`Color(0.42, 0.42, 0.36)`	*(not documented)*
`plain_joint`		`Color(0.08, 0.08, 0.07)`	*(not documented)*

6. NPCs, dialogue, shops and equipment

Characters to talk to, conversations, buying and charms.

61. `MDSNPC`

Is a: `MDSNPC` → `Area2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/npc/mds_npc.gd`

A character to talk to. Standing at it shows `prompt` over it (like an `MDSGate`'s); pressing `interact` starts its conversation (`talk`) in the game's `MDSDialogueBox` (the node in group `mds_dialogue_box`).

`alt_dialogues` ({condition, dialogue}) replace `dialogue` as the story moves on: the first whose MDS condition string (`boss:Warden`, `flag:met_elder`, `item:red_key`, an ability) is met wins. Lines are skipped while their own condition is unmet and set their flag when shown; with no dialogue box in the game the flags are set at once. NPCs that move with the story can check a flag when their room loads, or use MDS's `carry_over` group.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSNPC`. Also on the Room view's **Effects > Gameplay Objects** menu.

Use it in your game: A character to talk to: standing at it shows `prompt`, Up starts `dialogue` in the game's dialogue box. `alt_dialogues` switch the conversation on conditions (a flag, an item).

Inspector properties:

Property	Type	Default	What it does
`npc_name`	String	`""`	Its name, shown with its lines.
`dialogue`	MDSDialogueData		What it says (`MDSDialogueData`).
`alt_dialogues`	Array[Dictionary]	`[]`	{"condition": String, "dialogue": MDSDialogueData}, the first met wins.
`prompt`	String	`"Talk"`	Shown over it while the player can talk ({name}: `npc_name`).
`size`		`Vector2(36, 64)`	*(not documented)*
`body_color`		`Color(0.55, 0.62, 0.78)`	*(not documented)*
`face_player`	bool	`true`	Turns toward the player standing at it.

Signals (Node dock > Signals, to connect): `talked(dialogue: MDSDialogueData)`, `conversation_ended`

62. `MDSDialogueTrigger`

Is a: `MDSDialogueTrigger` → `Area2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/world/mds_dialogue_trigger.gd`

A conversation with anything: put it on (or beside) a door, a chest, a statue, a sign, a lever, a sleeping boss, and the player talks to that. Standing in it shows `prompt` and pressing `interact` starts `dialogue` in the game's `MDSDialogueBox` (INTERACT), or the conversation starts as the player walks in (TOUCH).

`alt_dialogues` ({condition, dialogue}) replace `dialogue` as the story moves on: the first whose MDS condition string is met wins. While `condition` is unmet it stays silent. With `once` it has its say the first time only (kept like a collected item, also after a save and load). The other side of the conversation is `speaker_name`, else its parent's name in conversations (`get_dialogue_name()`: an NPC's, a boss's), else the parent node's name. Its area of `size` stands on its origin. Without a dialogue box in the game, the lines' flags are set at once.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSDialogueTrigger`. Also on the Room view's **Effects > Gameplay Objects** menu.

Use it in your game: Lets the player talk to anything: a door, a chest, a statue, a sign. Give it a collision shape and `dialogue`; INTERACT waits for Up, TOUCH starts when the player walks in.

Inspector properties:

Property	Type	Default	What it does
`dialogue`	MDSDialogueData		The conversation started.
`alt_dialogues`	Array[Dictionary]	`[]`	{"condition": String, "dialogue": MDSDialogueData}, the first met wins.
`trigger`	Trigger	`Trigger.INTERACT`	INTERACT (press Up in it) or TOUCH (walk into it).
`speaker_name`	String	`""`	Who speaks (empty: its parent's name, see the class description).
`prompt`	String	`"Look"`	Shown over it while the player can talk (INTERACT).
`once`	bool	`false`	Only the first time (kept in the save).
`condition`	String	`""`	Silent while this MDS condition string is unmet (empty: always talks).
`size`		`Vector2(64, 96)`	Its area, standing on its origin.

Signals (Node dock > Signals, to connect): `started(dialogue: MDSDialogueData)`, `ended`

63. `MDSDialogueData`

Is a: `MDSDialogueData` → `Resource` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/resources/mds_dialogue_data.gd`

A conversation: its lines in order (`MDSDialogueLine`), shown by `MDSDialogueBox`.

Add it in Godot: FileSystem dock > right-click a folder > **Create New > Resource...** > search `MDSDialogueData` > Save as `.tres`. Then drag the file onto the property that takes it (or click that property > **New MDSDialogueData**).

Use it in your game: A conversation: a list of `MDSDialogueLine`s, optionally with its own text reveal (letter by letter, word by word).

Inspector properties:

Property	Type	Default	What it does
`id`	String	`""`	*(not documented)*
`lines`	Array[MDSDialogueLine]	`[]`	The conversation's lines (`MDSDialogueLine`).
`reveal`	MDSTextReveal		How its lines come onto the screen (a line's own `MDSDialogueLine.reveal` wins). Empty: the dialogue box's.

64. `MDSDialogueLine`

Is a: `MDSDialogueLine` → `Resource` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/resources/mds_dialogue_line.gd`

One line of a conversation (`MDSDialogueData`). It is skipped while `condition` (an MDS condition string: `dash`, `item:red_key`, `flag:met_elder`, `boss:Warden`...) is unmet. Showing it sets `set_flag`. `choices` ({text, next: line index or -1 to end, set_flag}) let the player answer.

A line `from_player` is the player's side of the conversation: the dialogue box names it after the player (`MDSDialogueBox.player_name`) and shows its portrait on the right.

Add it in Godot: FileSystem dock > right-click a folder > **Create New > Resource...** > search `MDSDialogueLine` > Save as `.tres`. Then drag the file onto the property that takes it (or click that property > **New MDSDialogueLine**).

Inspector properties:

Property	Type	Default	What it does
`speaker`	String	`""`	Who says it.
`text`	String (multiline)	`""`	What is said.
`portrait`	Texture2D		*(not documented)*

Property	Type	Default	What it does
`from_player`	bool	`false`	The player says it (named and pictured on the right), not the one talked to.
`reveal`	MDSTextReveal		How it comes onto the screen (`MDSTextRevealLetters`, `MDSTextRevealWords`...). Empty: the conversation's, else the dialogue box's.
`condition`	String	`""`	Shown only when this condition is met (`flag:x`, `item:y`...).
`set_flag`	String	`""`	A story flag set when the line is shown.
`choices`	Array[Dictionary]	`[]`	Answers the player picks from.

65. `MDSShop`

Is a: `MDSShop` → `MDSNPC` → `Area2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/npc/mds_shop.gd`

A shopkeeper: talking to it says its `dialogue` (a greeting, if any), then opens the game's shop screen (the node in group `mds_shop_screen`, an `MDSShopScreen`) on its `inventory`. Each entry has a price (0: the item's `MDSItemData.value`), a stock (-1: endless) kept in the save (`ext.gameplay.shops`) and a condition (an MDS condition string) that hides it while unmet.

Bought items are added to the inventory (`MDSGameplay.add_item`); an UPGRADE with an effect (more health, more mana) applies it at once.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSShop`. Also on the Room view's **Effects > Gameplay Objects** menu.

Use it in your game: A shopkeeper NPC: set `inventory` (an `MDSShopInventory` of items, prices and stock). Talking opens the shop screen.

Inspector properties (plus those of `MDSNPC`):

Property	Type	Default	What it does
`inventory`	MDSShopInventory		What it sells (`MDSShopInventory`).
`shop_id`	String	`""`	Its key in the save. Empty: its object id (room/path).

Signals (Node dock > Signals, to connect): `bought(item: MDSItemData, price: int)`, `shop_opened`, `shop_closed`

66. `MDSShopInventory`

Is a: `MDSShopInventory` → `Resource` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/resources/mds_shop_inventory.gd`

What a shop (`MDSShop`) sells: each entry {item: MDSItemData, price (0: the item's value), stock (-1: endless), condition (an MDS condition string; empty: always)}. The stock left is kept in the save.

Add it in Godot: FileSystem dock > right-click a folder > **Create New > Resource...** > search `MDSShopInventory` > Save as `.tres`. Then drag the file onto the property that takes it (or click that property > **New MDSShopInventory**).

Inspector properties:

Property	Type	Default	What it does
`entries`	Array[Dictionary]	`[]`	*(not documented)*

67. `MDSEquipmentData`

Is a: `MDSEquipmentData` → `MDSItemData` → `Resource` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/resources/mds_equipment_data.gd`

A charm: equipped while resting on a bench (`MDSGameplay.equip`), it costs `slot_cost` of the player's `MDSGameplay.equip_slots`, changes stats while equipped (`stat_mods`: `max_health`, `max_mana`, `defense`, `damage`, or any `MDSMovementConfig` field, added) and lends abilities (`grants`) that are not kept when it is taken off.

Add it in Godot: FileSystem dock > right-click a folder > **Create New > Resource...** > search `MDSEquipmentData` > Save as `.tres`. Then drag the file onto the property that takes it (or click that property > **New MDSEquipmentData**).

Use it in your game: A charm: its `slot_cost`, stat changes (`stat_mods`: damage, max_health, max_run_speed...) and abilities it lends while worn (`grants`).

Inspector properties (plus those of `MDSItemData`):

Property	Type	Default	What it does
`slot_cost`	int	`1`	Equipment slots it takes.
`stat_mods`	Dictionary	`{}`	Stat -> amount added while equipped ({"max_run_speed": 40, "damage": 1}).
`grants`	PackedStringArray	`[]`	Abilities active while equipped (not written to `MDSWorldGame.abilities`).

68. `MDSItemEffect`

Is a: `MDSItemEffect` → `Resource` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/resources/mds_item_effect.gd`

What using an item does (`MDSGameplay.use_item`): override `apply`. The built-ins are `MDSHealEffect`, `MDSMaxHealthEffect`, `MDSManaEffect` and `MDSBuffEffect`.

Add it in Godot: FileSystem dock > right-click a folder > **Create New > Resource...** > search `MDSItemEffect` > Save as `.tres`. Then drag the file onto the property that takes it (or click that property > **New MDSItemEffect**).

Inspector properties:

No Inspector properties of its own.

69. `MDSHealEffect`

Is a: `MDSHealEffect` → `MDSItemEffect` → `Resource` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/resources/mds_heal_effect.gd`

Restores `amount` health (`MDSHealth.heal`).

Add it in Godot: FileSystem dock > right-click a folder > **Create New > Resource...** > search `MDSHealEffect` > Save as `.tres`. Then drag the file onto the property that takes it (or click that property > **New MDSHealEffect**).

Inspector properties:

Property	Type	Default	What it does
`amount`	int	`1`	*(not documented)*

70. `MDSMaxHealthEffect`

Is a: `MDSMaxHealthEffect` → `MDSItemEffect` → `Resource` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/resources/mds_max_health_effect.gd`

Raises maximum health by `amount` (kept in the save, see `MDSGameplay.max_health`) and fills it.

Add it in Godot: FileSystem dock > right-click a folder > **Create New > Resource...** > search `MDSMaxHealthEffect` > Save as `.tres`. Then drag the file onto the property that takes it (or click that property > **New MDSMaxHealthEffect**).

Inspector properties:

Property	Type	Default	What it does
`amount`	int	`1`	*(not documented)*

71. `MDSManaEffect`

Is a: `MDSManaEffect` → `MDSItemEffect` → `Resource` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/resources/mds_mana_effect.gd`

Restores `amount` mana, and raises maximum mana by `max_up`.

Add it in Godot: FileSystem dock > right-click a folder > **Create New > Resource...** > search `MDSManaEffect` > Save as `.tres`. Then drag the file onto the property that takes it (or click that property > **New MDSManaEffect**).

Inspector properties:

Property	Type	Default	What it does
`amount`	int	`33`	*(not documented)*
`max_up`	int	`0`	*(not documented)*

72. `MDSBuffEffect`

Is a: `MDSBuffEffect` → `MDSItemEffect` → `Resource` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/resources/mds_buff_effect.gd`

For `duration` seconds, adds `amount` to a stat of the player: `damage`, `defense`, or a field of its `MDSMovementConfig` (`max_run_speed`...). See `MDSPlayer.add_buff`.

Add it in Godot: FileSystem dock > right-click a folder > **Create New > Resource...** > search `MDSBuffEffect` > Save as `.tres`. Then drag the file onto the property that takes it (or click that property > **New MDSBuffEffect**).

Inspector properties:

Property	Type	Default	What it does
`stat`	String	`"damage"`	*(not documented)*
`amount`	float	`1.0`	*(not documented)*
`duration`	float	`30.0`	*(not documented)*

73. `MDSLevelCurve`

Is a: `MDSLevelCurve` → `Resource` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/resources/mds_level_curve.gd`

Experience levels, with the LEVELS progression setting: reaching level n takes `base_xp * growth^(n - 2)` more XP than level n - 1, and each level past the first gives `health_per_level` health and `damage_per_level` damage.

Add it in Godot: FileSystem dock > right-click a folder > **Create New > Resource...** > search `MDSLevelCurve` > Save as `.tres`. Then drag the file onto the property that takes it (or click that property > **New MDSLevelCurve**).

Inspector properties:

Property	Type	Default	What it does
`base_xp`	int	`20`	*(not documented)*
`growth`	float	`1.5`	*(not documented)*
`max_level`	int	`30`	*(not documented)*
`health_per_level`	int	`1`	*(not documented)*

Property	Type	Default	What it does
`damage_per_level`	float	`0.0`	*(not documented)*

74. `MDSTextReveal`

Is a: `MDSTextReveal` → `Resource` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/resources/mds_text_reveal.gd`

How a line of dialogue comes onto the screen in an `MDSDialogueBox`. This one shows the whole line at once; `MDSTextRevealLetters` types it letter by letter and `MDSTextRevealWords` word by word. Set one on the dialogue box (every line), on an `MDSDialogueData` (a conversation: a boss speaking word by word) or on an `MDSDialogueLine` (one line).

Another style is a script extending this one that overrides `visible_characters`: The player's `MDSOptions.text_speed` speeds every style up or down (Instant shows lines whole), and pressing an advance action shows the rest of the line.

Add it in Godot: FileSystem dock > right-click a folder > **Create New > Resource...** > search `MDSTextReveal` > Save as `.tres`. Then drag the file onto the property that takes it (or click that property > **New MDSTextReveal**).

Inspector properties:

No Inspector properties of its own.

75. `MDSTextRevealLetters`

Is a: `MDSTextRevealLetters` → `MDSTextReveal` → `Resource` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/resources/mds_text_reveal_letters.gd`

Types the line out letter by letter (a typewriter), with a beat after punctuation.

Add it in Godot: FileSystem dock > right-click a folder > **Create New > Resource...** > search `MDSTextRevealLetters` > Save as `.tres`. Then drag the file onto the property that takes it (or click that property > **New MDSTextRevealLetters**).

Inspector properties:

Property	Type	Default	What it does
`letters_per_second`	float (range)	`45.0`	Letters shown per second.
`pause_after_stop`	float (range)	`0.25`	Seconds the typing waits after . ! ? and …
`pause_after_comma`	float (range)	`0.08`	Seconds it waits after , ; : and dashes.

76. `MDSTextRevealWords`

Is a: `MDSTextRevealWords` → `MDSTextReveal` → `Resource` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/resources/mds_text_reveal_words.gd`

Shows the line a word at a time, with a beat after the words that end in punctuation. A word comes with the spaces after it.

Add it in Godot: FileSystem dock > right-click a folder > **Create New > Resource...** > search `MDSTextRevealWords` > Save as `.tres`. Then drag the file onto the property that takes it (or click that property > **New MDSTextRevealWords**).

Inspector properties:

Property	Type	Default	What it does
`words_per_second`	float (range)	`6.0`	Words shown per second.
`pause_after_stop`	float (range)	`0.3`	Seconds it waits after a word ending in . ! ? or …
`pause_after_comma`	float (range)	`0.1`	Seconds it waits after a word ending in , ; : or a dash.

7. Game UI (the Gameplay Kit's screens)

Create game scene with *Include gameplay kit* adds these under the game's UI CanvasLayer. You mostly restyle them.

77. `MDSHud`

Is a: `MDSHud` → `Control` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/ui/mds_hud.gd`

The heads-up display, in the game's UI layer: the player's health as masks, its mana as a bar, its currency (and level, with LEVELS progression) top left, and the boss's name and health at the bottom while a boss fight is on (`MDSGameplay.boss_started` to `MDSGameplay.boss_ended`). MDS's minimap stays where Create game scene put it. It reads the game through `MDSGameplayUi.game_for`; delete it to use your own.

Add it in Godot: Under a `CanvasLayer` of the game (Create game scene puts the UI in `UI`): Scene dock > **Add Child Node** (Ctrl+A) > search `MDSHud`.

Use it in your game: Health masks, mana bar, currency, a boss health bar during fights. Restyle with its colours and sizes.

Inspector properties:

Property	Type	Default	What it does
`mask_size`	float	`26.0`	*(not documented)*
`mask_gap`	float	`6.0`	*(not documented)*
`full_color`		`Color(0.95, 0.93, 0.88)`	*(not documented)*
`empty_color`		`Color(0.25, 0.24, 0.28, 0.8)`	*(not documented)*
`mana_color`		`Color(0.55, 0.7, 1.0)`	*(not documented)*
`mana_px`	float	`1.6`	Mana bar width per point of mana.
`margin`		`Vector2(24, 20)`	*(not documented)*
`boss_bar_size`		`Vector2(560, 14)`	*(not documented)*

78. `MDSToast`

Is a: `MDSToast` → `Control` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/ui/mds_toast.gd`

Short notices at the top of the screen, one after another: what the kit announces (`MDSGameplay.notice`: an item picked up, "Requires Dash" when a gate won't let the player through, a level), and "Map updated" when a bench saves rooms visited since the last save. `show_text` adds one of your own.

Add it in Godot: Under a `CanvasLayer` of the game (Create game scene puts the UI in `UI`): Scene dock > **Add Child Node** (Ctrl+A) > search `MDSToast`.

Inspector properties:

Property	Type	Default	What it does
`show_time`	float	`2.2`	Seconds each notice stays.
`fade_time`	float	`0.25`	*(not documented)*
`max_queue`	int	`6`	Notices waiting beyond this are dropped (the oldest first).
`map_updated_text`	String	`"Map updated"`	*(not documented)*

79. `MDSAbilityPopup`

Is a: `MDSAbilityPopup` → `Control` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/ui/mds_ability_popup.gd`

Shown when the player gains an ability (`MDSGameplay` calls `show_ability`): its icon, its name, what it does and the key or button of its input action. The game pauses until it is closed (jump, attack, interact or accept, after `min_time`). Put it under the game's UI CanvasLayer (Create game scene does); delete it for no popups.

Add it in Godot: Under a `CanvasLayer` of the game (Create game scene puts the UI in `UI`): Scene dock > **Add Child Node** (Ctrl+A) > search `MDSAbilityPopup`.

Inspector properties:

Property	Type	Default	What it does
`pause_game`	bool	`true`	Pause the game while it shows.
`confirm_actions`	PackedStringArray	`["ui_accept", "jump", "attack", "interact"]`	*(not documented)*
`min_time`	float	`0.6`	Seconds before it can be closed (so a held button doesn't skip it).
`gamepad_glyphs`	bool	`false`	Always show the gamepad button rather than the key. Off: the options' input glyphs choose (`MDSOptions.use_gamepad_glyphs`: keyboard, gamepad, or the gamepad when one is connected).

Signals (Node dock > Signals, to connect): `closed`

80. `MDSPauseMenu`

Is a: `MDSPauseMenu` → `MDSGameplayScreen` → `Control` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/ui/mds_pause_menu.gd`

The pause menu: `pause` (Esc, Start) opens it and closes it, `map` (Tab, Back) opens it on the map. Its tabs, switched with `menu_prev` and `menu_next` (Q / E, LB / RB): Map (a full-screen `MDSWorldMapView` of what was explored, `MDSWorldMapView.follow_player` off), Inventory (`MDSInventoryScreen`), Equipment (`MDSEquipmentScreen`), Journal (lore tablets read: flags `journal_<id>`), Bestiary (enemies defeated), Options (`MDSOptionsMenu`) and Quit. The game is paused while it is open. It opens only while nothing else holds the game (no other screen, not between rooms).

On the map the player writes pins of their own (`pin_editor`, an `MDSMapPinEditor`: a title, a short description, a kind): a click on the map opens it for a pin there, Pin where I am (a gamepad's way) for one where the player stands, and a click on one of their own pins to change it or Remove it. Markers the game put (a boss room, a quest's target) only show. The map zooms and moves (`MDSWorldMapView.zoomable`): the wheel or RT / LT zoom (out below the whole map too), a drag or the right stick moves it, and it opens on the player. A marker pointed at (the mouse over it, or in the middle of the map) shows its description.

Add it in Godot: Under a `CanvasLayer` of the game (Create game scene puts the UI in `UI`): Scene dock > **Add Child Node** (Ctrl+A) > search `MDSPauseMenu`.

Use it in your game: Esc/Start opens it; Tab/Back opens it on the Map tab (zoom, pins and markers there). Tabs: Map, Inventory, Equipment, Journal, Bestiary, Options, Quit. Set `quit_scene` to your title scene.

Inspector properties (plus those of `MDSGameplayScreen`):

Property	Type	Default	What it does
`quit_scene`	String (file)	`""`	The scene Quit goes to (a title scene). Empty: the game's `MDSTitleScreen`, else quit the game.

81. `MDSMapPinEditor`

Is a: `MDSMapPinEditor` → `PanelContainer` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/ui/mds_map_pin_editor.gd`

A box to write a pin of the player's own on the map: its title, a short description and its kind (`kinds`: a flag, a note, a danger...). `MDSPauseMenu`'s map opens it on a click on the map, on Pin where I am, and on a click on one of the player's own pins (to change it, or Remove it). OK (or Enter in the title) says `confirmed`, Cancel (or Esc, B) `cancelled`. It works with the keyboard and the gamepad (its fields are in the focus order; on a phone the screen keyboard opens for them).

In a menu of your own: `open_new` or `open_edit`, then put the pin with `MDSWorldGame.add_map_marker` when it is confirmed.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSMapPinEditor`.

Inspector properties:

Property	Type	Default	What it does
`kinds`	PackedStringArray	`["pin", "note", "danger", "treasure", "key", "quest"]`	The kinds the player picks from (see `MDSWorldMapView.marker_kinds`).
`title_length`	int	`40`	The longest title.

Signals (Node dock > Signals, to connect): `confirmed(title: String, description: String, kind: String)`, `removed`, `cancelled`

82. `MDSDialogueBox`

Is a: `MDSDialogueBox` → `MDSGameplayScreen` → `Control` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/ui/mds_dialogue_box.gd`

Shows a conversation (`MDSDialogueData`) at the bottom of the screen: the speaker's name and portrait, the text coming in, and the line's choices as buttons. How the text comes is a `MDSTextReveal`: letter by letter (`MDSTextRevealLetters`), word by word (`MDSTextRevealWords`), at once (`MDSTextReveal`) or a style of your own, set here for every line (`reveal`), on a conversation or on a line; the player's Text speed option (`MDSOptions.text_speed`) scales it. `advance_actions` show the rest of the line, then go to the next one. Lines whose condition is unmet are skipped; a line shown sets its flag, a choice picked sets its own and goes to its `next` line (-1: the end).

The other side of the conversation is whoever started it: an `MDSNPC` or `MDSShop`, an `MDSBoss` (its intro and its defeat), an `MDSLoreTablet`, an `MDSDialogueTrigger` on any object, named by its `get_dialogue_name()`. Lines `MDSDialogueLine.from_player` are the player's, named `player_name` with `player_portrait` on the right. They find it in group `mds_dialogue_box`. The game is paused while it is open.

Add it in Godot: Under a `CanvasLayer` of the game (Create game scene puts the UI in `UI`): Scene dock > **Add Child Node** (Ctrl+A) > search `MDSDialogueBox`.

Inspector properties (plus those of `MDSGameplayScreen`):

Property	Type	Default	What it does
`reveal`	MDSTextReveal		How lines come onto the screen when neither the line nor its conversation says. Empty: letter by letter at `chars_per_second`.
`chars_per_second`	float	`45.0`	Letters typed per second when `reveal` is empty (0: the whole line at once).
`use_text_speed_option`	bool	`true`	The player's Text speed option (`MDSOptions.text_speed`) scales every line's pace.
`advance_actions`	PackedStringArray	`["ui_accept", "interact", "jump", "attack"]`	*(not documented)*
`box_height`	float	`190.0`	*(not documented)*
Player			
`player_name`	String	`"You"`	The name over the player's lines (`MDSDialogueLine.from_player`).

Property	Type	Default	What it does
`player_portrait`	Texture2D		The portrait beside the player's lines, on the right (a line's own portrait wins).

Signals (Node dock > Signals, to connect): `line_shown(index: int, line: MDSDialogueLine)`, `choice_made(index: int, choice: Dictionary)`, `characters_shown(count: int)`, `finished`

83. `MDSShopScreen`

Is a: `MDSShopScreen` → `MDSGameplayScreen` → `Control` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/ui/mds_shop_screen.gd`

The screen an `MDSShop` opens (it finds it in group `mds_shop_screen`): what is on sale with prices and stock, the item's description, the currency carried. Accept buys the focused entry; cancel leaves. The game is paused while it is open.

Add it in Godot: Under a `CanvasLayer` of the game (Create game scene puts the UI in `UI`): Scene dock > **Add Child Node** (Ctrl+A) > search `MDSShopScreen`.

Inspector properties (plus those of `MDSGameplayScreen`):

No Inspector properties of its own.

84. `MDSInventoryScreen`

Is a: `MDSInventoryScreen` → `HBoxContainer` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/ui/mds_inventory_screen.gd`

The pause menu's Inventory tab: the items carried by category (keys, consumables, upgrades, collectibles; equipment has its own tab), with the focused item's description. Accept uses a consumable (`MDSGameplay.use_item`).

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSInventoryScreen`.

Inspector properties:

No Inspector properties of its own.

85. `MDSEquipmentScreen`

Is a: `MDSEquipmentScreen` → `HBoxContainer` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/ui/mds_equipment_screen.gd`

The pause menu's Equipment tab: the charms owned with their slot costs, the ones worn marked, the slots used, and the focused charm's effects. Accept puts a charm on or takes it off; away from a bench (setting `equip_at_bench_only`) it is read-only.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSEquipmentScreen`.

Inspector properties:

No Inspector properties of its own.

86. `MDSOptionsMenu`

Is a: `MDSOptionsMenu` → `VBoxContainer` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/ui/mds_options_menu.gd`

The options (the pause menu's Options tab, and the title screen's): a volume per audio bus, the window, accessibility (game speed 50 to 100%, assist mode, screen shake and flash intensity, camera motion, text size, input glyphs, toggled glide) and the controls, each action rebindable for keyboard and gamepad. Every change applies at once and is kept in `user://mds_options.cfg` (`MDSOptions`). Sliders move with left and right; the other rows change with accept.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSOptionsMenu`.

Inspector properties:

Property	Type	Default	What it does
`actions`	PackedStringArray	`[]`	Actions listed for rebinding (empty: the kit's).

Signals (Node dock > Signals, to connect): `changed(option: String)`

87. `MDSTitleScreen`

Is a: `MDSTitleScreen` → `MDSGameplayScreen` → `Control` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/ui/mds_title_screen.gd`

The title and save slot selection screen: Continue or New game for 3 slots, via `MDSWorldGame.has_save_game`, `MDSWorldGame.read_save_game`, `MDSWorldGame.load_game` and `MDSWorldGame.new_game`.

Add it in Godot: Under a `CanvasLayer` of the game (Create game scene puts the UI in `UI`): Scene dock > **Add Child Node** (Ctrl+A) > search `MDSTitleScreen`.

Inspector properties (plus those of `MDSGameplayScreen`):

Property	Type	Default	What it does
`slot_count`	int	`3`	Number of save slots (default 3).
`title_text`	String	`""`	Title text displayed on the screen (empty: uses world name).

Signals (Node dock > Signals, to connect): `slot_selected(slot_index: int, is_new: bool)`

88. `MDSGameplayScreen`

Is a: `MDSGameplayScreen` → `Control` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/ui/mds_gameplay_screen.gd`

The base of the kit's screens that take over the game (pause menu, dialogue, shop, title): it covers the screen, pauses the game while open (`pause_game`; screens opened over one another unpause it when the last one closes), keeps running while paused, gives the focus to its first control (every screen works with only a gamepad or keyboard) and closes on `close_actions`.

Add it in Godot: Under a `CanvasLayer` of the game (Create game scene puts the UI in `UI`): Scene dock > **Add Child Node** (Ctrl+A) > search `MDSGameplayScreen`.

Inspector properties:

Property	Type	Default	What it does
`pause_game`	bool	`true`	Pauses the game while it is open.
`close_actions`	PackedStringArray	`["ui_cancel"]`	Actions that close it (empty: only code closes it).

Signals (Node dock > Signals, to connect): `opened`, `closed`

89. `MDSAreaTitle`

Is a: `MDSAreaTitle` → `Control` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/mds_area_title.gd`

An area's name, faded in near the top of the screen when the player enters the area (the way Hollow Knight and Silksong title their areas), held, and faded out. Put it in the UI (a CanvasLayer) of an `MDSWorldGame` scene.

The text is the area's `title` in the world file (else its name) and its `subtitle` under it.

Add it in Godot: Under a `CanvasLayer` of the game (Create game scene puts the UI in `UI`): Scene dock > **Add Child Node** (Ctrl+A) > search `MDSAreaTitle`.

Inspector properties:

Property	Type	Default	What it does
`game`	MDSWorldGame		The game whose areas are titled. Empty: `MDSWorldGame.instance`.
`first_visit_only`	bool	`false`	Only the first time each area is entered.
`fade_time`	float	`0.8`	*(not documented)*
`hold_time`	float	`2.5`	*(not documented)*
`title_size`	int	`44`	*(not documented)*
`subtitle_size`	int	`18`	*(not documented)*
`title_color`		`Color(0.95, 0.92, 0.84)`	*(not documented)*
`height`	float (range)	`0.14`	Where the title sits, as a share of the screen's height from its top.

Signals (Node dock > Signals, to connect): `shown(area: String)`

90. `MDSObjectiveBanner`

Is a: `MDSObjectiveBanner` → `Control` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/mds_objective_banner.gd`

A banner with the area's objective (`areas[name].objective` in the world file), shown when the player arrives in an area whose objective isn't done, and again, marked done, when it is completed (`MDSWorldGame.objective_completed`). Put it in the UI (a CanvasLayer) of an `MDSWorldGame` scene.

Add it in Godot: Under a `CanvasLayer` of the game (Create game scene puts the UI in `UI`): Scene dock > **Add Child Node** (Ctrl+A) > search `MDSObjectiveBanner`.

Inspector properties:

Property	Type	Default	What it does
`game`	MDSWorldGame		The game whose objectives are shown. Empty: `MDSWorldGame.instance`.
`every_visit`	bool	`false`	Show it every time an area is entered, done or not (off: only while it isn't done).
`heading`	String	`"Objective"`	*(not documented)*
`done_heading`	String	`"Objective complete"`	*(not documented)*
`fade_time`	float	`0.5`	*(not documented)*
`hold_time`	float	`4.0`	*(not documented)*
`text_color`		`Color(0.95, 0.9, 0.75)`	*(not documented)*
`done_color`		`Color(0.65, 1.0, 0.6)`	*(not documented)*

8. Presentation: camera moments, music and level views

Polish that sits on top of a working game.

91. `MDSCameraDirector`

Is a: `MDSCameraDirector` → `Node` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/mds_camera_director.gd`

Borrows an `MDSRoomCamera`'s camera for a moment when a fight needs framing, then gives it back exactly where the room's own camera would be. Add it as a child of the MDSRoomCamera, or call `MDSRoomCamera.get_director`.

- `focus_on`: hold on a node (a death, a finishing blow), optionally in slow motion.
- `frame_threat`: pull out until the player and a threat off screen are both in view.
- `reveal`: pull out to show an area (a boss's whole arena).
- `punch_in`: a short push in.
- `frame_world`: show an area beyond the room (the whole level, for `MDSLevelOverview`).

A focus beats a threat or a reveal, which beat a punch. Every view stays inside the room: a pull-out never shows anything outside the room's shape (the notches of an L or T room), and views at the room's zoom or closer stay inside a camera zone. Zoom and position glide there and back (cut, with the room camera's `MDSRoomCamera.transition_style` CUT). It works with both of the room camera's backends: with Camera2D it moves the camera itself, with Phantom Camera it takes over through a PhantomCamera2D of its own with the highest priority.

With `automatic` on it watches `watch_group` and frames an enemy that is off screen and after the player.

Add it in Godot: A child of the `MDSRoomCamera` (Create game scene adds it; `room_camera.get_director()` makes one).

Use it in your game: Borrows the camera for a moment: `frame_threat()` (keep a threat and the player in view), `reveal()` (show an area), `punch_in()`, `focus_on()` (a death, a finishing blow, with slow motion), `frame_world()` (the death overview). Boss arenas use it on their own; turn on `automatic` to frame chasing enemies off screen.

Inspector properties:

Property	Type	Default	What it does
`room_camera`	MDSRoomCamera		The camera it borrows. Empty: its parent.
`min_zoom`	float (range)	`0.6`	The farthest a pull-out goes, as a share of the room camera's zoom (0.6: the view grows to at most 1/0.6 of its size). Past it the player is too small to read.
`glide`	float	`5.0`	How quickly zoom and position close in on their target (per second, real time).
`player_margin`		`Vector2(140, 170)`	Space kept around the player and around a threat when framing both.
`threat_margin`		`Vector2(110, 130)`	*(not documented)*
`min_hold`	float	`1.0`	Nothing is framed for less than this (seconds), so the view doesn't flicker.
`max_release_time`	float	`1.5`	The camera is handed back after this long gliding back, even if the glide never arrives.
Automatic			
`automatic`	bool	`false`	Watch `watch_group` and frame an enemy off screen that is after the player.
`watch_group`	StringName	`&"enemy"`	Group of nodes `automatic` watches.
`engage_range`	float	`1500.0`	Enemies farther than this from the player are ignored.
`watch_interval`	float	`0.25`	Seconds between looks.

Property	Type	Default	What it does
`approach_step`	float	`18.0`	Closing in by this much (px) between two looks counts as chasing.
`quiet_states`	PackedStringArray	`["IDLE", "PATROL", "PASSIVE", "INTRO", "INACTIVE", "DEAD", "WANDER", "REST", "SLEEP"]`	An enemy whose `state` or `current_state` enum is named with one of these is minding its own business; any other state (chase, attack...) is after the player.

Signals (Node dock > Signals, to connect): `started(kind: int)`, `released`

92. `MDSMusic`

Is a: `MDSMusic` → `Node` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/mds_music.gd`

Exploration music per area for an `MDSWorldGame`: each area's `music` (a stream path in the world file) loops at its `music_volume_db`, and changing area crossfades to the next one. Walking between rooms of one area never restarts it, and an area with the same music keeps it playing.

Boss fights override it: `play_boss` starts the fight's music (the area's `boss_music` by default) from silence, and is safe to call twice; `end_boss` crossfades back to the area.

Add it in Godot: A direct child of the `MDSWorldGame`: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSMusic`. Create game scene adds it.

Use it in your game: Exploration music per area (each area's `music` on the map) with cross-fades, and boss music (`play_boss()`, `end_boss()`), called by boss arenas.

Inspector properties:

Property	Type	Default	What it does
`game`	MDSWorldGame		The game whose areas set the music. Empty: `MDSWorldGame.instance`.
`bus`	StringName	`&"Master"`	Audio bus of the music.
`crossfade_time`	float	`1.5`	Seconds of crossfade between areas.
`boss_fade_in`	float	`1.0`	Seconds a boss's music takes to rise from silence.

93. `MDSLevelOverview`

Is a: `MDSLevelOverview` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/mds_level_overview.gd`

A pull-back over the level: pictures of the rooms around the live one (`MDSRoomPictures`), each in its real place on the world map, while the camera pulls back until all of them are on screen. For a death screen (Lost in the Sky's fall), a level intro or a map preview.

Put it in an `MDSWorldGame` scene (a child of the game). `open` shows it and `close` takes it away and gives the camera back. The camera moves through the room camera's `MDSCameraDirector` (`MDSCameraDirector.frame_world`), or the game's Camera2D when there is no room camera.

Pictures already made (an `MDSRoomPictures` node makes them while the game is played) are shown at once; a room never pictured is drawn after the pull-back has started.

Add it in Godot: A direct child of the `MDSWorldGame`: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSLevelOverview`.

Use it in your game: A pull-back over the rooms around the player, for a death screen or a level intro. As a child of the game, with `MDSGameplay.death_overview_time` > 0, it plays when the player dies.

Inspector properties:

Property	Type	Default	What it does
`game`	MDSWorldGame		The game whose rooms are shown. Empty: `MDSWorldGame.instance`.
`scope`	MDSRoomPictures.Scope	`MDSRoomPictures.Scope.AREA`	Which rooms: the current area's, the current layer's or the whole world's.
`pull_time`	float	`2.4`	Seconds the pull-back takes (eased in and out).
`fit`	float (range)	`0.94`	Share of the screen the level fills at the end of the pull-back.
`margin`	float	`0.03`	Space around the level, as a share of its size.
`skip_after`	float	`1.0`	`show_for` can be skipped with any key or button after this many seconds.

Signals (Node dock > Signals, to connect): `opened`, `closed`

94. `MDSRoomPictures`

Is a: `MDSRoomPictures` → `Node` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/mds_room_pictures.gd`

Pictures of rooms, made before they are needed: an `MDSLevelOverview` (a death screen that pulls back over the level, a map preview) shows pictures of the rooms around the live one, and building a dozen rooms at that moment would stall the game.

The cache is static (shared by every node, kept for the session) and on disk in `CACHE_DIR`, named after the room's scene path and the time it was last saved: an edited room is drawn again, an unchanged one never is. The editor's World map reads the same cache for its scene previews.

As a node in an `MDSWorldGame` scene it makes the pictures of the rooms around the player in the background: first the ones already on disk (read off the main thread), then, one room every `bake_gap` seconds, the ones never drawn (the scene loaded off the main thread first). A room drawn for a picture is a copy made only to be looked at (`strip_for_preview`): no players, enemies, cameras, sounds or UI, nothing running.

Add it in Godot: A direct child of the `MDSWorldGame`: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSRoomPictures`. Create game scene adds it.

Inspector properties:

Property	Type	Default	What it does
`game`	MDSWorldGame		The game whose rooms are pictured. Empty: `MDSWorldGame.instance`.
`scope`	Scope	`Scope.AREA`	Which rooms: the current area's, the current layer's or the whole world's.
`start_delay`	float	`0.5`	Seconds after a room loads before any work starts.
`bake_gap`	float	`1.0`	Seconds between two rooms drawn (each draw costs a frame or two).

Signals (Node dock > Signals, to connect): `bake_started(path: String)`, `picture_ready(path: String, texture: Texture2D)`, `all_ready`

95. `MDSRoomDresser`

Is a: `MDSRoomDresser` → `Node` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/mds_room_dresser.gd`

Fits props to the floor and declutters rooms as they load (`MDSRoomDressing`), for rooms that are generated or dressed at runtime. Rooms dressed in the editor don't need it: use the Room view's Fit props to floor and Declutter, which are saved.

As a child of an `MDSWorldGame` it dresses every room the game loads; anywhere else, the room it is in (its owner, else its parent) when it enters the tree.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSRoomDresser`.

Inspector properties:

Property	Type	Default	What it does
`fit_to_floor`	bool	`true`	Stand the objects that stand on the floor.
`declutter`	bool	`true`	Separate the objects that overlap.
`gap`	enum	`MDSRoomDressing.GAP`	Space left between separated objects.

Signals (Node dock > Signals, to connect): `dressed(room: Node, edits: Array)`

9. Environment effects

32 nodes that share one base (`MDSEnvironmentEffect`): hazards first, then weather, set dressing and backgrounds. Place them with the Room view's Effects tool, or set a room's or area's weather and background on the world map.

96. `MDSEnvironmentEffect`

Is a: `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_environment_effect.gd`

Base of the environment effects: weather (`MDSDustStorm`, `MDSRain`, `MDSSnowfall`, `MDSLightning`, `MDSFog`), hazards (`MDSSteamVent`, `MDSLava`) and scenery in motion (`MDSWaterfall`, `MDSWater`, `MDSLightShafts`, `MDSEmbers`, `MDSFireflies`, `MDSFallingLeaves`, `MDSHeatHaze`) and Silksong-style scenery (`MDSSilkThreads`, `MDSWisps`, `MDSGnatSwarm`, `MDSDrips`, `MDSHangingMoss`, `MDSSwayingGrass`, `MDSCobwebs`, `MDSVoidTendrils`, `MDSIncenseSmoke`, `MDSForgeSparks`). Each one covers a rectangle (`get_effect_rect`, sized by `size`) and draws it with a shader from `shaders/environment/`, in the editor too.

Three ways to use them:

- Add one like any node (Add Child Node, then search "MDS") and set it up in the inspector.
- In the Room view, pick the Effects tool and drag an effect onto the room; select one to move it, resize it from its corner and edit it in the inspector.
- Give an area (or a room, or the whole world) weather: `MDSWorldGame` adds it to every room of the area as the room loads, sized to the room (see `MDSEnvironment`).

Effects that push or hurt (a dust storm's wind, steam vents, lava, waterfalls) act on the nodes of `affect_groups` whose position is inside them. A CharacterBody2D is moved with `CharacterBody2D.move_and_collide` (wind) or has its `velocity` raised (a steam jet's launch), and `take_damage(amount)` is called to hurt it. A body can handle these itself by implementing `mds_environment_push(velocity, delta, effect)`, `mds_environment_launch(velocity, effect)` and `mds_environment_hurt(amount, effect)`. Bodies held still (not processing, like the player while `MDSWorldGame` changes rooms) are left alone.

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSEnvironmentEffect` under the room's `Effects` node). Its rectangle is set by `size`.

Use it in your game: Every effect below has these properties too: the rectangle it covers (`size`), how strongly it shows, whether it follows the camera, and its drawing order.

Inspector properties:

Property	Type	Default	What it does
`size`		`Vector2(1152, 648)`	Size of the area the effect covers (px), see `get_effect_rect`.
`intensity`	float (range)	`1.0`	How strong it is (0: off).
`activation`	Activation	`Activation.ALWAYS`	*(not documented)*
`fade_time`	float (range)	`1.0`	Seconds to fade in or out.
`trigger_margin`	float	`0.0`	How far (px) around the effect a body starts it, with PLAYER_INSIDE.
`fit_room`	bool	`false`	Cover the whole room. Effects of an area's or a room's weather that have it are moved and sized to the room as it loads (in the game and in the Room view's weather preview).
`seed_value`	int	`0`	Varies the pattern, so two effects side by side don't move in step.
Bodies			
`affect_groups`	PackedStringArray	`["player"]`	Nodes in these groups are pushed or hurt by the effect and reported by `body_entered_effect`.

Property	Type	Default	What it does
Editor			
`preview_in_editor`	bool	`true`	Animate it in the editor (off: it is drawn only in the game).

Signals (Node dock > Signals, to connect): `body_entered_effect(body: Node2D)`, `body_exited_effect(body: Node2D)`, `body_hurt(body: Node2D, amount: float)`

97. `MDSLava`

Is a: `MDSLava` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_lava.gd`

A pool of lava (or acid, or cursed ooze): a heaving surface with a bright skin, molten flow with crust plates drifting on it, bubbles bursting, a heat glow above and embers rising. Put the rectangle in a pit: its top edge is where the surface rests. Bodies of `affect_groups` in it are hurt every `damage_interval` seconds (connect `MDSEnvironmentEffect.body_hurt` to kill or respawn the player instead).

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSLava` under the room's `Effects` node). Its rectangle is set by `size`.

Use it in your game: A pool of lava (or acid, or ooze). With the Gameplay Kit, touching it costs `damage` health and sends the player back to safe ground. Keep `damage` low (its default, 10, kills a 5-health player).

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`liquid`	Liquid	`Liquid.LAVA`	*(not documented)*
`crust_color`		`Color(0.16, 0.04, 0.03)`	*(not documented)*
`hot_color`		`Color(0.95, 0.30, 0.06)`	*(not documented)*
`bright_color`		`Color(1.0, 0.82, 0.42)`	*(not documented)*
`glow_color`		`Color(1.0, 0.38, 0.10, 0.55)`	*(not documented)*
Motion			
`wave_height`	float (range)	`5.0`	*(not documented)*
`wave_speed`	float (range)	`1.0`	*(not documented)*
`flow_speed`	float (range)	`1.0`	*(not documented)*
`crust`	float (range)	`0.6`	How much of the surface is crusted over.
`bubbles`	float (range)	`0.5`	*(not documented)*
Above			
`glow_height`	float (range)	`90.0`	How high the heat glow and the embers reach over the surface (px).
`glow`	float (range)	`0.7`	*(not documented)*
`embers`	float (range)	`0.5`	*(not documented)*

Property	Type	Default	What it does
Harm			
`damage`	float	`10.0`	Damage per hit to a body in the pool. 0: none (just `MDSEnvironmentEffect.body_entered_effect`).
`damage_interval`	float (range)	`0.5`	*(not documented)*
`contact_margin`	float	`24.0`	How far (px) over the surface a body's position still counts as in it (a player's origin is often at its middle).

98. `MDSLavaFall`

Is a: `MDSLavaFall` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_lava_fall.gd`

Lava falls: molten rock pouring straight down over a ledge, slow and thick, with straight sides (the flow never curves). A white-hot core with straight orange streaks racing down it, dark crust plates riding down in straight lanes with glowing cracks, a bright lip where it pours over and a splash where it lands throwing molten drops, a heat glow around it, embers rising and the air wavering in the heat. `liquid` switches it to magma, acid or cursed ooze. By default it falls behind the terrain, so the rock frames it. Bodies of `MDSEnvironmentEffect.affect_groups` in it are burnt every `damage_interval` seconds (connect `MDSEnvironmentEffect.body_hurt` to kill or respawn the player instead).

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSLavaFall` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`liquid`	MDSLava.Liquid	`MDSLava.Liquid.LAVA`	*(not documented)*
`crust_color`		`Color(0.16, 0.04, 0.03)`	*(not documented)*
`hot_color`		`Color(0.95, 0.30, 0.06)`	*(not documented)*
`bright_color`		`Color(1.0, 0.82, 0.42)`	*(not documented)*
`glow_color`		`Color(1.0, 0.38, 0.10, 0.55)`	*(not documented)*
Motion			
`flow_speed`	float (range)	`170.0`	How fast it falls in its middle (px/s); its sides drag slower.
`crust`	float (range)	`0.45`	How much crust forms on it as it falls.
Around			
`glow`	float (range)	`0.8`	*(not documented)*
`glow_spread`	float (range)	`90.0`	How far the glow, the embers and the wavering air reach out to each side (px).
`splash`	float (range)	`0.7`	Molten drops thrown out where it lands.
`splash_depth`	float (range)	`60.0`	How far below its foot the splash and the glow reach (px).
`embers`	float (range)	`0.6`	*(not documented)*
`heat_haze`	float (range)	`0.5`	*(not documented)*
Harm			
`damage`	float	`15.0`	Damage per hit to a body in it. 0: none (just `MDSEnvironmentEffect.body_entered_effect`).
`damage_interval`	float (range)	`0.4`	*(not documented)*

Property	Type	Default	What it does
`push_down`	float (range)	`0.0`	How hard (px/s) the falling rock presses bodies in it down. 0: it doesn't.

99. `MDSSteamVent`

Is a: `MDSSteamVent` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_steam_vent.gd`

A vent forcing out hot steam under pressure: thin wisps curl out of it, its mouth glows as a warning, then a jet blasts out (white-hot at the vent, cooling to a dark red crest, with glitter thrown up along it), holds, and dies away. Bodies of `affect_groups` caught in the jet are thrown along it (`launch_speed`) and scalded (`damage`).

The node stands on the vent and the jet rises from it: `size` is the jet's width and length. Rotate the node to point it another way (sideways out of a wall, down from a ceiling). Rows of vents take turns with `phase_offset`.

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSSteamVent` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`cycle`	Cycle	`Cycle.REPEAT`	*(not documented)*
`rest_time`	float (range)	`2.5`	Seconds between eruptions (REPEAT).
`warning_time`	float (range)	`0.6`	Seconds the mouth glows before the jet blasts out.
`blast_time`	float (range)	`1.2`	Seconds the jet blasts at full length.
`phase_offset`	float (range)	`0.0`	Seconds before the first eruption (vents in a row can take turns).
Look			
`vent_color`		`Color(1.0, 0.95, 0.88)`	*(not documented)*
`hot_color`		`Color(1.0, 0.47, 0.36)`	*(not documented)*
`crest_color`		`Color(0.64, 0.12, 0.11)`	*(not documented)*
`sparkle_color`		`Color(1.0, 0.74, 0.84)`	*(not documented)*
`sparkles`	float (range)	`0.6`	Glitter thrown up along the jet.
`rise_speed`	float (range)	`1.6`	How fast the billows climb.
`idle_wisps`	bool	`true`	Thin wisps out of the vent between eruptions.
Force			
`launch_speed`	float (range)	`900.0`	How fast (px/s) the jet throws bodies along it. 0: it doesn't.
`damage`	float	`0.0`	Damage dealt once per eruption to each body caught in it. 0: none.
`core_width`	float (range)	`0.6`	Share of the jet's width that throws and scalds (grazing its billows is free).

Signals (Node dock > Signals, to connect): `erupted`, `calmed`

100. `MDSRockfall`

Is a: `MDSRockfall` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_rockfall.gd`

Falling rocks: stones break off at the top of the rectangle (put it under a ceiling) and drop, tumbling, to crash on the ground in a puff of dust and a spray of chips, piling up there as rubble (a heap resting on the rectangle's bottom edge: put that on the floor). Pebbles trickle down between them. With `MDSEnvironmentEffect.activation` PLAYER_INSIDE the rocks start falling as the player comes near (a cave-in); MANUAL waits for `MDSEnvironmentEffect.start`. With `damage`, bodies under it are hit every `damage_interval` seconds.

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSRockfall` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`rock_color`		`Color(0.47, 0.42, 0.37)`	*(not documented)*
`rock_dark`		`Color(0.20, 0.17, 0.15)`	*(not documented)*
`dust_color`		`Color(0.66, 0.60, 0.52, 0.75)`	*(not documented)*
`rate`	float (range)	`0.5`	How often rocks fall (0: never).
`rock_size`	float (range)	`18.0`	The size of a big rock (px).
`gravity`	float (range)	`1400.0`	How fast they fall (px/s²).
Rubble			
`rubble`	float (range)	`0.7`	How much rubble lies on the ground.
`rubble_height`	float (range)	`60.0`	How high the rubble heap rises (px).
`dust`	float (range)	`0.7`	*(not documented)*
`pebbles`	float (range)	`0.5`	*(not documented)*
Harm			
`damage`	float	`0.0`	Damage per hit to a body under the falling rocks. 0: none (scenery).
`damage_interval`	float (range)	`1.0`	*(not documented)*

101. `MDSWater`

Is a: `MDSWater` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_water.gd`

A pool or a flooded passage: a rolling surface with a bright line along it, what is behind showing through, bent and tinted deeper toward the bottom, caustic light rippling near the surface and bubbles rising. Put the rectangle's top edge where the surface rests. Bodies of `affect_groups` going in and out are reported by `MDSEnvironmentEffect.body_entered_effect` and `MDSEnvironmentEffect.body_exited_effect` (for swimming); with `drag` they are slowed in it. With `swimmable` the Gameplay Kit's player swims in it (an `MDSSwimZone` over its rectangle).

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSWater` under the room's `Effects` node). Its rectangle is set by `size`.

Use it in your game: A pool or flooded passage. Turn on `swimmable` for the kit's swim zone (the `swim` ability).

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`shallow_color`		`Color(0.25, 0.62, 0.72, 0.45)`	*(not documented)*
`deep_color`		`Color(0.04, 0.16, 0.30, 0.85)`	*(not documented)*

Property	Type	Default	What it does
`surface_color`		`Color(0.85, 0.97, 1.0, 0.9)`	*(not documented)*
`caustic_color`		`Color(0.75, 0.95, 1.0)`	*(not documented)*
Motion			
`wave_height`	float (range)	`4.0`	*(not documented)*
`wave_speed`	float (range)	`1.0`	*(not documented)*
`refraction`	float (range)	`0.5`	*(not documented)*
`caustics`	float (range)	`0.6`	*(not documented)*
`bubbles`	float (range)	`0.3`	*(not documented)*
Force			
`drag`	float (range)	`0.0`	Share of a body's speed taken away each second it is in the water (0: none).
`swimmable`	bool	`false`	The Gameplay Kit's player swims in it (`swim` ability; without it, it sinks): an `MDSSwimZone` over the water is added when the game runs. Off by default.

102. `MDSWaterfall`

Is a: `MDSWaterfall` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_waterfall.gd`

A waterfall: a sheet of water pouring straight down the rectangle, with straight sides and straight streaks racing along it (the flow never curves), white water at the lip and where it plunges, and mist billowing out at its foot. What is behind shows through, bent by the water. By default it falls behind the terrain, so the rock it pours over frames it. With `push_down`, bodies under it are pressed down.

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSWaterfall` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`flow_speed`	float (range)	`520.0`	How fast the water falls (px/s).
`water_color`		`Color(0.42, 0.70, 0.85, 0.72)`	*(not documented)*
`foam_color`		`Color(0.92, 0.97, 1.0)`	*(not documented)*
`refraction`	float (range)	`0.6`	How much the water bends what is behind it.
Mist			
`mist`	float (range)	`0.7`	*(not documented)*
`mist_color`		`Color(0.85, 0.92, 0.97, 0.6)`	*(not documented)*
`mist_spread`	float (range)	`90.0`	How far the mist spreads to each side of the foot (px).

Property	Type	Default	What it does
`mist_depth`	float (range)	`70.0`	How far below the foot the mist reaches (px).
Force			
`push_down`	float (range)	`0.0`	How hard (px/s) the falling water presses bodies under it down. 0: it doesn't.

103. `MDSHotWaterfall`

Is a: `MDSHotWaterfall` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_hot_waterfall.gd`

Hot water falls: scalding mineral water pouring straight down the rectangle, milky and pale where it runs thick, with a warm glow deep in it, straight sides and straight streaks racing along it (the flow never curves), white water at its lip and where it plunges, steam pouring off its whole length and billowing up from its foot, and the air around it wavering in the heat. By default it falls behind the terrain, so the rock it pours over frames it. Bodies of `MDSEnvironmentEffect.affect_groups` under it are scalded every `damage_interval` seconds, and pressed down with `push_down`.

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSHotWaterfall` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`flow_speed`	float (range)	`420.0`	How fast the water falls (px/s).
`water_color`		`Color(0.45, 0.80, 0.80, 0.78)`	*(not documented)*
`foam_color`		`Color(0.95, 0.98, 0.96)`	*(not documented)*
`glow_color`		`Color(1.0, 0.72, 0.45, 0.45)`	The warm light deep in the water and in the pool it plunges into.
`refraction`	float (range)	`0.5`	How much the water bends what is behind it.
Steam			
`steam`	float (range)	`0.8`	*(not documented)*
`steam_color`		`Color(0.94, 0.95, 0.96, 0.7)`	*(not documented)*
`steam_rise`	float (range)	`60.0`	How fast the steam climbs (px/s).
`steam_spread`	float (range)	`110.0`	How far the steam drifts out to each side (px).
`steam_height`	float (range)	`140.0`	How far the steam climbs over the lip (px).
`mist_depth`	float (range)	`90.0`	How far below the foot the steam billows (px).
`shimmer`	float (range)	`0.6`	The air wavering in the heat beside the sheet.
Force			
`push_down`	float (range)	`0.0`	How hard (px/s) the falling water presses bodies under it down. 0: it doesn't.
Harm			

Property	Type	Default	What it does
`damage`	float	`4.0`	Damage per hit to a body under it: it scalds. 0: none.
`damage_interval`	float (range)	`0.8`	*(not documented)*

104. `MDSRain`

Is a: `MDSRain` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_rain.gd`

Rain: streaks at three depths falling at an angle, splashes along the bottom of the rectangle (put it on the ground) and a low mist. A far plane falls behind the terrain. Pair it with `MDSLightning` for a storm (the "storm" weather preset does).

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSRain` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`angle`	float (range)	`12.0`	Lean of the rain (degrees): positive falls toward the right.
`fall_speed`	float (range)	`1100.0`	How fast the drops fall (px/s).
`density`	float (range)	`0.55`	*(not documented)*
`drop_length`	float (range)	`34.0`	*(not documented)*
`drop_width`	float (range)	`1.4`	*(not documented)*
`color`		`Color(0.72, 0.80, 0.90, 0.55)`	*(not documented)*
`splashes`	float (range)	`0.6`	Splashes along the bottom edge.
`mist`	float (range)	`0.15`	A mist hanging low.
`mist_color`		`Color(0.45, 0.52, 0.60)`	*(not documented)*
Depth			
`far_plane`	bool	`true`	*(not documented)*
`far_z`	int	`-5`	*(not documented)*

105. `MDSSnowfall`

Is a: `MDSSnowfall` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_snowfall.gd`

Snowfall: soft flakes at four depths drifting down, swaying, carried by the wind. Turn up `blizzard` for driven snow streaking across. A far plane falls behind the terrain.

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSSnowfall` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`fall_speed`	float (range)	`70.0`	How fast the flakes fall (px/s).
`wind`	float (range)	`25.0`	Sideways drift (px/s): positive toward the right.
`sway`	float (range)	`0.5`	*(not documented)*
`flake_size`	float (range)	`3.0`	*(not documented)*
`density`	float (range)	`0.5`	*(not documented)*

Property	Type	Default	What it does
`color`		`Color(0.96, 0.97, 1.0, 0.9)`	*(not documented)*
`blizzard`	float (range)	`0.0`	Driven snow: faster, slanted flakes and streaks racing along.
Depth			
`far_plane`	bool	`true`	*(not documented)*
`far_z`	int	`-5`	*(not documented)*
Wind			
`push`	float (range)	`0.0`	With a blizzard, how hard (px/s) the wind shoves bodies along it. 0: never.

106. `MDSFog`

Is a: `MDSFog` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_fog.gd`

Drifting fog or mist: warped banks rolling slowly sideways, over the whole rectangle or hugging its bottom (`ground`: marsh mist, cave floor haze). Drawn in front of the room by default; set z_index below 0 to put it behind the terrain.

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSFog` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`color`		`Color(0.74, 0.79, 0.84, 1.0)`	*(not documented)*
`density`	float (range)	`0.5`	*(not documented)*
`drift`	float (range)	`18.0`	Drift (px/s): positive toward the right.
`bank_size`	float (range)	`380.0`	Size of the banks (px).
`ground`	float (range)	`0.0`	0 even, 1 lying along the bottom.
`ground_height`	float (range)	`0.35`	How high ground fog reaches, as a share of the height.

107. `MDSDustStorm`

Is a: `MDSDustStorm` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_dust_storm.gd`

A dust storm blowing from one side: banks of dust billowing past in gusts, wisps racing ahead of them, dark grit and pale motes driven along (the wind-scoured wastes of a desert area). A far plane behind the terrain gives it depth. With `push`, the wind shoves the bodies of `affect_groups` downwind, harder in gusts.

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSDustStorm` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`blows_toward`	Toward	`Toward.RIGHT`	Which way the wind blows.
`wind_speed`	float (range)	`420.0`	How fast the dust travels (px/s).
`density`	float (range)	`0.6`	How thick the banks of dust are.
`bank_size`	float (range)	`520.0`	Size of the banks (px).

Property	Type	Default	What it does
`grit`	float (range)	`0.6`	Grit and motes driven along.
`gustiness`	float (range)	`0.5`	How much the wind rises and falls: 0 steady, 1 strong gusts.
`opacity`	float (range)	`0.85`	*(not documented)*
Colors			
`dust_color`		`Color(0.63, 0.63, 0.61)`	*(not documented)*
`shade_color`		`Color(0.22, 0.26, 0.30)`	*(not documented)*
`grit_color`		`Color(0.06, 0.06, 0.07)`	*(not documented)*
`mote_color`		`Color(0.93, 0.94, 0.96)`	*(not documented)*
Depth			
`far_plane`	bool	`true`	A second, fainter storm behind the terrain.
`far_z`	int	`-5`	z of the far plane (the room's terrain is at 0, its background below -5).
Wind			
`push`	float (range)	`110.0`	How hard (px/s) the wind shoves bodies downwind at full strength; gusts shove harder. 0: it only blows dust.

108. `MDSLightning`

Is a: `MDSLightning` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_lightning.gd`

Lightning: every few seconds a forked bolt cracks down through the sky (behind the terrain), the whole rectangle flashes and flickers, and the thunder follows. Strikes come at random within `interval`, or when you call `strike`.

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSLightning` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`interval`		`Vector2(3.0, 9.0)`	Seconds between strikes (random within the range). 0 and 0: only `strike`.
`flash_strength`	float (range)	`0.75`	How bright the flash is.
`flash_color`		`Color(0.80, 0.86, 1.0)`	*(not documented)*
`flash_spread`	float (range)	`0.35`	Share of the flash spread evenly over the sky, away from the bolt.
Bolt			
`bolts`	bool	`true`	Draw a bolt (off: only the flash, as from a strike out of sight).
`bolt_color`		`Color(0.85, 0.9, 1.0)`	*(not documented)*
`bolt_width`	float (range)	`3.0`	*(not documented)*
`forks`	int (range)	`3`	Forks off the main bolt.
`reach`		`Vector2(0.45, 1.0)`	How far down the bolt reaches, as shares of the height (random between them).

Property	Type	Default	What it does
`bolt_z`	int	`-6`	z of the bolt: behind the terrain by default.
Thunder			
`thunder`	AudioStream		Played after each strike (none: silent).
`thunder_delay`		`Vector2(0.3, 1.6)`	Seconds between the flash and the thunder (random between them).
`thunder_volume_db`	float	`0.0`	*(not documented)*

Signals (Node dock > Signals, to connect): `struck(at: Vector2)`

109. `MDSLightShafts`

Is a: `MDSLightShafts` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_light_shafts.gd`

Shafts of light slanting down from openings above (god rays), shimmering slowly, with dust motes drifting in them. Put the rectangle's top edge where the light comes in.

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSLightShafts` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`color`		`Color(1.0, 0.94, 0.78, 0.6)`	*(not documented)*
`angle`	float (range)	`18.0`	Slant (degrees): positive leans toward the right as the light goes down.
`spread`	float (range)	`150.0`	Average distance between shafts (px).
`softness`	float (range)	`0.5`	*(not documented)*
`reach`	float (range)	`0.8`	How far down they reach, as a share of the height.
`shimmer`	float (range)	`0.5`	*(not documented)*
`motes`	float (range)	`0.4`	Dust motes in the light.

110. `MDSFallingLeaves`

Is a: `MDSFallingLeaves` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_falling_leaves.gd`

Leaves or petals drifting down: each tumbles as it falls, showing its paler back, swaying on the breeze. Each leaf takes one of three colors. A far plane falls behind the terrain.

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSFallingLeaves` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`color_a`		`Color(0.85, 0.45, 0.14)`	*(not documented)*
`color_b`		`Color(0.72, 0.20, 0.10)`	*(not documented)*
`color_c`		`Color(0.90, 0.70, 0.22)`	*(not documented)*
`fall_speed`	float (range)	`55.0`	*(not documented)*
`wind`	float (range)	`30.0`	Sideways drift (px/s): positive toward the right.
`density`	float (range)	`0.35`	*(not documented)*

Property	Type	Default	What it does
`leaf_size`	float (range)	`9.0`	Leaf length (px).
`spin`	float (range)	`1.0`	*(not documented)*
`sway`	float (range)	`0.6`	*(not documented)*
Depth			
`far_plane`	bool	`true`	*(not documented)*
`far_z`	int	`-5`	*(not documented)*

111. `MDSFireflies`

Is a: `MDSFireflies` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_fireflies.gd`

Fireflies, glowing spores or glitter: motes wandering on slow loops, pulsing on and off. Pick two colors (each mote takes one). A far plane drifts behind the terrain.

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSFireflies` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`color`		`Color(0.85, 1.0, 0.45)`	*(not documented)*
`color_b`		`Color(0.55, 1.0, 0.75)`	*(not documented)*
`density`	float (range)	`0.35`	*(not documented)*
`spacing`	float (range)	`90.0`	Average distance between motes (px).
`mote_size`	float (range)	`2.0`	*(not documented)*
`glow_radius`	float (range)	`9.0`	Radius of each one's glow (px).
`speed`	float (range)	`1.0`	How fast they wander.
`blink`	float (range)	`0.7`	0 always lit, 1 they blink on and off.
Depth			
`far_plane`	bool	`true`	*(not documented)*
`far_z`	int	`-5`	*(not documented)*

112. `MDSWisps`

Is a: `MDSWisps` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_wisps.gd`

Will-o'-wisps, like the Wisp Thicket's in Silksong: ghostly flames drifting on slow, looping paths, a flickering tongue of fire trailing behind each one and licking upward, a halo lighting the air around it. Blue by default; make them orange for a haunted forge.

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSWisps` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`color`		`Color(0.35, 0.72, 1.0)`	The flame.
`core_color`		`Color(0.88, 1.0, 1.0)`	The heart of it.

Property	Type	Default	What it does
`density`	float (range)	`0.6`	*(not documented)*
`spacing`	float (range)	`220.0`	Average distance between wisps (px).
`wisp_size`	float (range)	`8.0`	*(not documented)*
`glow_radius`	float (range)	`44.0`	Radius of the light around each one (px).
`speed`	float (range)	`1.0`	*(not documented)*
`trail`	float (range)	`1.0`	How long the flame trails behind.
`flicker`	float (range)	`0.6`	*(not documented)*

113. `MDSEmbers`

Is a: `MDSEmbers` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_embers.gd`

Embers rising from fires below, wobbling and flickering out as they cool (a volcano, a burning village, a forge), or ash and soot drifting down. A far plane rises behind the terrain.

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSEmbers` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`kind`	Kind	`Kind.EMBERS`	*(not documented)*
`hot_color`		`Color(1.0, 0.80, 0.36)`	*(not documented)*
`cool_color`		`Color(0.92, 0.22, 0.06)`	*(not documented)*
`rise_speed`	float (range)	`70.0`	How fast they rise (px/s); negative: they fall.
`drift`	float (range)	`15.0`	Sideways drift (px/s).
`density`	float (range)	`0.3`	*(not documented)*
`mote_size`	float (range)	`2.2`	*(not documented)*
`wobble`	float (range)	`18.0`	*(not documented)*
`glow`	float (range)	`0.6`	A soft halo around each one.
`flicker`	float (range)	`0.7`	*(not documented)*
Depth			
`far_plane`	bool	`true`	*(not documented)*
`far_z`	int	`-5`	*(not documented)*

114. `MDSGnatSwarm`

Is a: `MDSGnatSwarm` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_gnat_swarm.gd`

A gnat swarm, like Bilewater's in Silksong: clouds of tiny gnats buzzing around swarm centres that drift about, wings catching the light. A swarm near a node of `MDSEnvironmentEffect.affect_groups` (within `lure_radius` px) is drawn to it and gathers around its head; `lure` 0 leaves them be.

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSGnatSwarm` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`color`		`Color(0.08, 0.08, 0.05, 0.9)`	*(not documented)*
`glint_color`		`Color(0.78, 0.82, 0.62, 0.6)`	Their wings catching the light (its alpha: how much).
`swarm_density`	float (range)	`0.5`	How many of the places a swarm could be have one.
`swarm_spacing`	float (range)	`320.0`	Average distance between swarms (px).
`swarm_size`	float (range)	`55.0`	How far a swarm spreads (px).
`gnats`	int (range)	`22`	Gnats in a swarm.
`gnat_size`	float (range)	`1.4`	*(not documented)*
Motion			
`buzz`	float (range)	`1.0`	How fast they buzz about.
`drift`	float (range)	`1.0`	How fast the swarms drift.
`lure`	float (range)	`0.6`	How much a swarm is drawn to a body near it.
`lure_radius`	float (range)	`260.0`	*(not documented)*

115. `MDSHeatHaze`

Is a: `MDSHeatHaze` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_heat_haze.gd`

Heat haze: everything drawn behind the rectangle shimmers, rippling upward, strongest low down. Put it over lava, a forge or a desert floor. It bends what has a lower z_index than its own (28 by default: the whole room).

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSHeatHaze` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`strength`	float (range)	`4.0`	How far things are bent (px).
`rise_speed`	float (range)	`40.0`	How fast the ripples rise (px/s).
`ripple_size`	float (range)	`60.0`	Size of the ripples (px).
`rise`	float (range)	`0.6`	0 even, 1 strongest along the bottom and gone at the top.
`tint`		`Color(1.0, 0.62, 0.40, 0.0)`	A cast over the haze (alpha: how much).

116. `MDSIncenseSmoke`

Is a: `MDSIncenseSmoke` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_incense_smoke.gd`

Incense smoke, like the Citadel's choral chambers in Silksong: ribbons of smoke curling up from censers along the rectangle's bottom edge (put it on a censer or a brazier), spreading and fading as they rise, an ember glowing at each censer and flecks of golden ash glinting in it.

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSIncenseSmoke` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`color`		`Color(0.86, 0.82, 0.93, 0.6)`	*(not documented)*
`ember_color`		`Color(1.0, 0.74, 0.36)`	The censers' embers and the ash in the smoke.

Property	Type	Default	What it does
`plumes`	int (range)	`1`	How many censers, spread along the bottom edge.
`rise_speed`	float (range)	`40.0`	*(not documented)*
`spread`	float (range)	`0.6`	How wide the smoke fans out as it rises.
`curl`	float (range)	`1.0`	How much the ribbons curl.
`thickness`	float (range)	`0.6`	*(not documented)*
`flecks`	float (range)	`0.4`	Golden ash in the smoke.
`sway`	float (range)	`1.0`	*(not documented)*

117. `MDSForgeSparks`

Is a: `MDSForgeSparks` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_forge_sparks.gd`

Forge sparks, like the Deep Docks' in Silksong: sparks spraying from a `source` (a forge, a grinding wheel, a cut chain; the middle of the top edge by default) in a fan, arcing down, bouncing off the rectangle's bottom edge (put it on the floor) and dying out, white-hot when thrown and cooling to red, a glow pulsing at the source. A steady stream, or bursts (a hammer striking) every `burst_interval` seconds. With `damage` they burn the bodies under them.

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSForgeSparks` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`color_hot`		`Color(1.0, 0.96, 0.76)`	*(not documented)*
`color`		`Color(1.0, 0.62, 0.2)`	*(not documented)*
`color_cool`		`Color(0.82, 0.18, 0.05)`	*(not documented)*
`source`		`Vector2(0.5, 0.0)`	Where they come from, a share of the rectangle ((0.5, 0): the middle of the top edge).
`count`	int (range)	`48`	How many sparks are in the air at once.
`rate`	float (range)	`1.0`	How fast the stream runs.
`angle`	float (range)	`0.0`	Which way they spray, in degrees from straight down (positive: to the right; 180: up).
`spread`	float (range)	`70.0`	How wide the fan is, in degrees.
`speed`	float (range)	`420.0`	*(not documented)*
`gravity`	float (range)	`900.0`	*(not documented)*
`bounce`	float (range)	`0.35`	How much of their speed they keep after hitting the bottom edge (0: they land and slide).
`life`	float (range)	`1.3`	How long one lives (s).
`spark_length`	float (range)	`10.0`	*(not documented)*
`glow`	float (range)	`0.6`	*(not documented)*

Property	Type	Default	What it does
Bursts			
`bursts`	bool	`false`	Bursts every `burst_interval` seconds instead of a steady stream.
`burst_interval`	float (range)	`1.6`	*(not documented)*
Harm			
`damage`	float	`0.0`	Damage per hit to a body under the sparks. 0: none (scenery).
`damage_interval`	float (range)	`0.6`	*(not documented)*

118. `MDSDrips`

Is a: `MDSDrips` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_drips.gd`

Dripping water, like the Wormways' and the Deep Docks' in Silksong: drops gathering under the rectangle's top edge (a ceiling), swelling, falling and splashing on its bottom edge (the floor) in a spreading ring and a few droplets, the ceiling glinting wet. Make it green for bile, or orange for molten drips.

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSDrips` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`color`		`Color(0.72, 0.86, 0.96, 0.75)`	*(not documented)*
`highlight`		`Color(1.0, 1.0, 1.0, 0.9)`	*(not documented)*
`density`	float (range)	`0.6`	How many of the places a drop could form drip.
`spacing`	float (range)	`70.0`	Distance between drips (px).
`drop_size`	float (range)	`3.5`	*(not documented)*
`gravity`	float (range)	`1200.0`	*(not documented)*
`interval`	float (range)	`2.5`	Average seconds a drop takes to gather.
`splash`	float (range)	`0.7`	*(not documented)*
`wet`	float (range)	`0.5`	Glints along the ceiling.

119. `MDSHangingMoss`

Is a: `MDSHangingMoss` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_hanging_moss.gd`

Hanging moss and vines, like Greymoor's and the Moss Grotto's in Silksong: leafy strands hanging from the rectangle's top edge (put it under a ceiling), swaying in a draught, a darker layer behind. They part around the nodes of `MDSEnvironmentEffect.affect_groups` that pass through them (the player), and fall back into place behind.

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSHangingMoss` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`color`		`Color(0.17, 0.27, 0.13)`	*(not documented)*
`tip_color`		`Color(0.45, 0.6, 0.27)`	Toward the tips.

Property	Type	Default	What it does
`shadow_color`		`Color(0.06, 0.09, 0.06)`	The layer behind.
`density`	float (range)	`0.7`	*(not documented)*
`strand_spacing`	float (range)	`9.0`	Distance between strands (px).
`strand_length`	float (range)	`140.0`	How far the longest hang (px).
`variation`	float (range)	`0.6`	How much their lengths vary.
`thickness`	float (range)	`5.0`	*(not documented)*
`leaves`	float (range)	`0.6`	Leafy bumps along them.
Motion			
`sway`	float (range)	`1.0`	*(not documented)*
`speed`	float (range)	`1.0`	*(not documented)*
`part_radius`	float (range)	`40.0`	How far around a body strands are pushed aside (px; 0: they don't part).

120. `MDSSwayingGrass`

Is a: `MDSSwayingGrass` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_swaying_grass.gd`

Swaying grass, like the meadows of Silksong's Far Fields and Hollow Knight's Greenpath: blades along the rectangle's bottom edge (put it on the floor) swaying in gusts of wind, a flower on some. They bend away from the nodes of `MDSEnvironmentEffect.affect_groups` walking through them (the player) and spring back behind.

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSSwayingGrass` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`color`		`Color(0.12, 0.29, 0.14)`	*(not documented)*
`tip_color`		`Color(0.46, 0.7, 0.3)`	Toward the tips.
`flower_color`		`Color(0.96, 0.86, 0.55)`	*(not documented)*
`density`	float (range)	`0.85`	*(not documented)*
`blade_spacing`	float (range)	`5.0`	Distance between blades (px).
`blade_height`	float (range)	`34.0`	*(not documented)*
`variation`	float (range)	`0.5`	How much their heights vary.
`blade_width`	float (range)	`3.0`	*(not documented)*
`flowers`	float (range)	`0.08`	A share of the blades with a flower.
Motion			
`sway`	float (range)	`1.0`	*(not documented)*
`wind`	float (range)	`0.15`	Which way the wind leans them (-1 left, 1 right).
`speed`	float (range)	`1.0`	*(not documented)*
`part_radius`	float (range)	`36.0`	How far around a body blades bend away (px).
`bend`	float (range)	`1.0`	How far they bend (0: bodies don't bend them).

121. `MDSCobwebs`

Is a: `MDSCobwebs` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_cobwebs.gd`

Cobwebs, like the Weavenest's and Shellwood's in Silksong: spider webs strung across the rectangle's corners (and a whole one in its middle if you like), spokes from the corner and sagging rings between them, some torn, dew glinting along the threads. They tremble while a node of `MDSEnvironmentEffect.affect_groups` moves through them, and with `stickiness` they slow it down.

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSCobwebs` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`color`		`Color(0.92, 0.92, 0.96, 0.55)`	*(not documented)*
`dew_color`		`Color(1.0, 1.0, 1.0, 0.95)`	*(not documented)*
Webs			
`top_left`	bool	`true`	*(not documented)*
`top_right`	bool	`true`	*(not documented)*
`bottom_left`	bool	`false`	*(not documented)*
`bottom_right`	bool	`false`	*(not documented)*
`middle`	bool	`false`	A whole web in the middle.
`web_radius`	float (range)	`150.0`	How far a corner web reaches (px).
`spokes`	int (range)	`9`	*(not documented)*
`rings`	int (range)	`9`	*(not documented)*
`sag`	float (range)	`0.35`	How much the rings sag between spokes.
`thickness`	float (range)	`1.0`	*(not documented)*
`dew`	float (range)	`0.5`	Dew drops along the threads.
`torn`	float (range)	`0.15`	Torn pieces of ring.
Bodies			
`tremble_strength`	float (range)	`0.8`	How hard they tremble when a body moves through them.
`stickiness`	float (range)	`0.0`	Share of a body's speed taken away each second it is in a web (0: none).

122. `MDSSilkThreads`

Is a: `MDSSilkThreads` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_silk_threads.gd`

Silk threads, like the Weavenest's in Silksong: strands of silk drifting and turning slowly in the air, light running along them, and threads hanging from the rectangle's top edge, swaying, a bead of silk at the end of each.

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSSilkThreads` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`color`		`Color(0.96, 0.93, 1.0, 0.85)`	*(not documented)*
`glint_color`		`Color(1.0, 0.82, 0.92)`	The light running along them.
`density`	float (range)	`0.6`	How many strands drift in the air.
`spacing`	float (range)	`150.0`	Average distance between drifting strands (px).
`strand_length`	float (range)	`120.0`	*(not documented)*
Hanging			
`hanging`	float (range)	`0.5`	How many threads hang from the top edge (0: none).
`hang_spacing`	float (range)	`70.0`	Distance between hanging threads (px).
`hang_length`	float (range)	`0.45`	How far they hang, a share of the height.
Motion			
`sway`	float (range)	`1.0`	*(not documented)*
`speed`	float (range)	`1.0`	*(not documented)*
`glow`	float (range)	`0.5`	*(not documented)*
`thickness`	float (range)	`1.0`	*(not documented)*

123. `MDSVoidTendrils`

Is a: `MDSVoidTendrils` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_void_tendrils.gd`

Void tendrils, like the Abyss's in Silksong and Hollow Knight: tendrils of black void rising from the rectangle's bottom edge (put it on the floor) out of a pool of darkness, writhing, a dark violet glow at their rims and motes of void drifting up. They lean toward the nodes of `MDSEnvironmentEffect.affect_groups` that come within `reach` px. With `damage`, a body touching them is hurt every `damage_interval` seconds.

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSVoidTendrils` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`color`		`Color(0.02, 0.01, 0.04)`	*(not documented)*
`glow_color`		`Color(0.5, 0.25, 1.0)`	Their rims and the motes.
`density`	float (range)	`0.75`	*(not documented)*
`spacing`	float (range)	`46.0`	Distance between tendrils (px).
`height`	float (range)	`160.0`	How tall the tallest rise (px).
`variation`	float (range)	`0.5`	How much their heights vary.
`thickness`	float (range)	`14.0`	*(not documented)*
Motion			
`writhe`	float (range)	`1.0`	*(not documented)*
`speed`	float (range)	`1.0`	*(not documented)*
`reach`	float (range)	`160.0`	How close (px) a body must come for them to reach for it (0: they don't).
Look			
`pool`	float (range)	`0.7`	The pool of darkness at their roots.

Property	Type	Default	What it does
`motes`	float (range)	`0.4`	*(not documented)*
Harm			
`damage`	float	`0.0`	Damage per hit to a body among them. 0: none (scenery).
`damage_interval`	float (range)	`0.8`	*(not documented)*

124. `MDSParallaxBackground`

Is a: `MDSParallaxBackground` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_parallax_background.gd`

A parallax background: layers behind the room, each moving at its own share of the camera's speed (`MDSParallaxLayer.scroll_scale`), the far ones slowest, so the distance has depth. Layers are drawn by a shader, so no art is needed (a sky, mountains, hills, forest, city, ruins, cave rock, dunes, clouds, fog, stars); a layer can also be your own picture, repeated sideways. Pick a `preset` to start from, then change its layers.

In the game it covers the whole view and follows the camera. In the editor (and the Room view) it covers its rectangle, and scrolls as you pan the view. Give an area (or a room, or the world) one in the Areas tab's Parallax field: `MDSWorldGame` adds it to every room of the area as the room loads.

`Foreground.` `foreground_layers` are drawn in front of the room and the player, moving faster than it (`MDSParallaxLayer.scroll_scale` above 1): leaves, rocks or mist passing close to the camera. Pick a `foreground` preset (foliage, cave rocks, ruins, mist) or set the layers yourself; a layer that `MDSParallaxLayer.hang`s comes down from its line (foliage along the top edge). Their `MDSParallaxLayer.horizon` is a share of the room's height (1: its bottom edge). In the editor they are see-through, so the room under them can still be painted. In a Parallax field: `parallax(preset=misty_forest, foreground=foliage)`, or `parallax(preset=none, foreground=mist)` for a foreground alone (see `with_foreground`).

It draws at z_index -100: behind the room's background tiles (-10) and everything else; its foreground layers at `FOREGROUND_Z`, over the room's foreground tiles and shapes.

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSParallaxBackground` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`preset`	Preset	`Preset.DUSK_MOUNTAINS`	*(not documented)*
`layers`	Array[MDSParallaxLayer]	`[]`	Farthest first.
`foreground`	Foreground	`Foreground.NONE`	What passes in front of the room: a `Foreground` preset (CUSTOM keeps `foreground_layers` as they are).
`foreground_layers`	Array[MDSParallaxLayer]	`[]`	Layers in front of the room, nearest last (see the class description).
`follow_camera`	bool	`true`	On: its layers scroll with the camera, each by its `MDSParallaxLayer.scroll_scale`, and in the game it covers the whole view. Off: it is a picture on its rectangle, fixed in the world: the camera moves nothing.

125. `MDSParallaxLayer`

Is a: `MDSParallaxLayer` → `Resource` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_parallax_layer.gd`

One layer of an `MDSParallaxBackground`: a sky, a silhouette drawn by a shader (mountains, hills, a forest, a city, ruins, cave rock, dunes), clouds, a fog band, stars, or your own picture repeated sideways. Layers further back move less as the camera moves (`scroll_scale`).

Add it in Godot: FileSystem dock > right-click a folder > **Create New > Resource...** > search `MDSParallaxLayer` > Save as `.tres`. Then drag the file onto the property that takes it (or click that property > **New MDSParallaxLayer**).

Inspector properties:

Property	Type	Default	What it does
`kind`	Kind	`Kind.MOUNTAINS`	*(not documented)*
`color`		`Color(0.16, 0.14, 0.25)`	The silhouette's color (the stars', the clouds' shadow; a picture is multiplied by it).
`color_top`		`Color(0.9, 0.9, 1.0)`	Snow on peaks, lit windows, the crest's rim, the clouds' light; the sky's top.
`color_bottom`		`Color(0.5, 0.4, 0.5)`	The haze at the silhouette's base; the sky's bottom.
`scroll_scale`		`Vector2(0.2, 0.1)`	How much the layer moves with the room: 0 stays on the screen (infinitely far), 1 moves with the room. Usually smaller vertically.
`horizon`	float (range)	`0.75`	Where its features stand, as a share of the background's height (0 top, 1 bottom). A foreground layer's (`MDSParallaxBackground.foreground_layers`) is a share of the room's height: 1 is the room's bottom edge, 0 its top.
`height`	float (range)	`220.0`	How tall its features rise (px).
`feature_size`	float (range)	`420.0`	How wide its features are (px).
`roughness`	float (range)	`0.5`	Jagged peaks, lit windows, star count...
`fill_below`	bool	`true`	The silhouette runs on down to the bottom of the view.
`haze`	float (range)	`0.3`	The air between the layer and the viewer: toward `color_bottom` near the base.
`alpha`	float (range)	`1.0`	*(not documented)*
`hang`	bool	`false`	Hangs from `horizon` instead of standing on it: the silhouette mirrored (a picture's top edge on it, the picture the right way up), `fill_below` filling above. For a foreground's top edge: foliage, roots, stalactites.
`autoscroll`	float (range)	`0.0`	Drift (px/s): clouds sailing by.
`seed_value`	int	`0`	*(not documented)*
Picture			
`picture`	Texture2D		Kind PICTURE: your art, its bottom edge on the horizon, repeated sideways.
`picture_scale`	float (range)	`1.0`	*(not documented)*
`repeat_x`	bool	`true`	*(not documented)*

126. `MDSAnimatedBackground`

Is a: `MDSAnimatedBackground` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_animated_background.gd`

An animated background drawn by a shader: soft, out of focus, as if far behind the room and seen through a lens. Twenty styles (`style`): bokeh lights, an aurora, a nebula, molten blobs, deep water, ember glow, storm clouds, a forest canopy, a sunset haze, a crystal cave, a frozen night, drifting spores, a starfield, a toxic swamp, your own picture blurred, and ones made for boss rooms (a void pulse, a blood moon, an arcane vortex, the infection, holy light). Picking a style gives it its colors and pace; change them after.

The other kind of background is the parallax one (`MDSParallaxBackground`). Both can be behind a room at once: this one is the farthest (z_index -110), the parallax layers are drawn over it (-100), so a parallax without a sky layer shows it between its silhouettes.

In the game it covers the whole view and follows the camera, moving a little with it (`scroll_scale`). In the editor (and the Room view) it covers its rectangle. Give an area, a room, the world or the boss rooms one in the Areas, Inspect and World settings tabs ("Animated bg" fields): `MDSWorldGame` adds it to the room as it loads (see `MDSEnvironment.background_for_room`).

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSAnimatedBackground` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`style`	Style	`Style.BOKEH`	The look: picking one gives the background its colors, pace and blur (change them after).
Colors			
`color_a`		`Color("#0b0f22")`	The deep: the bottom of the sky, the dark between the lights.
`color_b`		`Color("#241a3e")`	The top of the sky.
`color_c`		`Color("#ffbf73")`	The lights: the main glow (style PICTURE: the picture's tint).
`color_d`		`Color("#8fa6ff")`	The accent (style PICTURE: a haze over it, its alpha how much).
Motion			
`speed`	float (range)	`1.0`	How fast it moves (0: still).
`blur`	float (range)	`0.6`	How out of focus it is: 0 sharp, 1 very soft.
`feature_scale`	float (range)	`1.0`	Size of its features (2: twice as big).
`density`	float (range)	`0.5`	How many lights, motes, stars or blobs.
`brightness`	float (range)	`1.0`	*(not documented)*
`pulse`	float (range)	`0.0`	A throb like a heartbeat, for boss rooms (0: none).
`pulse_rate`	float (range)	`1.0`	Beats a second.
`vignette`	float (range)	`0.35`	Darkens the corners.
`grain`	float (range)	`0.3`	Film grain (it also hides banding in the soft gradients).
`scroll_scale`		`Vector2(0.04, 0.02)`	Following the camera, how much it moves with the room as it moves: 0 stays on the screen (infinitely far), 1 moves with the room.
`follow_camera`	bool	`true`	On: it lies on the screen, wherever the camera goes (moving with the room as `scroll_scale` says), and in the game covers the whole view. Off: it lies on its rectangle, fixed in the world: the camera moves nothing.
Picture			
`picture`	Texture2D		Style PICTURE: the image shown, blurred by `blur`.

127. `MDSMachineryBackground`

Is a: `MDSMachineryBackground` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_machinery_background.gd`

A background of giant machinery, soft and out of focus as if far behind the room: one kind of machine at a time (`machine`), several of them at different depths, turning, swinging, pumping and glowing. They move by time alone, and they are fixed in the world: each stands at its own place in the room, so the camera pans over them as over the room. Nothing the player or anyone else does moves them.

Forty kinds, drawn by two shaders:

- gothic castle machinery, for Castlevania-style castles (`GOTHIC_IDS`, shaders/environment/machinery_gothic.gdshader): giant wheels, clockwork, clock faces, pendulums, bells, chandeliers, rose windows, hanging chains, candelabra, winches, spiked rollers, hourglasses, organ pipes, banners, astrolabes, torches, windmills, swinging axes, saw blades and castle spires;
- alien and space-station machinery, for Metroid-style worlds (`SCIFI_IDS`, machinery_scifi.gdshader): turbines, reactor cores, gyro rings, pistons, energy conduits, radar dishes, holograms, ringed planets, robot arms, drills, tesla coils, exhaust fans, bio pods, specimen tanks, data streams, laser sweeps, warp portals, crystal pylons, press hammers and cargo lifts. Picking a machine gives the background its colors, numbers and layout (hanging from above, standing along the bottom, spread about, or one big one in front); change them after.

By default it stays fixed in the world: `scroll_scale` 1, so the camera pans over it as over the room. Less makes it far away, moving slower than the room (0 stays on the screen), and `follow_camera` makes it cover the whole view in the game, wherever the camera goes (in the editor and the Room view it covers its rectangle, and scrolls as you pan).

It covers its rectangle, in the game as in the editor and the Room view: given to a room (the "Animated bg" fields of an area, a room, the world or the boss rooms), the whole room. The rectangle is cut into screen-sized cells (about 1152 x 648 px, `CELL_SIZE`), each with `count` machines of its own laid out on it, so a big room has machinery all over and every screen of it looks like one. Values: `clockwork`, `turbines(count=8)`, or after a style (`storm_clouds, castle_spires`: then it draws no backdrop of its own).

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSMachineryBackground` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`machine`	Machine	`Machine.CLOCKWORK`	The kind of machine: picking one gives the background its colors, numbers and layout.
`count`	int (range)	`10`	How many on each screen-sized cell of the rectangle (up to 12).
Look			
`back_top`		`Color("#0a0705")`	*(not documented)*
`back_bottom`		`Color("#2a1c0c")`	The bottom of the backdrop, and the haze the far machines fade into.
`metal_color`		`Color("#3a2a18")`	The machines' iron, stone or steel.
`light_color`		`Color("#ffd27a")`	Their lights: flames, lit windows, energy.
`accent_color`		`Color("#8a6a34")`	Their other material (bronze, cloth, crystal) and second light.
`blur`	float (range)	`0.55`	How out of focus they are (0 sharp, 1 very soft).
`speed`	float (range)	`1.0`	How fast they move (0: still).
`brightness`	float (range)	`1.0`	*(not documented)*
`haze`	float (range)	`0.6`	How much the far ones fade into the haze.
`vignette`	float (range)	`0.4`	*(not documented)*
`grain`	float (range)	`0.25`	*(not documented)*

Property	Type	Default	What it does
`backdrop`	bool	`true`	Draw a backdrop of its own (off: only the machines, over whatever is behind, an animated background's style for one).
Camera			
`scroll_scale`	enum	`Vector2.ONE`	How much it moves with the room as the camera moves: 1 (the default) is fixed in the world, the camera panning over it as over the room; less is farther away, moving slower; 0 stays on the screen.
`follow_camera`	bool	`false`	On: it moves with the camera as `scroll_scale` says, and in the game covers the whole view, wherever the camera goes. Off (the default): it covers its rectangle, fixed in the world: the camera moves nothing.

128. `MDSCreatures`

Is a: `MDSCreatures` → `MDSEnvironmentEffect` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/nodes/environment/mds_creatures.gd`

A background of living things, made for boss rooms: one kind of creature at a time, many of them, each glowing and carrying its own light, which lights their habitat behind them as they move. Twenty kinds (`creature`): bees and wasps around their hive, lumaflies, moths and butterflies, dragonflies, a flock of small birds, crows circling, bats, firebirds, spiders hanging on silk and crawling, beetles, centipedes, snails leaving glowing trails, crabs, jellyfish, glowfish, eels, owls watching from their perches, worms coming up out of the earth.

Drawn soft and out of focus, the far ones smaller and blurrier. Being a background, nothing in it collides or hurts: the fight goes on in front of it. It watches the player (the nodes of `MDSEnvironmentEffect.affect_groups`) all the same:

- the faster the player moves, the more stirred up they get (`activity`): flocks scatter, spiders climb their threads, beetles run, snails stop, worms come up;
- a sudden jolt (a jump, a dash, a hit) startles them; `startle` does it from code;
- those near the player on screen glow brighter (`proximity_glow`);
- the game can keep them stirred with `excitement` (a boss's second phase).

Used as the background of a room, an area's boss rooms or the world's (the "Animated bg" fields take a creature id, like `bats`, or a style and a creature: `blood_moon, bats`), their light takes the area's color (`area_tint`).

Add it in Godot: Room view > **Effects** tool (or Scene dock > **Add Child Node** > search `MDSCreatures` under the room's `Effects` node). Its rectangle is set by `size`.

Inspector properties (plus those of `MDSEnvironmentEffect`):

Property	Type	Default	What it does
`creature`	Creature	`Creature.BEES`	The kind: picking one gives the background its colors, habitat, numbers and pace.
`count`	int (range)	`26`	How many (up to 32).
Look			
`glow_color`		`Color("#ffb43a")`	Their light, and the parts of them that glow.
`body_color`		`Color("#1e1408")`	Their bodies (dark: silhouettes lit by their own glow).
`habitat`	Habitat	`Habitat.HIVE`	Where they live, drawn behind them.
`back_color_a`		`Color("#170c03")`	The habitat's dark, and its features.
`back_color_b`		`Color("#6b3d0c")`	*(not documented)*
`backdrop`	float (range)	`1.0`	How much of the habitat is drawn behind them (0: only the creatures, over another background).
`creature_size`	float (range)	`0.028`	Size of one (a share of the view's height).

Property	Type	Default	What it does
`blur`	float (range)	`0.5`	How out of focus they are.
Light			
`light_radius`	float (range)	`0.14`	How far each one's light reaches (a share of the view's height).
`light_strength`	float (range)	`1.0`	*(not documented)*
`ambient`	float (range)	`0.3`	Light on the habitat with no creature near.
`area_color`		`Color(0, 0, 0, 0)`	The color of the area this background is in (`MDSWorldGame` and the Room view set it). Transparent: none.
`area_tint`	float (range)	`0.35`	How much their light takes the area's color.
Behavior			
`speed`	float (range)	`1.0`	How fast they go.
`reactivity`	float (range)	`0.8`	How much the player's moving about stirs them up (0: they ignore it).
`proximity_glow`	float (range)	`0.6`	How much brighter those near the player glow.
`excitement`	float (range)	`0.0`	Kept stirred up by the game (0..1), whatever the player does: a boss's second phase.
`scroll_scale`		`Vector2(0.04, 0.02)`	Following the camera, how much they move with the room as it moves: 0 stays on the screen (far away), 1 moves with the room.
`follow_camera`	bool	`true`	On: they live on the screen, wherever the camera goes (moving with the room as `scroll_scale` says), and in the game cover the whole view. Off: they live in the effect's rectangle, fixed in the world: the camera moves nothing.

10. The player's insides: state machine, input buffer and states

You only open these to add an ability or change how one behaves.

129. `MDSStateMachine`

Is a: `MDSStateMachine` → `Node` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/state/mds_state_machine.gd`

A finite state machine shared by the player, enemies and bosses: its `MDSState` children are the states, by name. Only the current one runs (its physics_update, update and handle_input). `transition_to` changes state when the target's `MDSState.can_enter` allows it; `force` changes it anyway (hurt, death, locks).

Add it in Godot: Part of the player or an enemy: they make it when it's missing. Add your own with **Add Child Node** > search `MDSStateMachine` to set its properties.

Use it in your game: The player's and enemies' state machine: one child per state, named after it (`Idle`, `Dash`...).

Inspector properties:

Property	Type	Default	What it does
`initial_state`	StringName	`&""`	The state to start in. Empty: the first child state.
`body`	Node		What the states drive. Empty: the machine's parent.
`self_driven`	bool	`true`	Drive the states from this node's own process callbacks. Off: the body calls `physics_step` (the player does, to order input, states and movement).

Signals (Node dock > Signals, to connect): `state_changed(from: StringName, to: StringName)`

130. `MDSState`

Is a: `MDSState` → `Node` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/state/mds_state.gd`

A state of an `MDSStateMachine`: a child of it, named after the state ("Idle", "Dash"). Override the virtuals; `body` is what the machine drives (the player, an enemy) and `machine` the machine itself. Only the current state's `physics_update`, `update` and `handle_input` run.

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSState`.

Inspector properties:

No Inspector properties of its own.

131. `MDSPlayerState`

Is a: `MDSPlayerState` → `MDSState` → `Node` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/player/mds_player_state.gd`

A state of `MDSPlayer`'s state machine. An ability state sets `required_ability`: it can only be entered while the player has that ability (`MDSPlayer.has_ability`: granted by `MDSWorldGame.grant_ability`, or lent by equipment). A game adds an ability with an `MDSAbilityData` .tres and a state script extending this one, added under the player's StateMachine; `MDSPlayer.try_air_moves` and `MDSPlayer.try_ground_moves` try every state whose `trigger` says where it starts.

The helpers (`run`, `fall`, `land_or_fall`) are the shared movement: run and air control, gravity, and the moves the player can go into from the ground or the air.

Add it in Godot: A child of the player's `StateMachine` node, named after the state.

Use it in your game: Base of a player state. To add an ability, extend it, set `required_ability` (the ability id) and `trigger` (GROUND, AIR, ANY), and add the node under the player's `StateMachine`.

Inspector properties:

Property	Type	Default	What it does
`required_ability`	String	`""`	The ability it needs (empty: none).
`trigger`	Trigger	`Trigger.NONE`	*(not documented)*
`priority`	int	`0`	The higher, the earlier the shared transitions try it.
`animation`	StringName	`&""`	Animation played on entering it (empty: the state's name in snake_case).

132. `MDSInputBuffer`

Is a: `MDSInputBuffer` → `Node` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/player/mds_input_buffer.gd`

Remembers presses for a moment, so a jump pressed just before landing still jumps, and a dash or an attack pressed during another move still happens after it. `MDSPlayer` calls `record` every physics frame, before its states run; states ask `has` and `consume`.

Add it in Godot: Part of the player or an enemy: they make it when it's missing. Add your own with **Add Child Node** > search `MDSInputBuffer` to set its properties.

Use it in your game: Remembers presses for a moment (jump buffering). Your code or tests can call `press("jump")`.

Inspector properties:

Property	Type	Default	What it does
`actions`	PackedStringArray	`["jump", "dash", "attack", "special", "grapple"]`	The actions it remembers.
`default_window`	float	`0.12`	How long (s) a press is remembered when the asker gives no time of its own.

133. `MDSPlayerVisual`

Is a: `MDSPlayerVisual` → `Node2D` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/player/mds_player_visual.gd`

The player's placeholder look: a small cloaked figure drawn in code (no art needed), facing the way it moves, squashed when crouching or sliding, stretched when dashing, leaning on walls. Swap it for an AnimatedSprite2D named "Visual" (animations named after the states: idle, run, jump, fall, dash...) and `MDSPlayer` plays those instead.

Add it in Godot: Part of the player or an enemy: they make it when it's missing. Add your own with **Add Child Node** > search `MDSPlayerVisual` to set its properties.

Inspector properties:

Property	Type	Default	What it does
`size`		`Vector2(28, 60)`	*(not documented)*
`cloak_color`		`Color(0.18, 0.2, 0.28)`	*(not documented)*
`body_color`		`Color(0.92, 0.9, 0.86)`	*(not documented)*
`trim_color`		`Color(0.85, 0.3, 0.3)`	*(not documented)*

134. `MDSGameplayDebug`

Is a: `MDSGameplayDebug` → `Node` · **File:** `addons/MetroidvaniaDeveloperSystem/gameplay/editor/mds_gameplay_debug.gd`

Runtime debug hotkeys (debug builds only, when project setting debug_keys is true): F1: overlay (state names, hitboxes, velocity) F2: grant all abilities in the world file F3: invulnerability toggle F4: noclip toggle F6: print get_save_data() F7: 25% speed toggle

Add it in Godot: In a room scene: Scene dock > **Add Child Node** (Ctrl+A) > search `MDSGameplayDebug`.

Inspector properties:

No Inspector properties of its own.

The player's states

Children of the player's `StateMachine`. Core states always work; ability states only once the ability is granted (`required_ability`). Each plays the animation of its name in snake_case (`wall_slide`) on an `AnimatedSprite2D` visual. Their shared Inspector properties come from `MDSPlayerState`: `required_ability`, `trigger` (where the shared transitions try it: NONE, GROUND, AIR, ANY), `priority` and `animation`.

State	Needs	What it does
`MDSPlayerIdleState`	always	Standing on the ground.
`MDSPlayerRunState`	always	Running on the ground.
`MDSPlayerJumpState`	always	Rising: a jump from the ground (or just after walking off it: coyote time), or in the air with the `double_jump` ability (an air jump). Releasing jump early cuts it short (`MDSMovementConfig.jump_cut_multiplier`). Entered with `{keep_velocity: true}` (a pogo bounce) it keeps the velocity it is given.
`MDSPlayerFallState`	always	In the air, going down (or anywhere in the air no other state is).
`MDSPlayerCrouchState`	always	Down held on the ground: the body is half as tall (low passages), and the player stops. Down + jump on a one-way platform drops through it. Getting up needs room overhead.
`MDSPlayerAttackState`	always	A rooted attack (the charged attack): the player stands its ground until the attack is over.
`MDSPlayerHurtState`	always	Hit: pushed back (`MDSKnockback`) and without control until the hit-stun ends.
`MDSPlayerDeadState`	always	Health ran out: down, no control, while `MDSGameplay` takes the player back to its bench.
`MDSPlayerRestState`	always	Sitting on a bench (`MDSBench`): still until the bench gets the player up.
`MDSPlayerLockedState`	always	While `MDSWorldGame` changes rooms: no input, no gravity, the velocity left as it is (the game gives it back on arrival). It ends with the room change.
`MDSPlayerWallSlideState`	`wall_jump`	With `wall_jump`: falling against a wall while pressing toward it slides down it slowly (`MDSMovementConfig.wall_slide_speed`); jump kicks off it (WallJump).
`MDSPlayerWallJumpState`	always	Kicked off a wall: up and away from it (`MDSMovementConfig.wall_jump_velocity`), steering locked for `MDSMovementConfig.wall_jump_lock` seconds.
`MDSPlayerDashState`	`dash`	With `dash`: a burst along the ground (`MDSMovementConfig.dash_speed` for `MDSMovementConfig.dash_time` s), then `MDSMovementConfig.dash_cooldown` s before the next.
`MDSPlayerAirDashState`	`air_dash`	With `air_dash`: a dash in the air, straight across (gravity waits), up to `MDSMovementConfig.air_dashes` times before landing.
`MDSPlayerSlideState`	`slide`	With `slide`: down + dash on the ground slides along it, the body half as tall, under low gaps (`MDSMovementConfig.slide_speed` for `MDSMovementConfig.slide_time` s). Under a low ceiling it ends crouching, until there is room to stand.
`MDSPlayerLedgeGrabState`	`ledge_grab`	With `ledge_grab`: falling past the top of a wall it faces, the player catches its edge and hangs there. Up or jump climbs on (LedgeClimb), down or away lets go.
`MDSPlayerLedgeClimbState`	always	Climbing up onto a ledge held by LedgeGrab: the player rises over its edge onto the top, then stands.
`MDSPlayerClimbState`	`climb`	With `climb`: on a ladder or vine (`MDSClimbable`), up or down climbs it, free of gravity (`MDSMovementConfig.climb_speed`); jump leaps off it.

State	Needs	What it does
`MDSPlayerSwimState`	`swim`	With `swim`: in water (`MDSSwimZone`) the player swims freely (`MDSMovementConfig.swim_speed`) instead of sinking; jump at the surface leaps out.
`MDSPlayerGlideState`	`glide`	With `glide`: falling with jump held, the player glides down slowly (`MDSMovementConfig.glide_fall_speed`) and steers; letting go of jump ends it. With `toggled` (the options' "Glide: press to toggle") a jump press while falling starts it and the next one ends it, so jump needn't be held.
`MDSPlayerGrappleState`	`grapple`	With `grapple`: pressing grapple hooks the nearest `MDSGrapplePoint` in reach (`MDSMovementConfig.grapple_range`) and in sight, and pulls the player to it (`MDSMovementConfig.grapple_pull_speed`); the speed carries on after.
`MDSPlayerGroundPoundState`	`ground_pound`	With `ground_pound`: down + dash in the air slams straight down (`MDSMovementConfig.ground_pound_speed`); it breaks `cracked_floor` breakables (`MDSBreakable`) it lands on and hits what is under it.

Appendix A: editor-only objects (not for your game)

These build the MetDevSys dock and the Room view inside the Godot editor. You never add them to a game.

- `MDSWorldPanel`: Non-linear mode: draw rooms freely on a world map, place scenes on it, connect them with Hollow Knight-style gates and group them into areas
- `MDSMetSysPanel`: MetSys mode: map viewer, room annotator and progression analyzer for MetSys' grid-based MapData.txt. Layout: a tool panel on the left (like MetSys' editor), the map canvas in the middle and a sidebar with Rooms / Inspect / Progress / Issues / Stats tabs.
- `MDSWorldCanvas`: Free-form world map editor for non-linear mode. Rooms are drawn and placed anywhere in world pixels (any size, any shape made of rectangles), grouped into colored areas and connected by named gates, like a Hollow Knight map
- `MDSMapCanvas`: Interactive schematic map of one MetSys layer. Rooms are drawn by their exact cell footprint (irregular rooms included) with outlines only on the room perimeter, passages as gaps, and ability gates as colored locks
- `MDSRoomView`: Room view (the "actual view" next to the map view): paint a room scene's terrain, background and decorations with brushes or shapes (rectangles, irregular blobs, curved sides), or generate a starting cave from the room's shape on the map
- `MDSRoomCanvas`: The Room view's painting surface: shows a room scene at its real size and paints its tile layers (see `MDSRoomPainter`). Left drag paints with the current brush, middle/right drag pans, the wheel zooms
- `MDSSidePanel`: The left tool panel of MetDevSys, like MetSys' editor: every button and picker stacked in foldable sections, so the map canvas gets the rest of the screen
- `MDSTilePalette`: Tile palette of the Room view: the room tileset's spritesheets, with tiles to pick for the Terrain, Background and Decor brushes
- `MDSGeometryChecker`: Runs `MDSRoomCheck` on room scenes in the background, one room per frame, for the Issues tab's Geometry category
- `MDSMovementInspectorPlugin`: Inspector plugin for `MDSMovementConfig`: previews the jump arc (height and distance computed from tunables) next to the default 1152x648 room size for scale.

Appendix B: code helpers (no Inspector, called from scripts)

Plain classes (`RefCounted`) you call from your own scripts or tools. A few useful ones: `MDSGameplaySetup` (input actions, layers, kit settings), `MDSRoomPainter` (edit a room scene from code), `MDSFreeformConverter` and `MDSFreeformDecorator` (Convert and Decorate freeform), `MDSWorldSceneTools` (create game and room scenes), `MDSAnalysis` (the Progress tab), `MDSDamageInfo` (one hit).

- `MDSAnalysis`: Metroidvania-oriented analysis of the room graph. - Progression "spheres": starting from the start room, everything reachable with the abilities you have is sphere 0; abilities found there unlock sphere 1, and so on (the same logic item randomizers use)
- `MDSAnalysisViews`: The Progress, Issues and Stats sidebar tabs, shared by the MetSys and non-linear panels. The host panel implements:
- `MDSAnnotations`: Designer annotations that MetSys' MapData.txt has no place for: room names and types, boss labels, build status, ability gates on doors, extra links (elevators, cross-layer transitions), map pins and notes. Stored as JSON next to the map file ("MapData.txt" -> "MapData.mds.json") so it can be versioned and diffed
- `MDSAreaGenerator`: Generates an area of the world map
- `MDSAreaTools`: Areas of the world map handled as one piece: their cells and outline on the paint grid, moving one (it slides up to its neighbours, never over them), and the doors (connected gate pairs) made where rooms touch: between the rooms of an area, and between two areas that come to touch (`AUTO_LINK` doors, taken out again when they part). A "group" is an area's name, or `#room_id` for a room that is in no area.
- `MDSDamageInfo`: One hit: how much, of what type, the push it gives, how long the game freezes when it lands, what dealt it, and which breakable kinds it breaks
- `MDSDrawingTracer`: Trace drawing: turns a picture of a room (a sketch, a paint-over, a level drawing made in any paint program) into the room's terrain, background and foreground
- `MDSEdgeBand`: Meshes for shader-skinned freeform shapes (`MDSFreeformStyle.fill_material`, `MDSFreeformStyle.edge_material`)
- `MDSEnvironment`: The environment effects (`MDSEnvironmentEffect`) and weather presets, and the weather of areas and rooms. Weather is written as a comma-separated list of: - effects, by id (`EFFECTS`): `rain`, `fog`, with settings in parentheses if you like: `rain(angle=-20, density=0.8)`, `dust_storm(blows_toward=left)`, `fog(color=#3a4350)`; - presets (`PRESETS`): `storm`, `sandstorm`, `blizzard`...; - scenes of effect nodes: `res://weather/my_storm.tscn` (effects in it with `MDSEnvironmentEffect.fit_room` are sized to the room)
- `MDSExporters`: Text exports of the map: JSON (tools/pipelines), Graphviz DOT (room graph) and a Markdown design document (reviews, sharing with the team).
- `MDSFreeformConverter`: Convert to freeform: turns a room blocked out with tiles, collision polygons or straight-edged freeform shapes into organic freeform terrain, keeping its layout: every doorway, step and floor stays where it was. - `Rock.` The solid tiles, static bodies and blockout shapes join into as few shapes as they make, running on past the room's edges (out of the camera's sight)
- `MDSFreeformDecorator`: Decorate freeform: scenery placed relative to a room's freeform rock, kept out of doorways and away from everything in the room. - `Hanging`: stamps (ivy, hanging moss...) hung under ceilings and ledges. - `Floor plants`: stamps (ferns, flowers, grass...) along floors. - `Structures`: background shapes from `MDSShapeGenerators` (broken columns, ruined arches, garden walls, mounds) standing on flat floors, with leaf stamps on top; and stalactite curtains under flat ceilings. - `Foreground framing`: leaf silhouettes in the room's free corners
- `MDSFullViewTools`: The Full view's own tools (see `MDSRoomView.open_full`), on `canvas` in its Full view mode
- `MDSGameplaySetup`: Project setup of the Gameplay Kit, run by the plugin when the editor starts: input actions, collision layer names and the kit's project settings (`metroidvania_developer_system/gameplay/...`)
- `MDSGameplayUi`: What the kit's screens share: the theme (`THEME_PATH`, a `Theme` a developer can swap for their own), text sizes scaled by the options, small widget builders, finding the game and the kit from a UI node, and `add_to`, which puts every kit screen under a game's UI layer (Create game scene uses it).
- `MDSGeometry`: Polygon helpers shared by the freeform tools (repair, Convert to freeform, Decorate, Fill outside shape) and the room checks
- `MDSGraph`: Mode-independent room graph consumed by `MDSAnalysis`. Built from a MetSys map (`from_metsys`) or a non-linear world (`from_world`), so progression, save distance, routes and design checks work the same in both modes.

- `MDSLegacy`: The names the plugin used before 3.1, from its first name, Interactive Dev Panel (IDP), so projects made with them keep working
- `MDSMapImageImporter`: Import map image: turns a picture of a map (a PNG exported from MDS or MetSys, a map made in another tool or drawn in a paint program, a screenshot of a game's map) into rooms on the world map, its areas and the doors between them
- `MDSMapModel`: Parsed, analysis-friendly view of a MetSys MapData.txt file. Mirrors MetSys' own loader (MapData.gd): cells are grouped into rooms by flood-filling across "no border" (-1) edges, so irregular multi-cell rooms (L, T, U shapes...) keep their exact footprint
- `MDSMapStyle`: How rooms are drawn on the non-linear map, in the editor and in-game (MDSWorldMapView). Rooms are drawn cell by cell on the world's paint grid, each cell picking the tile that matches which of its sides are room edges, tinted with the room's area color: - Built-in tilesets ("handdrawn", "blueprint", "chunky") are generated in code. - "tileset:res://path.png": your own tileset
- `MDSNotchFill`: Fill outside shape: an irregular room's scene covers its bounding box, but the cells of the box outside the room's shape on the map (its notches) belong to the neighbouring rooms
- `MDSOptions`: The player's options, kept in `path` (`user://mds_options.cfg`) and applied when the game starts (`MDSGameplay` loads them) and whenever `MDSOptionsMenu` changes one: bus volumes, key and button bindings, window mode, and accessibility: game speed, assist mode (damage taken ×1, ×0.5 or ×0), screen shake and flash intensity (0 turns them off), camera motion (MDS's Glide or Cut, never glide), text size, how fast dialogue text comes (Slow to Instant), input glyphs and toggled glide.
- `MDSPlayHere`: Editor side of "Play from here": picks the game scene that should host the room, writes a play request and runs the launcher scene, which boots that game scene in the room. No game code is needed for MetSys-style games: any scene whose root script extends MetSysGame, or simply has a `starting_map` property or a `load_room(path)` method, is found automatically
- `MDSRoomCheck`: Physics-level checks of a room's real geometry: whether the player actually gets through
- `MDSRoomDressing`: Room dressing helpers for quickly dressed or generated rooms, where props float, sink into floors or stand inside each other: - `fit_to_floor`: objects that stand (save points, shops, NPCs, see `MDSRoomObjects.stands`) are put on the floor under them, or lifted out of the ground they are buried in, then stepped along their floor out of any platform. - `declutter`: taking objects in order of importance, each one that overlaps another steps sideways along its own floor to the nearest clear spot, at least a gap away from everything; a decoration with nowhere to go is removed. - `overlaps`: the objects that stand in or behind each other (the Issues tab's "objects overlap"). Importance (`Rank`): fixtures (doors, machines, ladders, lifts, protected areas) and enemies never move, then objects that stand move first, then everything else, bigger first
- `MDSRoomObjects`: The things in a room scene that are not its terrain or scenery: save points, shops, characters, enemies, props, mechanisms
- `MDSRoomPainter`: Paints the actual contents of a room scene: its "Background", "Terrain", "Decor" and "Foreground" TileMapLayers (with Godot's terrain autotiling), its freeform shapes (`MDSFreeform`), its stamps (sprites from an `MDSStampSet`) and the environment effects in its "Effects" node (`MDSEnvironmentEffect`: rain, steam vents, lava...)
- `MDSRoomSketches`: Room outlines drawn at their actual size in the Full view (a rectangle, an ellipse, a polygon or a curvy freehand shape) before they are rooms on the map, and `generate`, which makes them rooms
- `MDSRuntime`: Runtime side of the panel's "Play from here" action. The editor writes a request file and runs the launcher scene (`nodes/mds_play_launcher.tscn`), which boots your game scene in the chosen room: it sets `starting_map` (and `custom_run`, if present) before the game's `_ready`, then places the player
- `MDSSceneScanner`: Scans room scenes for gameplay features (collectibles, bosses, save points...), ability metadata and terrain, so the map can show what is actually inside every room. Node metadata understood by the scanner (set in the Inspector, "Add Metadata"): - `mds_grants` (String or Array): abilities/items picked up here, e.g
- `MDSShapeGenerators`: Scenery outlines made from a few parameters, for freeform shapes: ruins and rock that stand on a floor (column, broken arch, garden wall, mound) or hang from a ceiling (stalactite curtain)
- `MDSTerrainShapes`: Shape brushes for the Room view: the cells of a dragged rectangle filled as a plain block, an irregular rock blob, or a mass with one curved side (floor hills, bowls, ramps, quarter-pipes, overhanging ceilings, curved walls...). A curved side is the free face of the shape; the opposite side is its flat base: "Top curved" is ground whose surface follows the curve, "Bottom curved" hangs from a ceiling, "Left curved" sticks out of a wall on the right, "Right curved" out of a wall on the left
- `MDSTilesetFactory`: Generates a starter pixel-art tileset ("mossy cave") so rooms can be painted before any art exists
- `MDSUi`: Small widget helpers shared by the MetSys and non-linear panels.
- `MDSValidator`: Checks the map for data errors and common metroidvania design problems

- `MDSWorldSceneTools`: Keeps rooms on the world map and their scenes in sync (non-linear mode). - `place_scene`: a scene dropped on the map becomes a room of the paint cells its terrain covers (`footprint`), with its gate nodes as map gates (`join_placed` connects them to the rooms it touches). - `create_scene_for_room`: a room drawn on the map becomes a new scene with bounds guides and one `MDSGate` per map gate. - `import_gates` / `write_gates`: sync gates in either direction. - `ensure_gate_nodes`: add the MDSGate nodes a scene lacks (the panel calls it whenever gates are added or connected on the map). - `fit_room_to_scene`: resize the room to the scene's actual content.
- `MDSWorldValidator`: Checks for non-linear worlds: rooms, scenes, gates and layout, plus the shared progression/design checks from `MDSValidator`.